

Large Language Model-Based Agents for Software Engineering: A Survey

JUNWEI LIU, Fudan University, China

KAIXIN WANG, Fudan University, China

YIXUAN CHEN, Fudan University, China

XIN PENG, Fudan University, China

ZHENPENG CHEN, Nanyang Technological University, Singapore

LINGMING ZHANG, University of Illinois Urbana-Champaign, USA

YILING LOU*, University of Illinois Urbana-Champaign, USA

The recent advance in Large Language Models (LLMs) has shaped a new paradigm of AI agents, *i.e.*, LLM-based agents. Compared to standalone LLMs, LLM-based agents substantially extend the versatility and expertise of LLMs by enhancing LLMs with the capabilities of perceiving and utilizing external resources and tools. To date, LLM-based agents have been applied and shown remarkable effectiveness in Software Engineering (SE). The synergy between multiple agents and human interaction brings further promise in tackling complex real-world SE problems. In this work, we present a comprehensive and systematic survey on LLM-based agents for SE. We collect 124 papers and categorize them from two perspectives, *i.e.*, the SE and agent perspectives. In addition, we discuss open challenges and future directions in this critical domain. The repository of this survey is at <https://github.com/FudanSELab/Agent4SE-Paper-List>.

CCS Concepts: • **Software and its engineering**; • **Computing methodologies** → **Machine learning**;

Additional Key Words and Phrases: Large Language Model, AI Agent, Software Engineering

1 Introduction

Large Language Models (LLMs) have achieved remarkable progress and demonstrated potential of human-like intelligence [319]. In recent years, LLMs have been widely applied in Software Engineering (SE). As shown by recent surveys [96, 116], LLMs have been adopted and shown promising performance in various software development and maintenance tasks, such as program generation [88, 152, 161, 163, 291], software testing [85, 146, 260], debugging [89, 100, 126, 131, 136, 278, 279, 324], and program improvement [79, 211, 237, 294].

AI Agents are artificial entities that can autonomously perceive and act on surrounding environments so as to achieve specific goals [275]. The concept of AI agents has been evolving for a long time (*e.g.*, early agents are constructed on symbolic logic or reinforcement learning [133, 135, 196, 224]). Recently, the remarkable progress

*Corresponding author

Authors' Contact Information: Junwei Liu, jwliu24@m.fudan.edu.cn, Fudan University, Shanghai, China; Kaixin Wang, kxwang23@m.fudan.edu.cn, Fudan University, Shanghai, China; Yixuan Chen, yixuanchen23@m.fudan.edu.cn, Fudan University, Shanghai, China; Xin Peng, pengxin@fudan.edu.cn, Fudan University, Shanghai, China; Zhenpeng Chen, zhenpeng.chen@ntu.edu.sg, Nanyang Technological University, Singapore, Singapore; Lingming Zhang, lingming@illinois.edu, University of Illinois Urbana-Champaign, Illinois, USA; Yiling Lou, yilingl@illinois.edu, University of Illinois Urbana-Champaign, Illinois, USA.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2026/3-ART

<https://doi.org/10.1145/3796507>

in LLMs has further shaped a new paradigm of AI agents, *i.e.*, LLM-based agents, which leverage LLMs as the central agent controller. Different from standalone LLMs, LLM-based agents extend the versatility and expertise of LLMs by equipping LLMs with the capabilities of perceiving and utilizing external resources and tools, which can tackle more complex real-world goals via collaboration between multiple agents or involvement of human interaction.

In this work, we present a comprehensive and systematic survey on LLM-based agents for SE. We collect 124 papers and categorize them from two perspectives, *i.e.*, both the SE and agent perspectives. Additionally, we discuss the open challenges and future directions in this domain.

From the *SE* perspective, we analyze how LLM-based agents are applied across different software development and maintenance activities, including individual tasks (*e.g.*, requirements engineering, code generation, static code checking, testing, and debugging) as well as the end-to-end procedure of software development and maintenance. From this perspective, we provide a comprehensive overview of how SE tasks are tackled by LLM-based agents.

From the *agent* perspective, we focus on the design of components in LLM-based agents for SE. Specifically, we analyze foundation LLMs and key components, including planning, memory, perception, and action, in these agents. Beyond basic agent construction, we also analyze multi-agent systems, including their agent roles, collaboration mechanisms, information flows, real-world applications, and human-agent collaboration. From this perspective, we summarize the characteristics of different components of LLM-based agents when applied to the SE domain.

In summary, this survey makes the following contributions:

- It provides a comprehensive survey of 124 papers that apply LLM-based agents to SE.
- It analyzes how existing LLM-based agents are designed and applied for software development and maintenance from both the SE and agent perspectives.
- It discusses research opportunities and future directions in this critical domain.

Survey Structure. Figure 1 summarizes the structure of this survey. Section 2 introduces background knowledge, while Section 3 presents the methodology. Section 4 and Section 5 present the relevant work from the SE perspective and the agent perspective, respectively. Finally, Section 6 discusses the potential research opportunities.

2 Background and Preliminary

In this section, we first introduce the background about the basic and advanced LLM-based agents, and then we discuss the related surveys.

2.1 Basic Framework of LLM-based Agents

LLM-based agents are typically composed of four key components: *planning*, *memory*, *perception*, and *action* [275]. The planning and memory serve as the key components of the *LLM-controlled brain*, which interacts with the environment through the perception and action components to achieve specific goals. Figure 2 illustrates the basic framework of LLM-based agents.

Planning. The planning component decomposes complex tasks into multiple sub-tasks and schedules the sub-tasks to achieve final goals. In particular, agents can (i) generate an initial plan by different reasoning strategies, or (ii) adjust a generated plan with the external feedback (*e.g.*, environmental feedback or human feedback).

Memory. The memory component records the historical thoughts, actions, and environmental observations generated during the agent execution [261, 275, 317]. Based on accumulated memory, agents can revisit and leverage previous records and experience to tackle complex tasks more effectively. The memory management

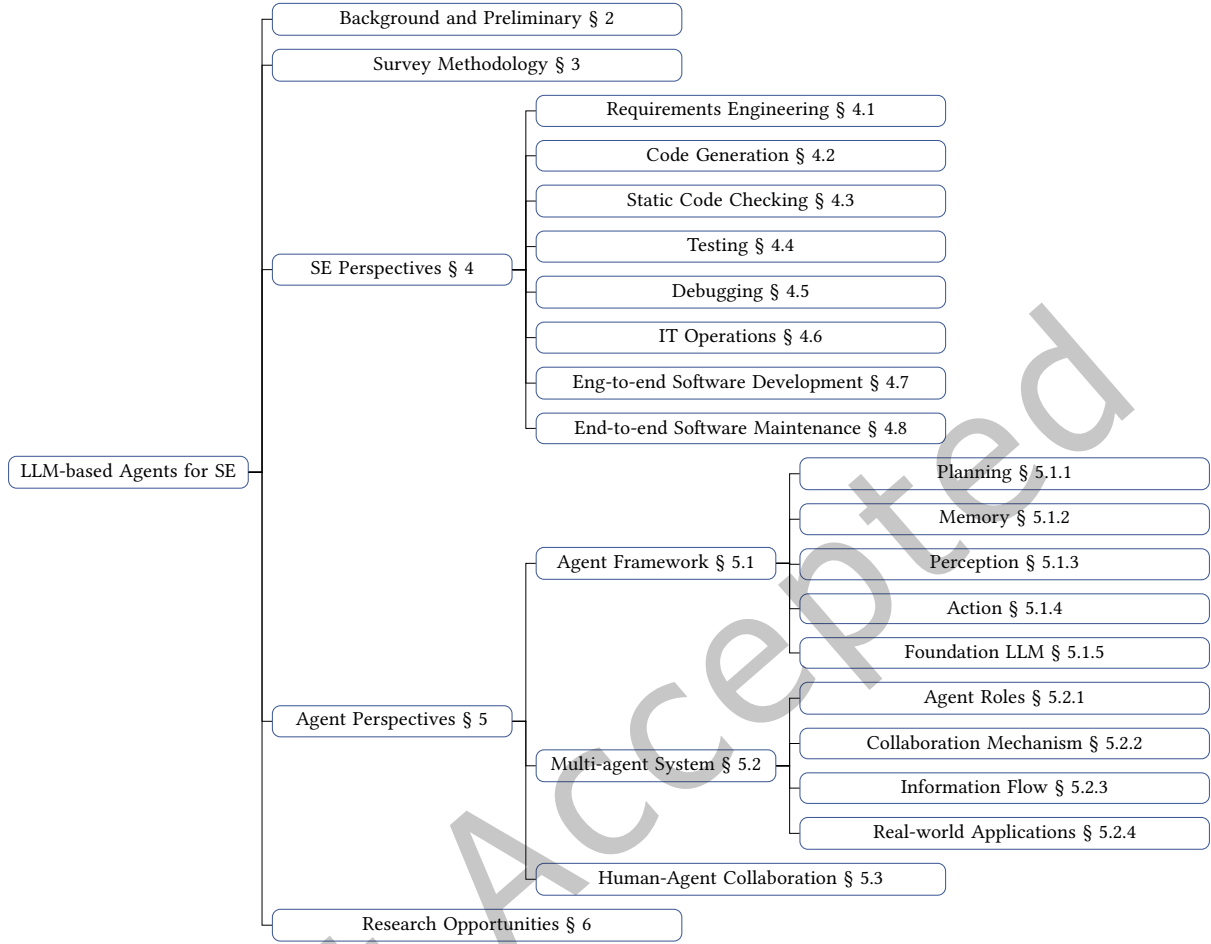


Fig. 1. Structure of This Survey

(*i.e.*, how to represent the memory) and utilization (*i.e.*, how to read/write or retrieve the memory) are essential, which directly impact the efficiency and effectiveness of the agent system.

Perception. The perception component receives the information from the environment, which can facilitate better planning. In particular, agents can perceive multi-modal inputs, *e.g.*, textual inputs, visual inputs, and auditory inputs.

Action. Based on the planning and decisions made by the brain, the action component conducts concrete actions to interact with and impact the environment. One essential mechanism in action is to control and utilize external tools, which can extend the inherent capabilities of LLMs by accessing more external resources and extending the action space beyond textual-alone interaction.

2.2 Advanced LLM-based Agent Systems

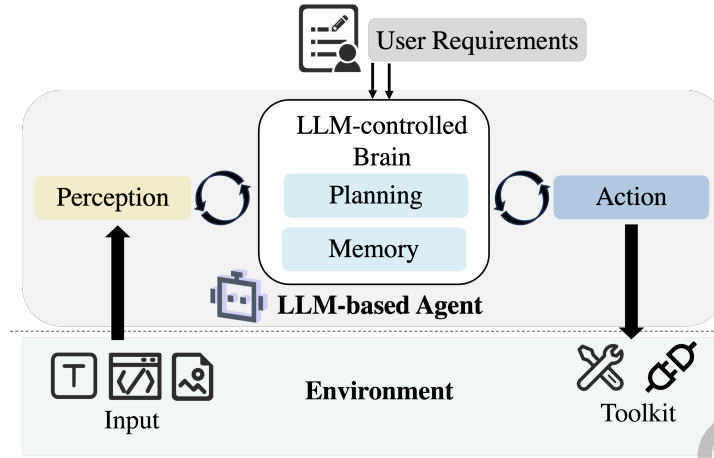


Fig. 2. Basic Framework of LLM-based Agents

Multi-agent Systems. While a single-agent system can be specialized to solve one certain task, enabling the collaboration between multiple agents (*i.e.*, *multi-agent systems*) can further solve more complex tasks associated with diverse knowledge domains. In particular, in a multi-agent system, each agent is assigned a distinct role and relevant expertise, making it specifically responsible for different tasks; in addition, the agents can communicate with each other and share progress/information as the task proceeds. Typically, agents can work collaboratively (*i.e.*, by working on different sub-tasks to achieve a final goal) or competitively (*i.e.*, by working on the same task while debating adversarially).

Human-agent Coordination. Agent systems can further incorporate the instructions from humans and then proceed with tasks under human guidance. This human-agent coordination paradigm facilitates better alignment with human preferences and uses human expertise. In particular, during human-agent interaction, humans can not only provide agents with task requirements and feedback on the current task status, but also cooperate with agents to achieve goals together.

2.3 Software Engineering

Since the 1960s, the discipline of “Software Engineering” has evolved, focusing on developing high-quality software in an efficient and cost-effective manner [198]. Generally, software engineering involves applying engineering principles throughout the entire software development and maintenance life cycle [142]. This process encompasses a variety of activities, from understanding user requirements to ensuring the software remains reliable and efficient over time. The key phases in software engineering are as follows:

- *Requirements Engineering*: Gathering, analyzing, and documenting functional and non-functional requirements to define the software’s goals and scope.
- *Software Design*: Planning the system architecture, components, and interactions to ensure scalability, maintainability, and performance.
- *Coding*: Writing the actual code based on the design specifications, following best practices like modularization, version control, and coding standards.
- *Static Checking*: Using static analysis techniques to analyze code for errors and vulnerabilities before execution. Code reviews also play a crucial role in identifying issues early by having peers manually inspect the code.

- *Software Testing*: Conducting testing activities to ensure the software meets its requirements and functions correctly. There are various types of testing, such as unit, integration, and system testing.
- *Maintenance*: Handling fault localization and repair, feature updates, and operational monitoring to ensure continued software stability, security, and alignment with evolving user needs.

These activities share some key characteristics and challenges, including but not limited to: (i) *Complexity*: Software systems typically comprise multiple modules and functionalities, requiring thorough analysis and careful design. (ii) *Adapting to changes*: Requirements evolve due to market trends, customer needs, or regulatory changes, while codebases continuously adapt and expand. (iii) *Iterating through development*: Software development follows an iterative approach, progressively refining through multiple versions. (iv) *Collaboration*: Development and maintenance require teamwork involving product managers, developers, testers, operations engineers, and other stakeholders.

To address these challenges, early-stage software engineering is predominantly human-centric, relying on methodologies and tools to enhance the productivity of individual engineers, reduce the complexity of team collaboration, and standardize software development and maintenance practices [54]. In recent years, advancements in artificial intelligence, particularly the emergence of LLM-based agents, have significantly accelerated the automation of software engineering tasks. Leveraging the aforementioned capabilities, LLM-based agents can address the challenges of software activities, including handling evolving requirements and codebases through perception and iterative mechanisms, decomposing and implementing complex modules via planning and action modules, and simulating human-team collaboration through multi-agent systems. These capabilities not only enhance their performance in single-phase tasks but also enable them to tackle complex multi-phase tasks, such as end-to-end software development and maintenance. In Section 4, we will elaborate on the specific applications of LLM-based agents across different software engineering tasks.

2.4 Related Surveys

LLM-based agents in general domains have been widely discussed and surveyed [51, 76, 107, 167, 195, 261, 275]. Different from these surveys, this survey focuses on the design and application of LLM-based agents specifically for the software engineering domain. In the software engineering domain, there have been several surveys or literature reviews on the general application of LLMs in software engineering [51, 96, 116, 125, 260]. As agents extend the capabilities of standalone LLMs with action, perception, planning, and memory, they can handle more complex and multi-turn tasks than standalone LLMs. Therefore, this survey differs from existing surveys on LLMs for SE by (i) covering wider range of SE tasks, *e.g.*, the end-to-end software development or end-to-end software maintenance; (ii) summarizing from the perspective of agent architectures, *e.g.*, building taxonomy of memory, planning, action components, multi-agent collaboration modes, and human-agent interaction modes. In addition, He *et al.* [112] present a vision paper on the potential applications and emerging challenges of multi-agent systems for software engineering. Different from the vision paper, this work focuses on conducting a comprehensive survey of existing agent systems (including both single-agent and multi-agent systems). In summary, to the best of our knowledge, this is a comprehensive survey specifically focusing on the literature on LLM-based agents for software engineering.

3 Survey Methodology

This section defines the scope of this survey and describes our approach to collecting and analyzing papers within the scope.

3.1 Survey Scope

We focus on the papers that apply *LLM-based agents* to tackle *SE tasks*. In the following, we specify the terms.

Table 1. Inclusion and Exclusion Criteria of Paper Collection

Inclusion Criteria
(1) The paper proposes a technique, framework, or tool that utilizes or enhances LLM-based agents for SE tasks.
(2) The paper presents an empirical study on the application of LLM-based agents for SE tasks.
Exclusion Criteria
(1) The agent framework is not based on LLM.
(2) The paper does not include any evaluation, or the evaluation does not involve any SE tasks.
(3) The paper only discusses LLM-based agents in the context of discussion or future work, without integrating them into the main approach.
(4) The paper merely employs a single LLM linear workflow, without any multi-agent setup or iterative interaction with the environment.
(5) The paper is less than 2 pages.
(6) The paper is a grey literature, <i>e.g.</i> , a technical report or blog post.
(7) Duplicate papers or different versions of similar studies by the same authors.

- *SE tasks.* Following previous surveys on the application of LLMs in SE [96, 116], we focus on all SE tasks along the software life cycle, including requirements engineering, software design, code generation, software quality assurance (*i.e.*, static checking and testing), and software improvement.
- *LLM-based agents.* A standalone LLM can work as a naive “agent” since it can take textual inputs and produce textual outputs, leaving it no clear boundary between LLMs and LLM-based agents. However, this could result in an overly broad scope and significant overlap with existing surveys on LLM applications in SE [96, 116]. Based on the widely-adopted consensus about AI agents, the key characteristic of agents is their ability to autonomously and iteratively perceive feedback from, and act upon, a dynamic environment [275]. To ensure a more focused discussion from the perspective of agents, this survey focuses on LLM-based agents that not only incorporate LLMs as the core of their “brains”, but also have the capacity to iteratively interact with the environment, taking feedback and acting in real time.

In addition, we position this paper as a comprehensive survey rather than a systematic literature review, with the goal of providing researchers with a quick overview of this rapidly evolving field. Therefore, we focus on the organization and synthesis of existing research on LLM-based agents in the SE domain. While we include experimental results from the collected papers to offer comparative insights and enhance the understanding of various technical approaches, conducting extra experimental analysis is beyond the scope of this survey.

3.2 Paper Collection

We apply the inclusion and exclusion criteria as shown in Table 1 for paper collection. Based on the criteria, we employ a collaborative process to inspect each paper. In particular, the first two authors independently review and label each paper to determine its relevance to the scope of this survey. When disagreements arise, a third author serves as an arbiter until consensus is reached. Our paper collection process includes three steps: keyword searching, snowballing, and author feedback collection.

3.2.1 Keyword Searching. We follow established practices in SE surveys [64, 74, 192, 303, 310] by using the DBLP database [30] for paper collection. Recent research [303] has demonstrated that papers gathered from other prominent publication databases are typically a subset of those available on DBLP, which encompasses over 7 million publications from more than 6,500 academic conferences and 1,850 journals in computer science [24]. DBLP also covers arXiv [27], a widely adopted open-access repository.

We employ an iterative trial-and-error approach, which is widely adopted in SE surveys [74, 158], to determine search keywords. Initially, all authors, with relevant research experience/publications in LLM and SE, convene to

Table 2. Statistics of Paper Collection

Keyword	Hits
agent llm language model + api	83
agent llm language model + bug	98
agent llm language model + code	915
agent llm language model + coding	70
agent llm language model + debug	95
agent llm language model + defect	22
agent llm language model + deploy	295
agent llm language model + evolution	1,349
agent llm language model + fault	685
agent llm language model + fix	318
agent llm language model + maintenance	64
agent llm language model + program	1,969
agent llm language model + refactor	15
agent llm language model + repair	137
agent llm language model + requirement	451
agent llm language model + software	2,151
agent llm language model + test	976
agent llm language model + verification	525
agent llm language model + vulnerab	144
After manual inspection	67
After snowballing	108
After author feedback collection	124

suggest papers relevant to our scope, yielding an initial set of relevant papers. Subsequently, the first two authors review the titles, abstracts, and introductions of these papers to identify an initial list of keywords, which includes “agent” AND (“code” OR “software” OR “requirement” OR “verification” OR “test” OR “debug” OR “repair” OR “maintenance”). We then conduct brainstorming sessions to expand and refine our search strings, incorporating related terms (such as “api”, “deploy”, and “evolution”), synonyms, and variations (such as “coding” in relation to “code”). This process enables the iterative enhancement of our search keyword list. For example, we observe that some studies, despite incorporating agent-based mechanisms, continue to refer to themselves as large language models. Therefore, we include “llm” and “language model” in our keyword list. With each newly added keyword, we perform an updated search and review the newly identified works to extract additional relevant keywords. If no new papers are found, we backtrack and explore alternative keywords, until we can no longer find any new papers. Through this iterative trial-and-error process, we identify the following additional keywords: “api”, “bug”, “coding”, “defect”, “deploy”, “evolution”, “fault”, “fix”, “program”, “refactor”, and “vulnerab”. The final keywords include (“agent” OR “llm” OR “language model”) AND (“api” OR “bug” OR “code” OR “coding” OR “debug” OR “defect” OR “deploy” OR “evolution” OR “fault” OR “fix” OR “maintenance” OR “program” OR “refactor” OR “repair” OR “requirement” OR “software” OR “test” OR “verification” OR “vulnerab”).

Based on the keywords, we conduct 57 searches on DBLP on July 1st, 2024, and obtain 10,362 hits. Table 2 presents the statistics of papers collected through keyword searching. The first three authors manually review

each paper to filter out those not within the scope of this survey. As a result, we identify 67 relevant papers through this process.

3.2.2 Snowballing. To enhance the comprehensiveness of our survey, we adopt snowballing approaches to identify papers that are transitively relevant and expand our paper collection [74]. Specifically, between July 1 and July 10, 2024, we conduct both backward and forward snowballing. Backward snowballing involves examining references in each collected paper to identify relevant ones within our scope, while forward snowballing uses Google Scholar to find relevant papers citing the collected ones. This iterative process continues until no new relevant papers are found. In this process, we retrieve an additional 41 papers.

3.2.3 Author Feedback Collection. To further enhance the accuracy and comprehensiveness of our survey, we reach out to the authors of the papers gathered through keyword searches and snowballing after drafting the initial version. We invite these authors to review our descriptions of their work, ensuring correctness, and to recommend additional relevant papers. In total, 321 authors were contacted via email, and we received 36 valid responses. Among them, eleven authors confirmed that our descriptions were accurate and required no changes; sixteen authors recommended 29 related papers, of which 16 papers were included in the survey after relevance filtering based on the inclusion and exclusion criteria presented in Table 1; and thirteen authors suggested revisions to the survey, including five updates on paper publication status, six suggestions for improving method descriptions, and two formatting refinements. This feedback helps ensure that the survey accurately reflects the findings and perspectives of the original research.

3.3 Statistics of Collected Papers

As shown in Table 2, we have collected a total of 124 papers for this survey. Figure 3a presents the cumulative number of papers published over time, up to September 11, 2024.¹ We observe that there is a continuous increase of research interest in this field, highlighting the necessity and relevance of this survey. Additionally, Figure 3b shows the distribution of publication venues for the papers, covering diverse research communities such as software engineering, artificial intelligence, and human-computer interaction. In particular, approximately 75% of the references are peer-reviewed publications from reputable journals and conferences, reflecting the academic rigor of our sources. The remaining citations are preprints from arXiv, which reference cutting-edge or emerging work not yet formally published. This mix provides both foundational support and timely insights, balancing scholarly reliability with the most recent developments in this field.

4 Analysis from SE Perspectives

In this section, we organize the collected papers from the perspective of different SE tasks. Figure 4 presents the SE tasks along the common life cycle of software development and maintenance.

It is worth noting that, LLM-based agents can be designed not only to tackle individual SE tasks but also to support end-to-end software development or maintenance processes involving multiple SE activities. From the collected papers, we observe LLM-based agents designed for (i) *end-to-end software development* and (ii) *end-to-end software maintenance*. Specifically, agents for end-to-end software development can generate a complete program based on requirements by performing multiple SE tasks, such as requirements engineering, design, code generation, and code quality assurance (e.g., verification, static checking, and testing); agents for end-to-end software maintenance can generate patches for user-reported issues by supporting multiple SE maintenance activities, such as debugging (e.g., fault localization and repair) and feature maintenance. As shown in previous papers [96, 116], standalone LLMs are primarily specialized in tackling single SE tasks and

¹The most recent date of the papers collected during the author feedback process.

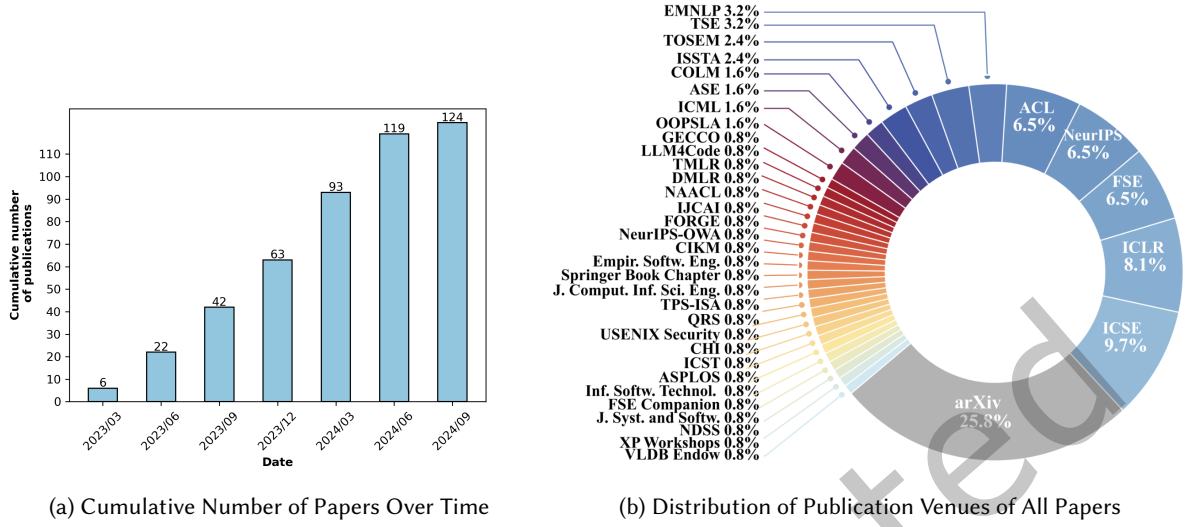


Fig. 3. Statistics of Collected Papers

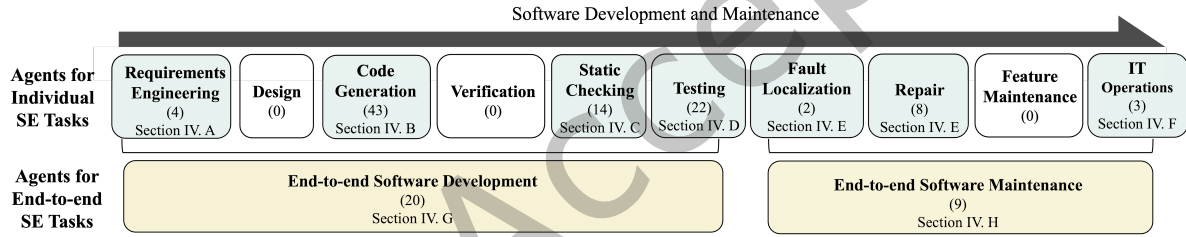


Fig. 4. Agent Distribution along Software Development and Maintenance Tasks

are generally inadequate for complex end-to-end software development and maintenance processes. In contrast, LLM-based agents, through their components (*i.e.*, planning, memory, perception, and action), coordination among multiple agents, and human interaction, provide the autonomy and flexibility necessary to tackle these complex tasks.

Distribution of LLM-based agents in different SE activities. In Figure 4, the numbers in brackets indicate the count of collected papers in each category. Notably, if LLM-based agents are designed for end-to-end software development or maintenance, they are only reported at the end-to-end level rather than at the level of individual tasks. Overall, we observe that the majority of LLM-based agents focus on individual-level SE tasks, especially for code generation and code quality assurance (*e.g.*, static checking and testing); in addition, a portion of agents are designed for end-to-end software development or maintenance tasks, indicating the promise of LLM-based agents in tackling more complex real-world SE tasks.

4.1 Requirements Engineering

Requirements Engineering (RE) is a crucial phase for initiating the software development procedure. Generally, it covers the following phases [202, 210, 236].

Table 3. Existing LLM-based Agents for Requirements Engineering

Agents	Multi-Agent	Covered RE Phases				
		Elicitation	Modeling	Negotiation	Specification	Verification Evolution
Elicitron [47]	✓	✓				
SpecGen [185]	×				✓	
Arora <i>et al.</i> [45]	✓	✓		✓	✓	✓
MARE [129]	✓	✓	✓		✓	✓

- *Elicitation*: New requirements are elicited and collected.
- *Modeling*: Abstract yet interpretable models, *e.g.*, Unified Modeling Language (UML) [56] and Entity-Relationship-Attribute (ERA) model [130], are constructed to describe the original requirements.
- *Negotiation*: Negotiation plays a crucial role in facilitating communication among different stakeholders and ensuring consistency, especially in conflicting requirements.
- *Specification*: Requirements are determined and documented in a formal format.
- *Verification*: Requirements and models are validated to ensure that they fully and unambiguously reflect the intent of stakeholders.
- *Evolution*: Requirements evolution refers to the ongoing process of refining and adapting requirements in response to changing needs and conditions.

In real-world software development, RE takes a lot of manual effort due to the demand for massive interactions with different stakeholders. To reduce the manual effort in RE, early works introduce various automation tools based on text mining [218], simple NLP techniques (*e.g.*, POS tagging and parsers) [199], and machine learning [147]. However, due to the limited natural language understanding capabilities of these methods, a prior study demonstrates that approximately 60% of these tools remain semi-automated, still requiring human intervention [253]. In recent years, the strong natural language understanding demonstrated by deep learning has opened up new possibilities for further automating requirements engineering. Researchers have adopted deep learning models (including standalone LLMs) to enhance requirements engineering activities, but most of them still work on individual RE tasks, such as classification [183], specification [141], information retrieval [302], evaluation [227], and enhancement [180] of existing requirements. In comparison, the latest agent systems are designed to automate not only individual but also multiple RE phases. Table 3 summarizes existing LLM-based agents specifically designed for RE.

Framework: Figure 5 illustrates the common framework of LLM-based agents on requirements engineering. Current studies primarily leverage the role-playing and collaboration capabilities of LLM-based agents to simulate real-world requirements engineering roles, such as users, stakeholders, requirements engineers, modelers, checkers, and documenters, with the aim of producing a complete requirements document. This cycle includes requirements elicitation, modeling, negotiation, specification, and verification. In certain specific stages, such as requirements verification, external validation tools can be integrated as action modules to provide feedback, which can be achieved through the agent’s tool usage and iterative refinement capabilities.

4.1.1 Multi-agent Collaboration Strategy. Multi-agent collaboration approaches simulate real-world software engineering teams by assigning different roles and personas to multiple agents. One example is Elicitation [47], which is a multi-agent framework designed for requirement elicitation. Within the designed context, Elicitation initializes multiple agents with different personas. These agents will simulate interactions with the target product

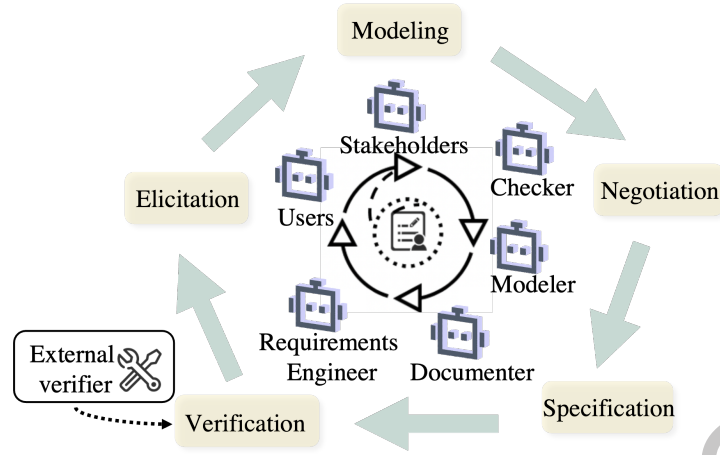


Fig. 5. Pipeline of LLM-based Agents for Requirements Engineering

from different user viewpoints and document the records (*i.e.*, actions, observations, and challenges). The latent requirements are then identified through the agent interviews and filtered by the provided criteria. Experimental results indicate that Elicitation can uncover and categorize hidden needs while reducing costs compared with conventional methodologies such as user studies.

Similarly, Arora *et al.* [45] propose a multi-agent pipeline spanning four RE phases: elicitation, specification, analysis (negotiation), and validation. Their approach employs role-playing strategies in which agents assume roles such as actual users and software architects to collaboratively negotiate requirement priorities, thereby facilitating cross-phase coordination.

Along the same lines, MARE [129] also utilizes the role-playing strategy to construct a multi-agent framework that performs a different RE pipeline, including elicitation, modeling, verification, and specification. In the elicitation phase, a set of stakeholder agents expresses their needs, which would then be organized into a draft by the collector agent. Subsequently, the modeler agent identifies entities and relationships in the draft and constructs a requirement model. In the verification phase, the checker agent assesses the quality of the current requirements draft based on its criteria and hands it over to the document agent, who will write the requirement specifications or report errors. All of these agents are equipped with predefined actions and can communicate within a shared workspace, enabling the seamless exchange of intermediate information.

4.1.2 Tool-enhanced Single-agent Strategy. In contrast, tool-enhanced single-agent approaches integrate external verification or analysis tools to iteratively improve the output quality generated by a single agent. SpecGen [185] exemplifies this approach by combining an LLM-based agent with the OpenJML verifier [77]. The agent generates Java Modeling Language (JML) specifications and refines them based on the error messages returned by OpenJML in an iterative process. Failed specifications undergo mutation and re-verification to produce a more diverse and accurate set of specifications. Experimental evidence shows that SpecGen significantly outperforms existing purely LLM-based methods and traditional specification generation tools such as Houdini [101] and Daikon [92], with improvements of 15.84%, 47.01%, and 53.76%, respectively.

4.1.3 Comparison of Multi-agent and Tool-enhanced Single-agent Strategy. In summary, multi-agent collaboration approaches enhance RE phases by simulating real-world team dynamics through role-playing and communication mechanisms. This allows coverage of individual or multiple RE phases but introduces challenges related to quality assurance in collaborative settings. On the other hand, tool-enhanced single-agent methods rely on feedback from external tools to iteratively improve the generated artifacts, yielding higher quality results at the cost of limited task scalability due to the single-agent setup.

4.1.4 Challenges of LLM-based Agents in RE. LLM-based agents offer promising support for RE, but several challenges remain. First, the generated requirements may remain vague, irrelevant, or incorrect [45, 47]. One reason is the lack of sufficient domain knowledge, which existing methods often struggle to incorporate effectively and consistently. Second, existing approaches often underemphasize human-agent interaction. RE depends on continuous communication with stakeholders, but current LLM-based agent systems simply replace human roles with LLM-based agents instead of supporting effective human-agent collaboration, reducing stakeholder involvement and trust. Lastly, as shown in Table 3, current agents lack mechanisms to support requirements evolution, limiting their usefulness in iterative and long-term development processes where change is constant.

4.2 Code Generation

Code generation has been extensively explored with the development of AI technology [108, 125]. Due to being pre-trained on massive textual data (especially large code corpus), LLMs demonstrate promising effectiveness in generating code for given code contexts or natural language descriptions. Nevertheless, the code generated by LLMs can sometimes be unsatisfactory due to issues such as the notorious hallucination [315]. Therefore, beyond simply leveraging standalone LLMs for code generation, researchers also build LLM-based agents that can enhance the capabilities of LLMs via planning and iterative refinement.

Framework: Figure 6 illustrates how existing studies extend standalone LLMs to LLM-based agents in code generation. Overall, current studies primarily leverage the capabilities of agents to plan and take actions, thus transforming one-time code generation into a “plan-generate-refine” model to improve generation correctness. During the planning phase, in addition to the commonly used natural language form, some research also generates plans that use code as an intermediate representation, such as pseudocode, intermediate code, or code skeleton. In the iterative refinement process, various forms of feedback are utilized to further enhance code generation accuracy, which include model feedback (feedback from the LLM itself), tool feedback (feedback from external tools), human feedback (clarifications from humans), and hybrid feedback (combinations of different types of feedback).

4.2.1 Code Generation with Planning. LLM-based agents employ advanced strategies to extend the code generation capabilities, which can be basically divided into prompt engineering strategy and agentic strategy.

Prompt engineering strategy. Prompt engineering strategy refers to prompting an agent to break down the code generation task into step-by-step sub-tasks and achieve higher generation correctness. Chain-of-thought (CoT) [269] is the most popular prompt engineering strategy, which has two basic types: zero-shot CoT [140] and few-shot CoT [269]. The difference between them is whether to provide completed task examples (shots) in the prompt as references. Among our collected papers, the number of studies using zero-shot CoT [113, 132, 143, 299] and few-shot CoT [118, 119, 175, 263] is roughly equal. Researchers who choose few-shot CoT often have additional formatting requirements for the output, necessitating at least one example as the format reference [119, 263]. For example, AgentCoder [119] applies CoT on code generation with four predefined steps, *i.e.*, problem understanding and clarification, algorithm and method selection, pseudocode creation, and code generation. Therefore, it provides

Table 4. Existing LLM-based Agents for Code Generation

Agents	Multi-Agent	Iterative Refinement			
		Model Feedback	Tool Feedback	Human Feedback	Hybrid Feedback
Reflexion [233], SEIDR [176], Self-Repair [203], AutoGen [274], INTERVENOR [259], TGen [193], AutoCoder [145], CodeChain [143], RRR [86]	✓	✓	✓		✓
CAMEL [149], AgentForest [151], DyLAN [175]	✓	✓			
Self-Debugging [67], μ FiX [250], AlphaCodium [225], LDB [320], LATS [321]	×	✓	✓		✓
ToolCoder [304], SelfEvolve [127], KPC [223], Lemur [284], CodeAgent [305], LLM4TDD [208], CodeCoT [118], CodeAct [262], InterCode [289], CodePlan [50], ToolGen [257]	×		✓		
Self-Refine [188]	×	✓			
Flows [132]	✓	✓	✓	✓	✓
MINT [263]	×	✓	✓	✓	
ClarifyGPT [197]	×		✓	✓	
Self-Edit [306], AgentCoder [119], Gentopia [283], AutoDev [252], SoA [121], MapCoder [122], 3DGen [94], CoCoST [113]	✓		✓		
Parsel [299], RAT [265]	✓				

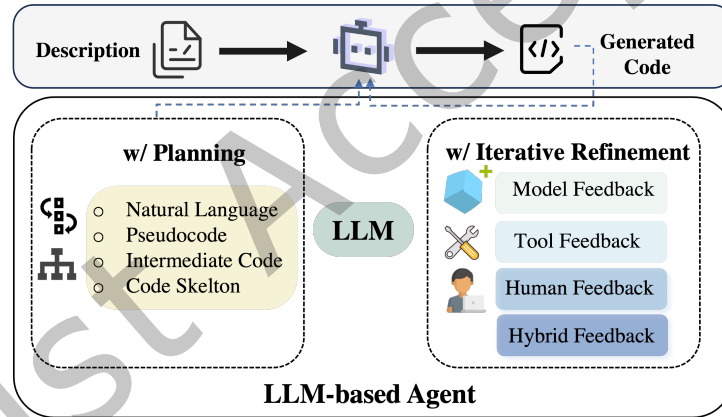


Fig. 6. Pipeline of LLM-based Agents for Code Generation

an example to guide the model in following these steps during planning. Moreover, these studies typically include merely one static pre-defined example rather than dynamically selecting examples based on relevance or similarity [232, 318]. Despite the widespread application of CoT, the effectiveness of some other advanced prompt engineering strategies, such as self-consistency [264] and least-to-most prompting [322], has not been explored in code generation tasks yet, which could be a potential direction for future research.

Agentic strategy. On the other hand, *agentic strategy* refers to instructing agents to dynamically adapt the code generation plan based on historical thoughts, actions, and observations [50, 132, 143, 233, 262, 289]. For example, CodePlan [50] employs an adaptive planning algorithm that dynamically detects the affected code snippets in the

repository and adapts the modification plan accordingly. In addition, some works explore multi-path planning strategies. For example, LATS [321] simulates all possible generation paths as a tree and optimizes the plan with the Monte Carlo Tree Search algorithm. In MapCoder [122], the planning agent generates multiple plans along with confidence scores for sorting. The highest-scoring plan is used to generate the target code. If the code is erroneous, the plan with the next highest confidence is selected to continue the iterative generation process. Almost all of these agentic planning works assign only one agent as the planner; the only exception is Flows [132], which compares the effectiveness of using a single planner versus a dual planner. However, the results demonstrate that the collaborative approach using two planners does not surpass the performance of using a single planner.

In addition to planning strategies, the planning representation also demonstrates significant diversity. Although text is still the most common form, some works propose to describe plans in several indirect code forms, such as *pseudocode* [119], *intermediate code* [299], and *code skeleton* [121, 143]. For example, AgentCoder [119] prompts the agent to generate pseudocode after problem understanding and algorithm selection phases, which serves as a draft for the final code. These code-based plans bridge the gap between the narrative-based steps and the final generated code, making them better suited for the code generation task.

Comparison of Prompt Engineering Strategy and Agentic Strategy. Overall, code generation with planning is an important approach to decomposing programming steps and improving code generation accuracy. The two mainstream strategies, prompt engineering and agentic strategies, exhibit significant differences with respect to generalizability and planning iterations. In terms of generalizability, prompt engineering strategies can be activated simply by incorporating straightforward instructions (e.g., “think step by step”) into the prompt, making them applicable to all instruction-following LLMs. In contrast, agentic strategies rely on environmental feedback and self-reflection mechanisms, making them suitable only for LLM-based agents. Regarding planning iterations, prompt engineering follows a one-time planning approach, where the plan is determined upfront and remains unchanged. In contrast, agentic strategies continuously refine the plan through iterative adjustments based on environmental feedback, allowing for more dynamic adaptability. It is also worth noting that some studies have attempted to integrate traditional CoT strategies with agentic planning strategies. For example, RAT [265] proposes an iterative CoT optimization strategy. It uses the prefix steps along with the original prompt to retrieve information from the *codeparrot/github-jupyter* [29] dataset, which is then fed back to the agent for revising the next step in CoT iteratively. This combination makes RAT achieve better code generation accuracy than the basic CoT strategy.

4.2.2 Code Generation with Iterative Refinement. One essential capability of agents is to act on the feedback from the environment. In the code generation scenario, agents also dynamically refine the previously-generated code based on the feedback via multiple iterations. We organize the relevant research based on the feedback sources, including model feedback, tool feedback, human feedback, and hybrid feedback. Table 4 summarizes existing LLM-based agents for code generation with iterative refinement. Figure 7 illustrates the four types of feedback.

Model Feedback. Model feedback can be classified into peer-reflection and self-reflection.

Peer-reflection refers to information exchange and interaction between models, *i.e.*, the feedback is provided by other agents. The most common approach to facilitating peer-reflection is through role specialization and structured communication [149, 151, 176, 233, 274], which underscores the specialized responsibilities of each role and how they exchange information based on their responsibilities. For example, in AutoGen [274], the SafeGuard agent will check the code safety and provide debugging feedback for the Writer agent. Moreover, there is a modality that treats each agent equally in expressing their opinions or engaging in debates to generate code, in which a selection mechanism is employed to retain the most suitable result. For example, AgentForest [151]

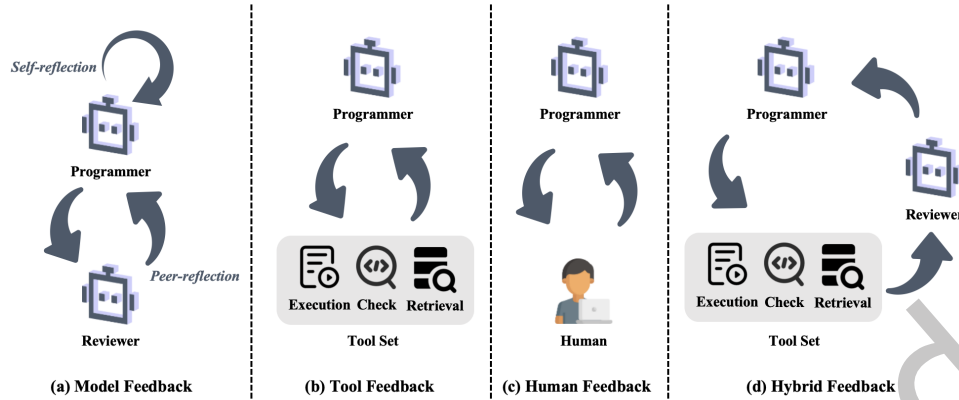


Fig. 7. Four Types of Feedback in Code Generation

and DyLAN [175] both use Bilingual Evaluation Understudy (BLEU) [205] to compute the similarity scores for code produced by each agent, aggregate these scores, and keep the top-scoring result.

Apart from the interaction between different models, some works conduct *self-reflection*, in which the model will iteratively optimize its generated code based on the previous output [67, 188, 250]. Le *et al.* [143] guide LLMs to generate modularized code, leveraging cluster representatives from previously generated sub-modules in each iteration. Self-Debugging [67] draws inspiration from the rubber duck debugging method used by programmers. During the explanation phase, the model provides a line-by-line explanation of the generated initial code. As Wang *et al.* [263] mention in their work, all models benefit from natural language feedback, with absolute performance gains by 2–17% for each additional turn of natural language feedback.

In summary, model feedback harnesses the contextual understanding and reasoning capabilities of LLMs. By mimicking the real-world iterative code-refinement process, the model can analyze generated code from either a programmer’s perspective (self-reflection) or that of other expert roles (peer-reflection). Through step-by-step code interpretation, error detection, and contextual enrichment, the programming agent progressively enhances its understanding of the target code, thus leading to more accurate code generation. The reasoning ability of agents plays a crucial role in the iterative code generation process, distinguishing them significantly from traditional approaches based on programming templates [179, 244] or simple machine/deep learning methods [41, 191].

Tool Feedback. The code generated by models can be of limited quality with numerous uncertainties. One solution to address this challenge is to equip LLM-based agents with tools that can collect informative feedback and assist the agents in generating and refining code.

- *Dynamic Execution Tools.* One common group is to invoke the compiler, interpreter, and execution engine to directly compile or execute the code. This approach leverages the outputs and run-time behaviors, such as test results or compilation errors, as feedback for code improvement [94, 113, 118, 119, 121, 122, 127, 143, 197, 208, 252, 262, 263, 283, 284, 289, 305, 306].
- *Static Checking Tools.* Agents can get more restricted knowledge of code constraints by applying code analysis tools. For example, some agents apply static analysis tools to obtain syntactically-valid program symbols/tokens [257] or dependencies between code during code generation [50, 305]. Including the analyzed information in the prompt can guide LLMs toward generating valid code.
- *Retrieval Tools.* Agents can access rich external resources by applying retrieval or searching tools. For example, some agents retrieve local knowledge repositories [113] such as private API documentations [304, 305] and code

repository [252] to facilitate better code generation; in addition, some apply online search engines [113, 283, 304, 305] or web crawling [223] to collect information such as content from relevant websites (e.g., *StackOverflow* and *datagy.io*) [113, 283, 304, 305] and official online documentations [113, 223]. Including the retrieved resources in the prompt can provide additional knowledge for language models. For example, ToolCoder [304] integrates the agent with online search and local documentation search tools that provide helpful information for both public and private APIs, alleviating the hallucination of LLMs.

In summary, tool feedback enhances code generation by integrating a wealth of mature coding tools and resources from the software engineering domain into LLM-based agents. This integration provides essential knowledge and diagnostic support, including but not limited to static analysis, runtime behavior monitoring, and performance evaluation across multiple dimensions. By leveraging these capabilities, tool feedback helps mitigate the inherent quality issues in LLM-generated code caused by hallucinations and randomness [125, 315].

Human Feedback. Another approach involves incorporating human feedback into the process, as humans play a critical role in clarifying ambiguous requirements. For instance, in software development, humans can check whether the generated code aligns with their initial intent. Discrepancies are often attributed to vagueness or incompleteness in the requirements, prompting a revision of the requirement documents [132, 263]. To minimize human involvement in detecting vagueness, some methods enable the agent to handle the task of observing execution results. For example, ClarifyGPT [197] automatically identifies potential ambiguities in the manually-given requirements and proactively poses relevant questions for humans; then the responses from humans are further used to refine the requirements.

To sum up, the remarkable natural language understanding capabilities of LLM-based agents enable direct interaction with humans. In the context of code generation, this human-agent interaction allows the agent to confirm the user intent. By incorporating human feedback at key decision-making stages, such as requirement analysis, the agent can obtain clarifications and confirmations from the user, effectively preventing deviations from the original user intent and enhancing the consistency between generated code and user expectations.

Hybrid Feedback. To leverage the advantages of different types of feedback, some studies have explored integrating multiple feedback types to provide agents with a hybrid feedback mechanism. While the three types of feedback (i.e., model feedback, tool feedback, and human feedback) could theoretically be paired in various ways, we have merely observed the combination of tool feedback and model feedback in existing studies, which might stem from efforts to minimize human involvement and enhance automation. Typically, these approaches first obtain precise error feedback of the generated code from program execution or testing tools. Then, an agent will analyze the tool feedback and provide explanations, suggestions, or new instructions accordingly [67, 86, 121, 132, 145, 176, 193, 203, 225, 233, 250, 259, 274, 320, 321]. For example, in INTERVENOR [259], a teacher agent is designated to observe the program execution results and provide error explanations and bug-fixing plans for the student coder to review and regenerate the code. Nevertheless, for programs with complex data structures and control flows, merely analyzing the final execution outputs might be sub-optimal. Therefore, LDB [320] proposes a novel hybrid feedback mechanism that collects and analyzes the intermediate execution status of the generated program. Specifically, it divides the program into different code blocks based on the control flow graph and uses a breakpoint tool to collect the runtime states of variables before and after each code block's execution. Subsequently, the agent analyzes the runtime execution information along with the task description to explain the execution flow and assess the correctness of each block. The feedback is then fed to the agent to debug and regenerate a refined program. Experimental results show that LDB achieves better performance in code generation tasks compared to methods that merely analyze the final execution output (e.g., Self-Debugging [67]), demonstrating the unique contribution of intermediate state analysis. Overall, the current hybrid feedback methods leverage the strengths of both tool feedback and model feedback to provide interpretable information based on environmental feedback, thereby significantly improving the accuracy of model-generated code.

Comparison of Iterative Feedback Mechanisms. The four types of feedback contribute to code generation from different perspectives, but each also comes with its own challenges. While model feedback provides interpretable feedback, the inherent randomness and hallucination issues [125, 315] of feedback models can lead to cascading errors. Tool feedback, on the other hand, offers multi-dimensional code evaluation information, but this information often lacks necessary explanations and contextual relevance. Human feedback helps align the agent with human intent, but it reduces the method’s autonomy and introduces additional human effort. However, these feedback mechanisms are complementary. For example, the lack of explanations and contextual relevance in tool feedback can be addressed through model feedback, while errors in model feedback can be mitigated by the precision of tool feedback. By adopting a hybrid feedback approach, the complementary strengths of different feedback mechanisms can be leveraged to maximize the advantages of all approaches. However, in the current hybrid feedback approaches, the final feedback is still provided by the agent itself. Therefore, it may remain susceptible to cascading errors, requiring more innovative designs to mitigate this issue.

4.2.3 Common Failure Causes of LLM-based Agents in Code Generation. LLM-based agents have shown strong potential in automating code generation tasks, but they still face a number of common failure cases that limit their reliability and robustness in practice. These failures span across interaction quality, testing reliability, feedback effectiveness, and context management.

Coordination Failures in Agent Collaboration. For agentic systems with multiple agents, it is common to encounter breakdowns in collaborative interactions due to poor coordination and role management. In systems like CAMEL [149], agents may repeat user instructions without contributing new information, generate vague promises like “I will fix it”, or even fall into infinite conversational loops such as repeatedly saying “thank you” or “goodbye.” These issues typically stem from the agent’s inability to maintain consistent roles and task focus. To address these failures, potential solutions include limiting dialogue turns, enforcing token usage thresholds, or introducing explicit task-completion signals to terminate unproductive conversations.

Low-Quality Tests Undermine Code Generation Accuracy. In many LLM-based code generation agents, the correctness of the generated code is assessed using automatically generated or existing test suites [67, 113, 233, 250]. However, flaky, incomplete, or poorly designed tests can mislead the self-correction process [122, 233]. False positives occur when faulty tests pass incorrect solutions, causing premature task completion, while false negatives happen when correct code fails unreliable tests, leading to unnecessary revisions. Besides, insufficient test coverage is also a common bottleneck [250], which limits the confidence in the correctness of the generated code. Enhancing the reliability of test suites remains an open research direction.

Cascading Errors from Incorrect or Noisy Feedback. Another common failure mode is the cascading effect caused by faulty feedback during iterative refinement. For LLM-based agents relying on model-generated feedback, the model may provide incorrect debugging suggestions [151, 175, 188]. For instance, in Self-Refine [188], 33% of failed cases stemmed from feedback inaccurately identifying the error location, while 61% resulted from feedback proposing inappropriate fixes. Similarly, for LLM-based agents that utilize tool feedback, misleading tool outputs can propagate errors in subsequent iterations [113, 257]. For example, CoCoST [113] depends on an online search tool to retrieve relevant information, but the retrieved content may contain inaccuracies. These findings highlight the critical importance of maintaining high-quality, well-calibrated feedback mechanisms throughout the generation process.

Degraded Long-context Reasoning Capability. As the interaction history grows, LLM-based agents struggle to retain relevant information and maintain reasoning consistency. For example, both AutoGen [274] and InterCode [289] show that large volumes of accumulated context in multi-step refinement make it harder for agents to extract useful information for future actions. This leads to degraded performance over time. Potential

solutions include increasing context window size, integrating memory management or retrieval modules, and developing adaptive planning strategies to maintain relevance across turns.

4.2.4 Challenges of LLM-based Agents in Code Generation. LLM-based agents for code generation face several key challenges. First, they heavily rely on test feedback to improve code quality, but high-quality tests are often unavailable or difficult to generate reliably, leading to concerns about test coverage and correctness [118, 119, 165, 193, 197, 203, 208, 225, 233]. Second, these agents typically require iterative refinement of the generated code, which introduces significant overhead compared to standalone LLM methods and poses challenges in terms of efficiency and cost [122, 223, 250, 321]. Third, they depend on external tools to obtain feedback from the environment, but the reliability of these tools can be problematic [50, 86, 113, 257, 265, 305]. For example, the relevance of retrieved content or the accuracy of static analysis results may not always be guaranteed, affecting the agent’s overall performance.

4.3 Static Code Checking

Static code checking refers to examining the quality of code without executing the code. In particular, static code checking has been essential in the modern continuous integration pipeline, as it can identify diverse categories of code quality issues (e.g., different bugs, vulnerabilities, or code smells) before extensively executing the tests. In practice, it is common to adopt static analysis techniques to automatically detect bugs/vulnerabilities (i.e., static bug detection) or involve peer reviews to check the quality of code (i.e., code review).

4.3.1 Static Bug Detection. Preliminary studies [96, 116] show that LLMs can help identify potential quality issues in the given code under inspection. For example, fine-tuning LLMs on existing buggy/correct code or simply prompting LLMs has demonstrated promising effectiveness in identifying bugs, vulnerabilities, or code smells in the given code snippets [100, 295]. However, given the diversity and complexity of the root causes of different code issues as well as the long code contexts under inspection, standalone LLMs exhibit limited accuracy and recall in the real-world static code checking scenario [90]. Recently, researchers have built LLM-based agents to enhance the capabilities of isolated LLMs in vulnerability detection. Table 5 summarizes these agents.

Framework: Figure 8 illustrates the common framework of LLM-based agents for static bug detection. The agent systems will detect the input buggy programs and output a bug/vulnerability report. Based on responsibilities, there might be four types of roles in this process: the *detector* is used to identify bugs in the code, the *validator* is used to confirm and filter the detected bugs, the *ranker* is used to rank the suspicious results, and different works might also involve some assistant roles (e.g., planner and reporter). These agents are equipped with a series of tools, including traditional bug detection tools such as CodeQL [49] and UBITect [301].

Co-inspection with Multi-agent. One effective vulnerability detection strategy focuses on the perspective of multi-agent collaboration. Mao *et al.* [189] propose an approach for vulnerability detection through mutual discussion and consensus among the developer agent and the tester agent, which mimics the real-world code debugging process. GPTLens [117] is a two-stage framework for detecting vulnerabilities in smart contracts, where multiple auditor agents first generate potential vulnerabilities, and a critic agent then ranks the candidates to get the top-k vulnerabilities as the output. The evaluation on 13 real-world smart contracts demonstrates a successful vulnerability detection rate of 76.9%. Fan *et al.* [97] propose a code analysis framework named ICAA, with bug detection as a typical application. ICAA involves a linear pipeline for bug detection. After task planning and code preprocessing, ICAA assigns a react-based analysis agent which is equipped with a series of tools (e.g., retrieval and static analysis tools) to identify bugs. The detected bugs will then be extracted by the report agent and filtered by the false pruner agent, with the refined report as the output. The iAudit [186] framework is a multi-agent system for smart contract auditing with justifications. It finetunes a Detector model alongside a

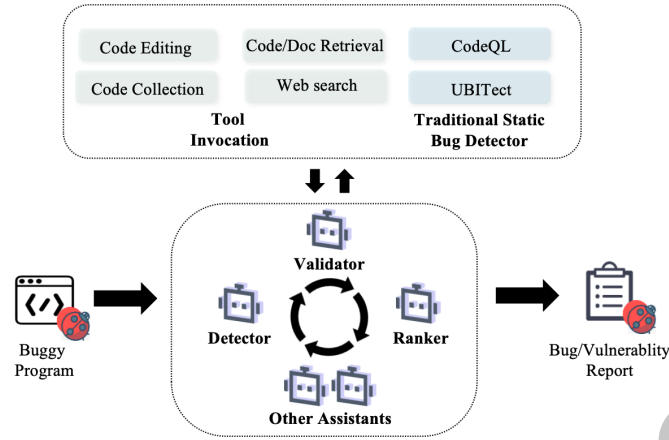


Fig. 8. Pipeline of LLM-based Agents for Static Bug Detection

Reasoner to determine whether the code is vulnerable and provide candidate explanations. Subsequently, an iterative debate ensues between the Ranker and the Critic to select the most compelling justification.

Additional Knowledge from Tool Execution. Another research direction is to enhance the knowledge of LLMs through tool invocation. ART [206] is a framework designed to finish tasks by automatically generating multi-step reasoning decompositions and choosing appropriate tools (like search engines and code execution tools) with the help of retrieved task demonstrations. Employing ART for bug detection outperforms both few-shot prompting and the automated generation of CoT reasoning. ICAA [97] also provides a document retrieval tool and a web search tool to enhance the bug analysis agent with both local and online knowledge. LLM4Vuln [242] enhances the vulnerability reasoning capabilities of LLMs by integrating various knowledge. First, it retrieves both the related vulnerability reports and the summarized vulnerability knowledge from self-constructed databases. Second, the agent invokes tools to obtain further context about the target code (e.g., function or variable definitions). With the help of enriched knowledge, the agent identified 14 zero-day vulnerabilities in four pilot bug bounty programs. PropertyGPT [168] is an agent designed to generate properties for the formal verification of smart contracts. When developing customized properties for the subject code, it first retrieves the knowledge base to find similar code and their reference properties. Based on this knowledge, PropertyGPT produces a set of candidates and refines them iteratively, resolving compilation issues through feedback from the compiler. These properties are then ranked and verified by the prover.

Combined with Traditional Static Bug Detection. Some researchers have combined LLM-based agents with traditional static checking techniques to improve their static bug detection capability. LLift [150] is an agent framework for detecting Use-Before-Initialization (UBI) bug in Linux kernel, which is built on traditional UBI detection tool UBITect [301]. UBITect uses a two-stage pipeline: first, it performs flow-sensitive but path-insensitive static analysis to identify potential UBI bugs, which is fast but imprecise. In the second stage, symbolic execution filters out false positives by exploring feasible paths, but 40% of the bugs are discarded due to time or memory limits. Based on these undecided bugs reported, LLift utilizes agents to identify potential initializers for a suspicious variable from a bug report, extract post-conditions for each initializer, and summarize the initialization status of the variable based on these initializers. Variables without any initializer that must initialize them are potential vulnerabilities. E&V [109] is an agent designed to perform a static analysis of the Linux kernel code. The

Table 5. Existing LLM-based Agents for Static Bug Detection

Agents	Multi-Agent	Tool Utilization		Dataset	Target Program	Bug Category
		Tool Category	Specific Tools			
ART [206]	×	Custom Toolkit	Search Tool Code Generation Tool Code Execution Tool	BigBench [240]	Python Program	Code Errors
GPTLens [117]	✓	-	-	Self-curated	Smart Contract	Smart Contract Vulnerability
ICAA [97]	✓	Custom Toolkit	Context Splitting Tool Code Retrieval Tool Document Retrieval Tool Web Search Tool	NFBugs [217] Self-curated	Python Program Java Program	Non-functional Bugs API Misusage
E&V [109]	×	Static Analysis	Clang [28]	Sampled syzbot [37]	Linux Kernel	Kernel Address Sanitizer Bugs
LLM4Vuln [242]	×	Custom Toolkit	Database Retrieval Tool Context Collection Tool	Self-curated	Smart Contract	Smart Contract Vulnerability
Mao <i>et al.</i> [189]	✓	-	-	SySeVR [157]	C/C++ Program	Library/API Function Call Arithmetic Expression Array Usage Pointer Usage Path-Traversal
IRIS [156]	×	Static Analysis	CodeQL [49]	CWE-Bench-Java [156]	Java Program	OS Command Injection Cross-Site Scripting Code Injection
LLift [150]	×	Static Analysis	UBITect [301]	Rnd-300 [150]	Linux Kernel C Program	UBI Bugs
LLM4DFA [258]	✓	Static Analysis	tree-sitter [39] Z3 Solver [81]	Sampled Juliet Test Suite [55]	Java Program	Divide-By-Zero (DBZ) bugs Cross-Site-Scripting (XSS) bugs
PropertyGPT [168]	×	Custom Toolkit	Database Retrieval Tool	Self-curated	Smart Contract	Smart Contract Vulnerability
iAudit [186]	✓	-	-	Self-curated	Smart Contract	Smart Contract Vulnerability

workflow of E&V is a loop of employing an LLM-based agent for static analysis through pseudo-code execution, verifying the output of pseudo-code, and providing feedback for reanalysis. To mitigate hallucinations from missing necessary code (e.g., inter-procedural call graphs), it retrieves required functions via traditional static analysis tools (e.g., Clang [28]). IRIS [156] is an agent augmented with CodeQL (a static analysis tool) [49] for vulnerability detection. IRIS first utilizes CodeQL to extract candidate APIs in the given repository. Then, it labels these APIs as potential sources or sinks of the given vulnerability via querying the LLM-based agent, which will be further handed over to CodeQL for detecting vulnerable paths. The final verdict is achieved by prompting the LLM agent to analyze the vulnerable paths and the surrounding code of the source and sink. LLM4DFA [258] is a multi-agent system that employs data flow analysis to pinpoint dataflow-related bugs (e.g., divide-by-zero bugs and cross-site-scripting bugs). The process unfolds in three stages: initially, it synthesizes scripts to extract sources and sinks from code using a parsing library (*i.e.*, tree-sitter [39]). Then, the summarizer agent discerns dataflow facts within functions via few-shot learning and CoT prompting. Finally, the agent generates scripts to validate the dataflow facts against path conditions by invoking an SMT solver [81]. The extraction and validation scripts are both refined by the agent through self-evaluation.

Comparison of Bug Detection Enhancement Strategies. In summary, the three most extensively employed approaches in static code checking agents are multi-agent collaboration, knowledge enhancement, and integration of static analysis tools. Multi-agent collaboration enhances detection effectiveness through the division of labor

and task distribution across specialized agents. For single-agent approaches, it is more necessary to integrate additional knowledge and tools, which contribute to bug detection by providing the necessary vulnerability knowledge and code contexts. In particular, traditional static analysis tools can also be integrated into the agent-based methodology, which balances the semantic understanding ability of LLM with the precision of rule-based tools, thereby enhancing detection efficiency and accuracy. In addition, these three strategies can be combined to achieve better performance. For example, LLM4DFA [258] employs both multi-agent collaboration and static analysis tools.

4.3.2 Code Review. Developers review each other's code changes to ensure and improve the code quality before merging the changes into the branch. To mitigate the manual efforts in code review, researchers leverage learning approaches to automate the code review procedure. In particular, code review is formulated as a binary classification problem (*i.e.*, code quality classification [138]) or a sequence-to-sequence generation problem (*i.e.*, review comment generation [153]), which are tackled by fine-tuning or prompting deep learning models (including LLMs). Different from these works, LLM-based agents mimic the real-world peer review procedure by including multiple agents as different code reviewers. Table 6 summarizes existing agents for code review.

Table 6. Existing LLM-based Agents for Code Review

Agents	Multi-Agent Roles	Review Target			
		Consistency	Vulnerability	Code Smell	Code Optimization
CodeAgent [248]	User, CEO, CPO, CTO, Coder, Reviewer	✓	✓	✓	✓
Rasheed <i>et al.</i> [220]	Code Review, Bug Report, Code Smell, Code Optimization Agent		✓	✓	✓
ICAA [97]	Context & Prompt Incubation Agent, Consistency Checking Agent, Report Agent	✓			
CORE [254]	Proposer LLM, Ranker LLM				✓

Process-based Multi-agent Code Review. Process-based multi-agent systems organize agents to follow a structured, sequential workflow that mirrors traditional human-driven code review pipelines. By dividing the review process into distinct stages, these systems coordinate multiple agents with specialized roles to collaboratively complete tasks such as information gathering, code analysis, revision, and documentation in an orderly manner.

CodeAgent [248] is a multi-agent system that simulates a waterfall-like pipeline with four stages (*i.e.*, basic information synchronization, code review, code alignment, and document) and sets up a code review team with six agents of different characters (*i.e.*, user, CEO, CPO, CTO, coder, and reviewer). In the basic information synchronization phase, the CEO, CPO, and coder agents analyze the input modality and programming language. After that, the coder and reviewer agents collaborate to conduct a code review and produce the analysis report. In the code alignment phase, the coder and reviewer agents continue to revise the code based on the analysis reports. Finally, in the document phase, the CEO, CTO, and coder agents cooperate to document the holistic code review process. Experimental results demonstrate the effectiveness and efficiency of CodeAgent in various code review tasks, including consistency analysis, vulnerability analysis, format analysis, and code revision.

ICAA [97] designs a multi-agent system to identify code-intention inconsistencies. It first uses the Context & Prompt Incubation Agent to collect necessary information from the code repository through a thinking-decision-action loop. The Consistency Checking Agent will then analyze collected information and identify inconsistencies, which will be handed over to the Report Agent to form a final report.

CORE [254] designs a system with two agents, along with traditional static analysis tools to fix code quality issues automatically. Specifically, the Proposer agent takes the static analysis report, the suspicious file, and the

issue documentation from language-specific static analysis tools (e.g., CodeQL [49]) and the tool provider (e.g., the QA team), and proposes candidate revisions for each suspicious file. After that, static analysis tools will prune revisions that still have issues, while the rest will be scored and re-ranked based on their likelihood of acceptance by the Ranker agent.

Goal-based Multi-agent Code Review. Goal-based multi-agent systems focus on assigning agents to specialize in individual, well-defined tasks within the code review domain. Instead of following a fixed pipeline, each agent independently addresses a specific aspect of code quality, such as bug detection or code optimization, allowing modular development and targeted expertise for different review objectives. For example, Rasheed *et al.* [220] design an approach with each agent specialized for a single code review task. Notably, it proposes four agents, including the code review agent, bug report agent, code smell agent, and code optimization agent. Each agent is trained on relevant GitHub data and evaluated on 10 AI-based projects. The results demonstrate the potential of applying multi-agent systems in the code review task.

Comparison of Different Multi-agent Code Review Strategies. Process-based multi-agent systems emphasize structured workflows where agents collaborate through well-defined sequential stages, enabling comprehensive and coordinated management of the code review lifecycle. This approach ensures clear role assignments and orderly progression of tasks. In contrast, goal-based systems prioritize specialization by assigning agents to independently tackle distinct code review subtasks. Such modularity allows focused expertise and flexible scalability but may lack the integrated coordination found in process-oriented designs. In summary, process-based frameworks offer end-to-end orchestration suitable for holistic review, while goal-based frameworks optimize for task-specific performance. Combining these paradigms may yield more effective multi-agent code review systems.

4.3.3 Challenges of LLM-based Agents in Static Code Checking. LLM-based agents face several challenges in static code checking tasks. First, these agents often incorporate traditional static bug detection tools such as UBITect and CodeQL. However, the integration remains relatively shallow. In many cases, the agents merely filter false positives based on the outputs of these tools, rather than enabling deeper collaboration. A tighter integration between the model and external tools could potentially improve performance and reduce noise [150, 156]. Second, static analysis tasks often require LLM-based agents to possess both a strong understanding of code and the capability to generate intermediate representations, such as pseudocode [109], execution specifications [109], or dataflow summaries [258]. These requirements place greater demands on the model's reasoning capabilities and often necessitate the use of high-performing proprietary models such as GPT-4 [109, 150]. Finally, LLM-based agents may still produce false positives, and there is a lack of effective automated mechanisms for verifying or filtering these incorrect results, which further limits their practical reliability [97, 117, 186].

4.4 Testing

Software testing is essential for software quality assurance. LLMs have demonstrated promising proficiency in test generation, including generating test code, test inputs, and test oracles. However, generating high-quality tests in practice can be challenging, as the generated tests should not only be syntactically and semantically correct (*i.e.*, both the inputs and oracles should satisfy the specification of the software under test) but also be sufficient (*i.e.*, the tests should cover as many states of the software under test as possible). As shown by previous work [296], the tests generated by standalone LLMs still exhibit correctness issues (*i.e.*, compilation errors, run-time errors, and oracle issues) and unsatisfactory coverage. Therefore, researchers build LLM-based agents to extend the capabilities of standalone LLMs in test generation.

Table 7. Existing LLM-based Agents for Unit Testing

Agents	Multi-Agent	Feedback Goal	Feedback Source	Target Language
ChatTester [296]	×	Reduce compilation/execution errors	Error messages	Java, Python
TestPilot [231]	×	Reduce compilation/execution errors	Error messages	JavaScript
ChatUnitTest [68]	×	Reduce compilation/execution errors	Error messages	Java
AgentCoder [119]	×	Reduce compilation/execution errors	Error messages	Python
TELPA [285]	×	Increase coverage	Program analysis results	Python
CoverUp [42]	×	Increase coverage	Execution results & Coverage	Python
MuTAP [80]	×	Enhance fault detection	Surviving mutants	Python
AutoDev [252]	✓	Reduce compilation/execution errors Increase coverage	Error messages	Java
Mokav [93]	×	Enhance fault detection	Execution results	Python

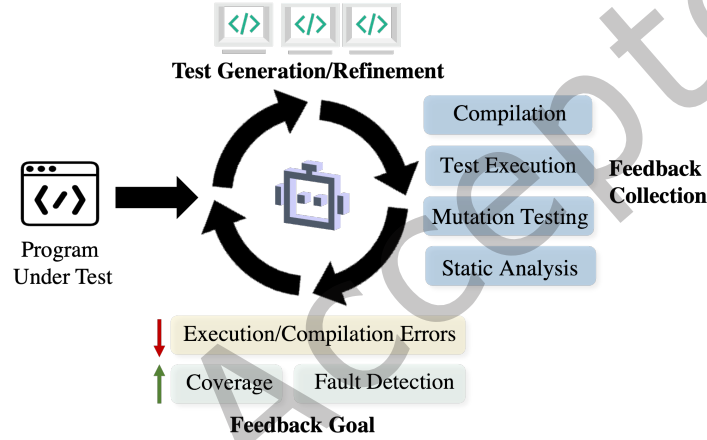


Fig. 9. Pipeline of LLM-based Agents for Unit Testing

4.4.1 Unit Testing. Unit testing checks the isolated and small unit (*e.g.*, method or class) in the software under test, which helps quickly identify and locate bugs, especially for complicated software systems. Yuan *et al.* [296] perform a study showing the potentials of LLMs (*e.g.*, ChatGPT) in generating unit tests with decent readability and usability. However, the unit tests generated by standalone LLMs still exhibit compilation/execution errors and limited coverage. Therefore, recent works have built LLM-based agents that extend standalone LLMs by iteratively refining the generated unit tests with distinct strategies and targets. We organize this section around the three primary enhancement directions, including reducing compilation/execution errors, improving test coverage, and enhancing the fault detection capability of tests. Table 7 summarizes the existing LLM-based agents for unit test generation.

Framework: Figure 9 illustrates the common framework of LLM-based agents for unit testing. Agent-based testing systems take the program under test (PUT) as input, generate initial test cases, and iteratively refine them by leveraging feedback from external tools such as compilation/testing outputs and static analysis results. The refinement target includes reducing execution/compilation errors, improving test coverage, and enhancing the fault detection capability.

Iterative Refinement to Fix Compilation/Execution Errors. The test cases directly generated by LLMs can exhibit compilation or execution errors. Therefore, inspired by program repair [280], LLM-based agents further eliminate such errors by iteratively collecting the error messages and fixing the buggy test code [68, 119, 231, 252, 296]. Existing LLM-based agents used for unit test generation all follow a generate-verify-fix pipeline, where error messages from test execution are fed back to refine the test script. The primary differences among these agents lie in the context integrated into their prompts. ChatUniTester [68] integrates code context into the prompt, including the focal method and class, as well as dependent methods and classes. AgentCoder [119] designs a test generation prompt with three clear objectives: (i) to generate basic test cases, (ii) to cover edge test cases, and (iii) to cover large-scale inputs. This prompt enhances the accuracy and adequacy of the generated test scripts. TestPilot [231] designs a complicated prompt, including the signature, definition, doc comment, and usage snippets extracted from the documentation of the focal function. ChatTester [296] introduces an intention prompt before the test generation prompt, which instructs the LLM to understand the intention of the focal methods first. Different from the above approaches, AutoDev [252], as a React-style agent, primarily relies on feedback to improve the accuracy of test generation instead of prompt construction. In summary, different approaches for unit testing generation tend to focus on different contexts, such as more detailed code and documentation context, more specific test generation goals, and clearer method intentions. However, due to the different evaluation datasets and criteria used by each method, direct cross-method comparisons of their effectiveness are not feasible, which poses a challenge for establishing a unified standard.

Iterative Refinement to Increase Coverage. In addition to enhancing the success rate of test execution, the feedback mechanism in LLM-based agents can also be employed to improve the coverage of unit testing. CoverUp [42] is an LLM-powered test generation system aimed at achieving higher coverage rates. It initially employs the coverage analysis tool SlipCover [209] to measure existing test suite coverage and identify uncovered code segments using abstract syntax trees. This information, along with the methods under test, is provided to the LLM to generate new tests. If coverage does not increase after test execution (verified by iteratively invoke SlipCover with near-zero overhead), error messages are fed back to the LLM for re-generation. In addition to iterative feedback, the LLM is equipped with a tool to query types or variables in the code segments, which facilitates test generation. TELPA [285] is an LLM-based agent for enhancing test generation for hard-to-cover branches through iterative feedback. Based on the tests generated by existing tools (e.g., the search-based software test technique Pynguin [181] and the LLM-based technique CodaMosa [146]) to cover easy-to-reach branches, it performs backward and forward method invocation analyses to extract relevant information for constructing complex objects and understanding inter-procedural dependencies. Finally, TELPA employs a feedback-based process with the LLM, using counter-examples to iteratively refine and generate tests that improve coverage of difficult branches. AutoDev [252] iteratively generates, executes, and revises tests, achieving 99.3% test coverage on the HumanEval [65] dataset, which is comparable to the human-written tests' coverage.

Iterative Refinement to Increase Fault Detection Capabilities. Besides execution success rate and test coverage, the quality of test cases has also garnered research attention, particularly in generating test cases with enhanced fault detection capabilities. MuTAP [80] is a single LLM-based agent system that aims at generating unit tests of better bug detection capabilities with the feedback of mutation testing. It employs prompt augmentation with surviving mutants and refining steps to correct syntax and intended behavior. During each iteration, the LLM first generates initial test cases and self-refines their syntax errors and wrong behaviors, with the help of the Python parser to locate the erroneous line. Then the tests run against the mutated programs, while the surviving mutants serve as feedback to direct the LLM in improving the test cases. Mokav [93] is an agent to generate difference-exposing tests. It first summarizes the descriptions of the two programs under test. Subsequently, Mokav engages in an iterative process, crafting tests that are refined based on the feedback from execution outcomes, until the tests are capable of exposing the distinctions between the two programs.

4.4.2 System Testing. System testing is a comprehensive process that assesses an integrated software system/component to guarantee that it fulfills its specification and operates as intended across diverse settings. For example, fuzzing testing and GUI (Graphical User Interface) testing are common testing paradigms at the system level. Leveraging LLMs for system testing can be challenging, as generating valid and effective system-level test cases should satisfy the constraints that are contained implicitly and explicitly in the specifications or domain knowledge of the software system under test. Besides, the entire system not only involves multiple interacting components and modules but also contains different execution paths and behaviors, resulting in numerous dependencies, interactions and scenarios that standalone LLMs may struggle to fully obtain and account for when generating test cases. LLM-based agents are designed to better incorporate the domain knowledge and dynamically explore the software system under test compared to generating system-level tests via standalone LLMs. We then organize these works according to the software systems under test. Table 8 summarizes the existing agents for different software systems.

OS Kernel. KernelGPT [287] is an LLM-based agent for kernel fuzzing. Initially, KernelGPT uses a code extractor and analysis LLM to identify device operation handlers and infer device names and initialization specifications. It then iteratively analyzes the source code to generate syscall specifications, including command values, argument types, and type definitions. Finally, it invokes the Syzkaller tool [38] (e.g., *syz-extract* and *syz-generate*, which can detect errors in the generated specifications), and repairs any invalid specifications by consulting the LLM with error messages iteratively.

Compiler. WhiteFox [286] encompasses two LLM-based agents, an analysis agent and a generation agent. While the former examines the low-level optimization source code and produces requirements on the high-level test programs that can trigger the optimizations, the latter crafts test programs based on summarized requirements. The generation agent further incorporates tests that have successfully triggered optimizations as feedback during the iterative process, thereby producing more satisfactory tests. LLM4CBI [251] is a single agent that aims at isolating compiler bugs by generating test cases with better fault detection capabilities. The agent utilizes tools to collect static information about the program (e.g., srcSlice [200] for data flow) to construct precise prompts to guide the LLM for program mutation. The memorized component records meaningful prompts and selects better ones to instruct LLMs to generate variants. The generated programs undergo validation by a static analysis tool (e.g., the Frama-C [4], which is an open-source static analysis toolset for C language), and the feedback helps LLMs to avoid the same mistakes. The final test cases are used to identify suspicious files with spectrum-based fault localization techniques.

Mobile Applications. LLM-based agents are proposed to automate the testing process of mobile applications, including *GUI testing*, *bug replay*, and *user acceptance testing*.

Some agents are developed to execute *GUI testing* for mobile applications. GUI testing is a commonly used software testing method aimed at verifying whether the user interface meets service specifications and user requirements. Previous LLM-based GUI testing approaches lack adequate autonomy, long-term planning, and coherence [170, 271]. The emergence of LLM-based agents enables GUI testing to focus more on higher-level test objectives [171–173, 245, 292], such as clear task objectives, without relying on specific GUI states. Liu *et al.* [171] propose a framework called GPTDroid, where the LLM iterates the entire process by perceiving GUI page information, generating test scripts in the form of Q&A, executing these scripts through tools, and receiving feedback from the application. GPTDroid keeps a long-term memory to retain testing knowledge, which would help to improve the reasoning process. The DroidAgent [292] framework employs multiple LLM-based agents coordinating through different memory modules and can set its own tasks according to the functionalities of the apps under test. It is composed of four LLM-based agents: planner, actor, observer, and reflector, each with specific roles and supported by memory modules that enable long-term planning and interaction with external tools. AXNav [245] is another multi-agent system designed for replaying accessibility tests on mobile apps. It includes the planner agent, the action agent, and the evaluation agent, which together form the LLM-based UI navigation

Table 8. Existing LLM-based Agents for System Testing

Software System	Agents	Multi-Agent	Tool		Output
			Tool Category	Specific Tools	
OS Kernel	KernelGPT [287]	×	Static Analysis	syz-extract [38] LLVM Toolchain [7]	Syzkaller Specifications
Compiler	WhiteFox [286]	✓	-	-	Test Cases
	LLM4CBI [251]	×	Static Analysis	OCLint [9] srcSlice [200] Gcov [17] Frama-C [4]	Mutated Programs
Mobile App	GPTDroid [171]	×	Execution Environment	VirtualBox [23] pyvbox [22] Android UIAutomator [11] Android Debug Bridge [13]	Test Scripts
		✓	Custom Toolkit	Navigation Action Toolkit	Test Scripts
		×	Execution Environment	Android UIAutomator [11]	Unusual Text Inputs
	InputBlaster [172]	×	Execution Environment	Android UIAutomator [11]	Unusual Text Inputs
	AXNav [245]	✓	Custom Toolkit	Navigation Action Toolkit	Bug Replay Video
	AdbGPT [100]	×	Execution Environment	Genymotion [18] Android UIAutomator2 [14] Android Debug Bridge [13]	Bug Replay Steps
		✓	Execution Environment	VirtualBox [23] pyvbox [22] Android UIAutomator [11] Android Debug Bridge [13]	Detected Bugs
	XUAT-Copilot [268]	✓	-	-	Test Scripts
Web App	RESTSpecIT [83]	×	-	-	OpenAPI Specification
	Fuzz4All [277]	✓	-	-	Test Cases
Universal	PentestGPT [84]	✓	Testing Tool	Metasploit [8]	Test Operations
	Fang <i>et al.</i> [98]	×	Custom Toolkit	Web Browsing Tool File Creation and Editing Tool	Exploit Actions
			Execution Environment	Terminal Code Interpreter	

system. These agents translate test instructions into executable steps, conduct tests on a cloud-based iOS device, and summarize the test results in a chaptered video annotated with potential issues in the application, respectively. InputBlaster [172] is an agent designed to generate unusual text inputs for mobile app crash detection. Initially, it infers the input constraints and generates a valid text input, based on the GUI page information. Building on this valid input and constraints, it then generates appropriate mutation rules with corresponding test generators, in the form of natural language and code snippets, respectively. Each test generator produces a batch of test inputs, and the test execution feedback will help the agent to produce more diversified outcomes. Additionally, the agent can retrieve relevant examples of buggy input for a better understanding of the task. VisionDroid [173] is a multi-agent system designed to detect non-crash functional bugs via multimodal LLM. It is composed of the function-aware explorer and the logic-aware bug detector. The explorer takes both the image and text information to comprehend the GUI page, generating actions to explore the functionality of the app and memorize the testing

history. The intra-page bugs can be found by the explorer. The detector then segments the exploration history to check the inconsistency between the process logic and the GUI change history, which leads to the detection of inter-page bugs.

For automating Android *bug replay*, Feng *et al.* [100] introduce AdbGPT. Equipped with the knowledge of Step-to-Reproduce (S2R) entity specifications (*i.e.*, predefined actions and action primitives), AdbGPT analyzes bug reports to translate identified entities into a sequence of actions for bug reproduction using the CoT strategy. It then perceives GUI states dynamically and maps the S2R entities to actual GUI events to replicate the reported bug.

To increase the automation of the *user acceptance testing* process, Wang *et al.* [268] propose XUAT-Copilot. The system is primarily comprised of three LLM-based agents responsible for action planning, state checking, and parameter selection, as well as two additional modules for state awareness and case rewriting. These agents interact with the testing equipment collaboratively, making human-like decisions and generating action commands.

Web Applications. RESTful APIs are popular among web applications as they provide a standardized, stateless, and easily integrable means of communication that enhances scalability and performance through a resource-oriented approach. RESTSpecIT [83] leverages LLMs to automatically infer RESTful API specifications and conduct black-box testing. Given an API name, RESTSpecIT generates and mutates HTTP requests through a reflection loop. By sending these requests to the API endpoint, it analyzes the HTTP responses for inference and testing. The LLM uses valid requests as feedback to refine the mutations in each iteration. Requests are validated based on the status code and message of the returned response.

Universal Software Categories. Some agent systems are not designed with a task-specific workflow, enabling them to be universally applicable across various target software systems. Xia *et al.* [277] present Fuzz4All, the first universal LLM-based fuzzer for general and targeted fuzzing across multiple programming languages. For a higher cost-effectiveness ratio, Fuzz4All consists of two agents: (i) the distillation LLM for user input distillation and initial prompt generation, and (ii) the generation LLM for fuzzing input generation. They are powered by LLMs with different capabilities. In the fuzzing loop, the generation LLM refers to the previously generated samples and dynamically adjusts its strategy, thereby producing diverse fuzzing inputs. Deng *et al.* [84] design a modular framework, PentestGPT, to conduct Penetration Testing. The system includes inference, generation, and parsing modules. With the planning strategy of Pentesting Task Tree (which is based on the cybersecurity attack tree [194]) and CoT methods, PentestGPT solves the problems of context loss and inaccurate instruction generation that may be encountered during automated penetration testing. Fang *et al.* [98] develop a benchmark consisting of 15 one-day vulnerabilities to assess the efficacy of their agent framework in exploiting such weaknesses, utilizing various LLM backbones. Their agents are imbued with an understanding of the Common Vulnerabilities and Exposures (CVE) descriptions and are capable of harnessing a suite of tools to facilitate the exploitation process. These tools include web browsing capabilities for navigation, web search functionalities for traversing web pages, as well as terminal and code interpreter access for the generation and execution of scripts.

4.4.3 Challenges of LLM-based Agents in software testing. LLM-based agents face several unique challenges in software testing tasks. First, the complexity of test environments often requires analyzing testing targets that are embedded within class-level or even project-level contexts. This issue arises not only in system-level testing but also in unit testing, which can depend on subtle interactions across multiple components. As a result, accurate test generation frequently demands sophisticated context augmentation mechanisms that can incorporate broader structural and semantic information from the codebases and documentations [42, 68, 245, 285]. Second, despite the availability of mature traditional testing tools, effectively integrating them into the execution workflow of LLM-based agents remains a significant challenge. For example, WhiteFox [286] can guide input generation to support traditional fuzzing methods, such as NNSmith [162], and TestPilot [231] can produce initial tests to

support traditional feedback-directed techniques, such as Nessie [46]. However, these tools are typically used alongside agents rather than being tightly integrated into a unified testing strategy. Finally, current LLM-based testing agents tend to focus on narrow, task-specific goals. For example, InputBlaster [172] targets the generation of unusual text inputs for mobile app crash detection. However, testing is inherently multidimensional, involving aspects such as test quality, coverage, security, and assertion generation. Multi-agent architectures hold promise for coordinating these diverse concerns within a more complete testing workflow, but this direction remains underexplored.

4.5 Debugging

Software debugging typically includes two phases: *fault localization* [273] and *program repair* [104]. In particular, fault localization techniques aim at identifying buggy elements (e.g., buggy statements or methods) of the program based on the buggy symptoms (e.g., test failure information); then, based on the buggy elements identified in the fault localization phase, program repair techniques generate patches to fix the buggy code. In addition, recent works also propose *unified debugging* to bridge fault localization and program repair in a bidirectional way [144]. We then organize the works in LLM-based agents for debugging into three parts, *i.e.*, fault localization, program repair, and unified debugging.

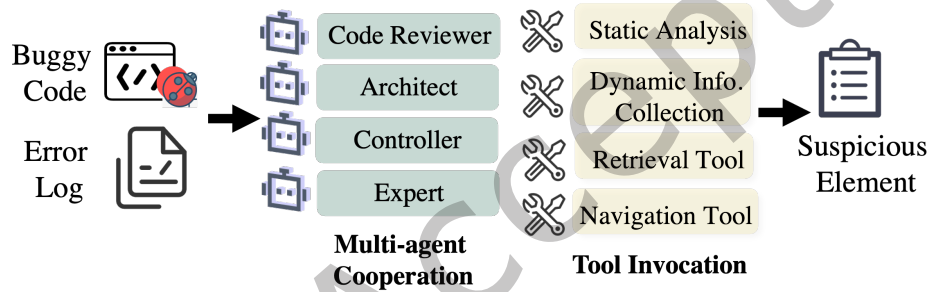


Fig. 10. Pipeline of LLM-based Agents for Fault Localization

Table 9. Existing LLM-based Agents for Fault Localization

Agents	Multi-Agent	Tools		Input Context	FL Granularity	Target Language
		Tool Category	Specific Tools			
AgentFL [216]	✓	Static Analysis	Tree-sitter [39]	Project Level	Method	Java
AutoFL [136]	×	Custom Toolkit	Repository Retrieval Tools	Project Level	Method	Java

4.5.1 Fault Localization. Learning-based fault localization has been widely studied before the era of LLM, which typically trains deep learning models to predict the probability that each code element is buggy or not [154]. However, precisely identifying the buggy elements of software is challenging, given the scale of software systems and the massive, diverse error messages, which are often beyond the capabilities of standalone learning models, including LLMs. Therefore, recent works build LLM-based agents, which incorporate multi-agents and tool usage to help LLMs tackle these challenges. Table 9 summarizes the existing LLM-based agents for fault localization.

Framework: Figure 10 illustrates the common framework of LLM-based agents for fault localization. Generally, the buggy code and error log will be fed into the agent systems, in which common roles like code reviewer,

architect, and other experts will cooperate to explore the suspicious elements in the buggy code. In this process, these agents can invoke pre-defined tools, such as static analysis and code retrieval tools, to collect relevant code segments within the repository.

Multi-agent Synergy. AgentFL [216] is a multi-agent system for project-level fault localization. The main insight of AgentFL is to scale up LLM-based fault localization to project-level code context via the synergy of multiple agents. The system consists of four distinct LLM-driven agents: test code reviewer, source code reviewer, software architect, and software test engineer. Each agent is customized with specialized tools and a unique set of expertise. With the four agents, AgentFL streamlines the project-level fault localization process by breaking it down into three phases: fault comprehension, codebase navigation, and fault confirmation.

Tool Invocation. AutoFL [136] is a single-agent system, which enhances standalone LLMs with tool invocation (*i.e.*, four specialized function calls) to better explore the repository. It first performs root cause explanation, invoking tools to oversee the source code repository for pertinent information, requiring only a single failing test and its failure stack. During this stage, it autonomously decides whether to continue function calling or to terminate with the production of a root cause explanation. Subsequently, a post-processing step is used to correlate the outputs with exact code elements, aiming at bug localization. In addition, AgentFL [216] also incorporates tool invocation (*e.g.*, static analysis, dynamic instrumentation, and code base navigation) into its framework.

4.5.2 Program Repair. Fine-tuning and fixed prompting are the most widely adopted paradigms for program repair techniques based on standalone LLMs. In particular, program repair is formulated as a translation problem [71] (*i.e.*, translating the buggy code to correct code) or a generation problem [279] (*e.g.*, infilling the correct code in the buggy code context). However, patches generated by LLMs in a single iteration are not always correct; they may fail to pass all tests or overfit to the test cases. Therefore, existing LLM-based agents follow an iterative paradigm to refine patch generation based on the tool or model feedback in each iteration. Table 10 summarizes the existing LLM-based agents for program repair.

Framework: Figure 11 illustrates the common framework of LLM-based agents for program repair. First, they generate an initial candidate patch for the buggy code. The patch is then validated against predefined test cases through compilation and execution. Based on the feedback from these validation steps, including compilation errors, runtime failures, or test case outcomes, the patch undergoes iterative refinement. This cycle continues until the patch meets the acceptance criteria, such as achieving full test suite pass coverage.

Table 10. Existing LLM-based Agents for Program Repair

Agents	Multi-Agent	Feedback Source	Target Software	Benchmark	Correct Fix	
					Rate	Metric
ChatRepair [281]	×	Execution/Compilation	Java/Python	Sampled Defects4J [134] QuixBugs [159]	162/337 (Defects4J) 80/80 (QuixBugs)	Pass Tests Semantic Equivalence (Manual)
CigaR [114]	×	Execution/Compilation	Java	Sampled Defects4J HumanEval-Java [65]	69/267 (Defects4J) 102/162 (HumanEval)	Pass Tests AST Match
RepairAgent [57]	×	Execution/Compilation	Java	Defects4J	164/835	Pass Tests Syntax Match (Automatic) Semantic Equivalence (Manual)
AutoSD [137]	✓	Execution	Java/Python	Defects4J BugsInPy [272] Almost-Right HumanEval	189/835 (Defects4J) 187/200 (HumanEval)	Pass Tests Semantic Equivalence (Manual)
ACFix [309]	✓	Static Checking Model Debate	Smart Contract	Self-curated Dataset	112/118	Comparison with Author Fixes Execute Exploit Scripts Manual Inspection
FlakyDoctor [69]	✓	Static Checking Execution	Java	Sampled IDoFT [5] DexFix dataset [311] Sampled ODRepair dataset [148]	311/541 (Implementation-Dependent Flakiness) 189/332 (Order-Dependent Flakiness)	Manual Inspection
SRepair [282]	✓	-	Java	Sampled Defects4J QuixBugs	332/665 (Defects4J) 80/80 (QuixBugs)	Pass Tests Semantic Equivalence (Manual)

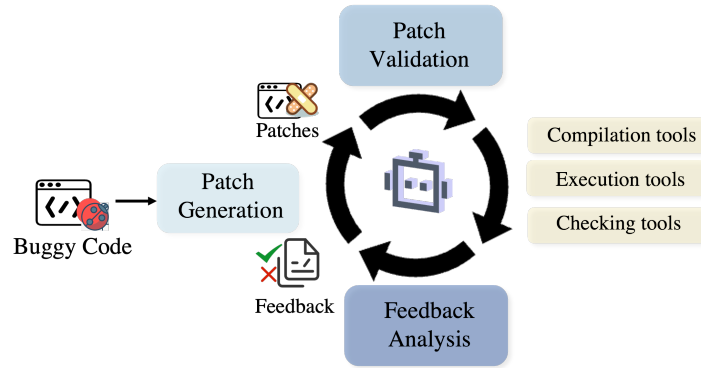


Fig. 11. Pipeline of LLM-based Agents for Program Repair

ChatRepair [280, 281] is the first automated approach to refine patch or program generation based on environmental feedback iteratively. Specifically, it incorporates previously generated patches and test execution feedback into the prompt and feeds it into the LLM to generate new patches, thereby learning from both incorrect and plausible patches to ultimately achieve correct repairs. Ultimately, considering that the test suite may be incomplete, patches generated based on it may not always cover all intended uses of the underlying code. Therefore, ChatRepair continues the iteration with only plausible patches provided to the LLM to generate more plausible patches for manual inspection. CigaR [114] adopts a similar approach, with the generated patches and test failure messages serving as the feedback for the LLM to generate new patches. It also employs a plausible patch multiplication stage to iteratively produce more plausible patches. A notable difference is that CigaR employs a reboot mechanism. If the LLM fails to generate a plausible patch within the maximum number of invocations, the entire repair process is restarted. Experimental results show that this reboot strategy enables the LLM to efficiently explore different parts of the search space, avoiding wasting tokens on dead ends.

RepairAgent [57] is a highly autonomous LLM-based agent. It consists of three main components: an LLM agent, a set of tools for interacting with the codebase (e.g., reading code, searching code base, and running tests), and a middleware, which is a state machine containing four states: *understand the bug*, *collect information to fix the bug*, *try to fix the bug* and *done*. The middleware accepts the action request from the LLM agent and invokes tools, with the results dynamically updating the prompt and then being fed into the LLM agent. Experimental results demonstrate that RepairAgent repairs 164 bugs in Defects4j [134], including 39 bugs not fixed by prior techniques.

AutoSD [137] is a multi-agent system that iteratively fixes the buggy program via simulating the scientific debugging [300]. AutoSD includes four components: LLM-based hypothesis generator, execution-based validator, LLM-based conclusion maker, and LLM-based fixer. In each iteration, the generator first generates a hypothesis about the bug, then invokes the debugger tool for hypothesis validation; the conclusion maker further identifies whether the hypothesis is rejected or not; and the fixer finally returns potential patches with explanations.

ACFix [309] is a multi-agent system for fixing the access control vulnerabilities in smart contracts. By specializing LLMs with different roles, ACFix includes a Role-based Access Control (RBAC) mechanism identifier, a role-permission pair identifier, a patch generator, and a validator. In particular, the validator checks the validity of the generated patches with both tool feedback (static grammar rule checking) and model feedback (multi-agent debate process). The feedback is further provided to iteratively refine the patch.

FlakyDoctor [69] is an agent to repair flaky tests. It takes the test execution results and the location of test failures into consideration. Following this, the agent generates targeted repairs and tests them for validation. This process is iterative, with the aim of continually refining the repairs until the issue of test flakiness is resolved.

SRepair [282] is a dual-agent system for function-level program repair. One agent first analyzes the auxiliary repair-relevant information (e.g., the buggy code, error messages) to produce fix suggestions via the CoT technique, while the other generates fixed functions with the help of the suggestions.

4.5.3 Unified Debugging. Instead of tackling fault localization or program repair as isolated phases, unified debugging techniques treat them as a unified procedure, which leverages the outputs of each phase to refine the other. In particular, traditional unified debugging techniques [52, 53, 177] primarily pre-define heuristic rules to refine fault localization based on the patch validation results during program repair. Recently, LLM-based agents have enhanced traditional unified debugging techniques with more flexibility by leveraging LLMs to comprehend, utilize, and unify the outputs of both fault localization and program repair.

FixAgent [144] is a multi-agent framework for automated debugging, with each agent simulating the “rubber duck debugging” strategies to explain their detailed work. It includes an LLM Localizer for identifying bugs, an LLM Repairer for generating patches, an LLM Crafter for creating additional test inputs to ensure patch generalizability, and an LLM Revisor for analyzing symptoms of the buggy code and the rationale of the patch. The agents cooperate in an iterative manner, in which the downstream agent depends on the results from the upstream agent but can also influence the upstream agent in the next turn. For instance, if the Repairer modifies code segments different from those identified by the Localizer, the Localizer will adjust its localization results accordingly. LDB [320] also adopts an iterative approach. In each iteration, it divides the current program into different blocks and precisely analyzes the changes in variables before and after each block during test execution. Based on this information, it queries LLMs to verify each block according to the task, thereby locating and repairing bugs.

In summary, to coordinate fault localization and program repair and achieve unified debugging, these agents employ a division of labor, with two different agents responsible for localization and repair separately. Moreover, these approaches involve fine-grained analysis. For example, FixAgent requires each agent to explain its work to a “rubber duck”, while LDB incorporates the runtime analysis. As a key distinction, FixAgent considers the interplay between the fault localization and program repair phases, which leverages the output of each phase to refine the other, distinguishing it from traditional single-stage fault localization and program repair works.

4.5.4 Common Failure Causes of LLM-Based Agents in Debugging. Based on the failure analyses of existing LLM-based agents in debugging, we summarize several common failure causes that reflect the current limitations of LLM-based debugging agents in real-world scenarios.

Lack of Understanding of Complex Project Context. Effective debugging often entails a comprehensive traversal of the software project to grasp its class hierarchies, inter-method dependencies, and underlying test architecture. This exploration is necessary to identify the root cause of bugs, but it can also introduce significant noise. For example, in AutoFL [136], most failures stemmed from the agent spending excessive rounds trying to understand project-specific structures such as custom classes, helper functions, and test frameworks, leaving insufficient budget for inspecting potentially buggy code. This suggests that current agents need more effective strategies for reducing the search space and selectively incorporating project-specific context.

Lack of coherence across multi-step debugging workflows. A successful debugging process typically involves a sequence of reasoning, hypothesis testing, and verification. However, these steps are not always well-aligned in current agents. For example, in AutoSD [137], the agent proposed a breakpoint that was never hit during test execution. Instead of revisiting the hypothesis or suggesting alternative breakpoints, the agent incorrectly suggested that the test was flawed. This disjointed reasoning between the analysis and verification

steps led to an incorrect debugging trajectory. Such inconsistencies highlight the need for tighter integration between reasoning and feedback in iterative debugging loops.

Other Failures. There are several lower-frequency but still impactful failure types. For instance, in some cases, buggy methods are too long to fit within the agent’s context window [136], resulting in truncated analysis or context overflow. In others, the agent introduces logical errors or edits only a subset of the necessary locations in its fix proposals [57, 136]. There are also errors caused by the agent generating tool-incompatible invocations. For example, AutoSD [137] might insert multiple print statements in contexts that only allowed one. While less prevalent, these issues expose limitations in the agent’s handling of edge cases, context constraints, and system-level instructions.

4.5.5 Challenges of LLM-based Agents in Debugging. LLM-based agents face several distinctive challenges in debugging tasks. Many current approaches rely heavily on failing test cases, but using intermediate execution signals alone to assess program correctness remains an open research direction [57, 136, 320]. In addition, LLM-based agent systems often incorporate static analysis or runtime tools to enhance fault localization. A key challenge lies in designing effective integration strategies that allow LLM-based agents to leverage these tools’ strengths while mitigating issues such as performance overhead, data latency, and inconsistency in results [282, 320]. Coordination mechanisms that enable tighter coupling between LLM reasoning and tool feedback are essential to improve debugging efficiency and accuracy. Furthermore, although multi-agent collaboration can potentially enhance the reasoning capabilities of debugging systems, it also introduces additional system complexity and significant performance bottlenecks. For example, AutoSD takes approximately five times longer to generate a patch compared to a standalone LLM [137, 144]. Finally, most existing debugging agents generate only plausible patches, and verifying whether these patches are semantically equivalent to the intended golden patch still requires manual effort. This limits both the level of automation and the scalability of current LLM-based debugging agents [282].

4.6 IT Operations

IT operations (Ops) involve managing and maintaining the technology infrastructure of an organization, ensuring systems run smoothly, and quickly addressing any issues that arise. With the rise of AI, Ops are becoming increasingly automated and intelligent [308]. However, standalone LLMs struggle with this task, as they lack the ability to directly interact with systems or perform real-time actions. In contrast, LLM-based agents can integrate with IT environments, enabling them to analyze data, automate processes, and respond to incidents, making them highly effective in IT operations. However, different LLM-based agents might adopt various collaborative diagnosis strategies, which we categorize into three types: self-consistency with embedding voting, agent chain with blockchain-inspired voting, and tree search with majority voting.

Framework: The overall pipeline for LLM-based agents in IT operations is illustrated in Figure 12. Upon receiving anomaly alerts, the controller will dispatch the diagnosis tasks to diagnostic expert agents, who will then collaborate to analyze the root cause through interaction with the target system. Each expert agent is equipped with predefined toolkits to interact with the environment or access domain-specific reference documents. During the diagnostic process, they will try different exploration paths, ultimately producing a diagnostic report or solution.

4.6.1 Self-Consistency with Embedding Voting. RCAgent [266] proposes a self-consistency mechanism that shares preliminary steps across multiple reasoning trajectories, and uses vector similarity in embedding space

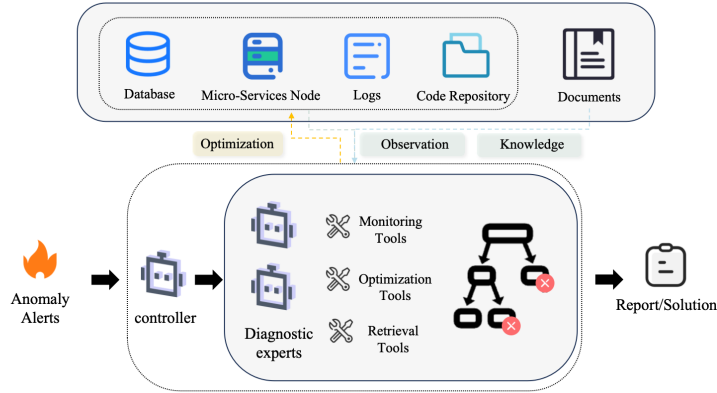


Fig. 12. Pipeline of LLM-based Agents for IT Operations

to aggregate and weigh outcomes. As a multi-agent system for root cause analysis in industrial cloud settings, RCAGENT includes two components: the controller agent and the expert agents. The controller agent oversees the comprehensive thought-action-observation cycle, while the expert agents act for specialized tasks (*i.e.*, code and log analysis). Through the self-consistency mechanism, RCAGENT mitigates computational overhead during inference, thereby enhancing both efficiency and robustness.

4.6.2 Agent Chain with Blockchain-Inspired Voting. mABC [313] proposes a blockchain-inspired voting mechanism to ensure agreement and reliability among different diagnostic experts. The main workflow of mABC is as follows: when an alert arises, the Alert Receiver agent will prioritize incoming alerts and select the most critical one for investigation. The Process Scheduler then breaks down the root cause analysis task into smaller subtasks and assigns them to specialized agents, including the Data Detective, Dependency Explorer, Probability Oracle, and Fault Mapper agents. Finally, the Solution Engineer agent proposes remediation strategies based on historical knowledge. Throughout this process, all agents compose a decentralized structure, Agent Chain, and participate in a blockchain-inspired voting mechanism to ensure reliable consensus on the root cause and solution.

4.6.3 Tree Search with Majority Voting. D-Bot [323] leverages LLM-based agents to diagnose database anomalies through several key steps. First, it extracts diagnostic knowledge from manuals and documentation offline. Then, it generates prompts automatically by matching relevant knowledge and diagnostic tools. For complex anomalies involving multiple root causes, D-Bot employs a collaborative multi-agent mechanism where multiple LLM agents (*e.g.*, CPU Expert and I/O Expert) work together. The core is an LLM-powered root-cause analysis using a tree-search approach that allows multi-step reasoning and exploration of alternative hypotheses. When the search tree branches into multiple candidate paths, the system leverages LLM-based majority voting to evaluate the plausibility of each branch. Multiple reasoning agents independently assess the intermediate evidence and potential outcomes of every path, casting votes for the most promising direction. The path with the highest consensus score is then selected for further exploration, ensuring that the reasoning process remains both diverse in exploration and robust against individual model errors.

Table 11. Existing LLM-based Agents for End-to-end Software Development

Agents	Multi-Agent	Process Model	Roles Creation	Collaboration Mode	Communication Protocol
Self-Collaboration [88]	✓	Waterfall	Pre-defined	Vertical	Memory
Low-code LLM [59]	✓	-	Pre-defined	Vertical	Direct Communication
Prompt Sapper [75]	×	-	Pre-defined	Vertical	Direct Communication
Talebirad <i>et al.</i> [246]	✓	-	Task-Adaptive	Vertical	Direct Communication
ChatDev [214]	✓	Waterfall	Pre-defined	Vertical + Horizontal	Direct Communication + Memory
MetaGPT [115]	✓	Waterfall	Pre-defined	Vertical	Direct Communication + Memory
AgentVerse [66]	✓	-	Task-Adaptive	Vertical	Direct Communication
AutoAgents [63]	✓	-	Task-Adaptive	Vertical	Direct Communication + Memory
Rasheed <i>et al.</i> [222]	✓	Waterfall	Pre-defined	Vertical + Horizontal	Direct Communication
Co-Learning [212]	✓	-	Pre-defined	Vertical + Horizontal	Direct Communication
AISD [312]	✓	Waterfall	Pre-defined	Vertical	Direct Communication
LLM4PLC [95]	×	-	Pre-defined	Vertical	Direct Communication
CodePori [221]	✓	Waterfall	Pre-defined	Vertical	Direct Communication
FlowGen _{Waterfall} [160]	✓	Waterfall	Pre-defined	Vertical + Horizontal	Direct Communication
FlowGen _{TDD} [160]	✓	Agile	Pre-defined	Vertical	Direct Communication
FlowGen _{Scrum} [160]	✓	Agile	Pre-defined	Vertical + Horizontal	Direct Communication
CodeS [298]	✓	-	Pre-defined	Vertical	Direct Communication
Qian <i>et al.</i> [213]	✓	-	Pre-defined	Vertical + Horizontal	Direct Communication
CTC [325]	✓	Waterfall	Pre-defined	Vertical + Horizontal	Direct Communication + Memory
AgileCoder [201]	✓	Agile	Pre-defined	Vertical	Direct Communication + Memory
MacNet [215]	✓	-	Pre-defined	Vertical + Horizontal	Direct Communication + Memory
Sami <i>et al.</i> [230]	✓	Waterfall	Pre-defined	Vertical	Direct Communication

4.6.4 Challenges of LLM-based Agents in IT Operations. LLM-based agents face several challenges in IT operations tasks. First, these agents are often required to monitor and analyze information from multiple system dimensions. For example, in D-Bot [323], upon receiving an alert, the assigner agent dispatches specialized expert agents to investigate potential root causes from various perspectives, such as CPU, memory, workload, and I/O behavior. This demands strong capabilities in both agent scheduling and the handling of heterogeneous data sources. Additionally, IT operations typically involve complex, iterative processes with multiple rounds of exploration. Designing more efficient planning algorithms to guide the agent through these exploratory steps is crucial to reducing computational overhead while improving diagnostic accuracy [313, 323]. Finally, most LLM-based agents focus primarily on root cause analysis, whereas other important operational tasks, such as system performance analysis and optimization, remain largely unexplored. Addressing these broader challenges will require new strategies for integrating LLM-based agents with existing operations tools, such as logging systems and performance profilers.

4.7 End-to-end Software Development

Given the high autonomy and the flexibility of multi-agent synergy, LLM-based agent systems can further tackle the end-to-end procedure of software development (e.g., developing a Snake Game application from scratch) beyond an individual phase of software development. In particular, like the real-world software development team, these agent systems can cover the entire software development life cycle (i.e., requirements engineering, architecture design, code generation, and software quality assurance) by incorporating the synergy between

multiple agents that are specialized with different roles and relevant expertise. Table 11 summarizes the existing LLM-based agents for end-to-end software development.

4.7.1 Software Development Process Model. End-to-end software development requires a well-structured workflow. In the field of software engineering, software development process models (e.g., waterfall [228], incremental model [110], unified process model [123], and agile development [44]) are used to describe methodologies for organizing software development processes. Inspired by these classic methods, the pipelines of some end-to-end software development agents are adapted from traditional process models, primarily Waterfall and Agile. Figure 13 illustrates the adapted development pipelines used by these approaches. For agents that do not adopt classic software development process models, we will discuss their collaboration mechanisms in Section 4.7.3.

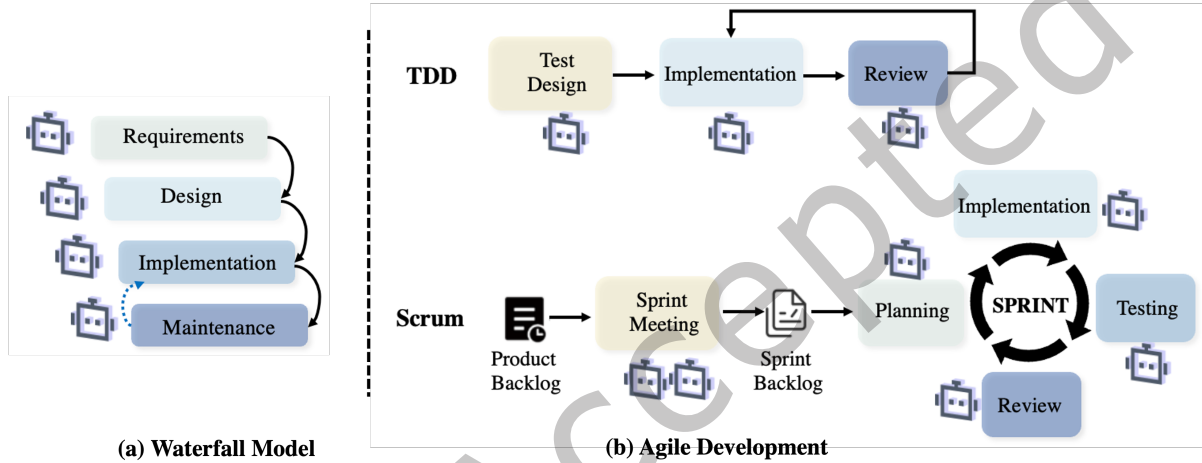


Fig. 13. Adapted Process Models Adopted by LLM-based Agents for End-to-end Software Development

Waterfall Process Model. The Waterfall model is the most popular approach among current LLM-based agent methods for end-to-end software development. The traditional waterfall process model [228] is a linear and sequential software development workflow that divides the project into distinct phases, *i.e.*, requirements engineering, design, code implementation, testing, deployment, and maintenance. Once a phase is finished, the project moves forward to the next phase without iteration. However, due to the randomness and hallucination of LLMs [125, 315], feedback mechanisms are often introduced to improve accuracy. Therefore, these end-to-end software development agents [88, 115, 160, 312] further extend the traditional waterfall process by including iterations in specific phases to ensure the high quality of the generated content. For example, the results of the testing phase might be fed back to the developer agent to revise the generated code.

Agile Development. Some works explore the potential of LLM-based agents with agile development, including Test-Driven-Development (TDD) [160] and Scrum [160, 201]. TDD prioritizes writing tests before the actual coding and fosters a cycle of writing test suites, implementing the code to pass the test suites, and concluding with a reflective phase of refinement. Scrum is an agile software development process model that breaks down software development into several sprints, achieving complex software systems through iterative updates. However, as shown in Figure 13, the actual Scrum model adopted in current LLM-based agents omits the “Daily Scrum”, which serves as a short meeting for team members to sync progress and discuss issues. This may be due to the fact that agents can share information through the memory mechanism. Experiments on function-level code generation

benchmarks show that the Scrum model can achieve the best and most stable performance, followed by the TDD model [160].

4.7.2 Role Specialization of Software Development Team. Imitating real-world software development teams, multi-agent systems for end-to-end software development often assign different roles to tackle specialized sub-tasks and collaborate throughout the software development life cycle.

Role Categories. Most end-to-end frameworks emulate real-world software development teams by assigning key roles as follows:

- *Managers.* In virtual software teams, managers serve as the team leaders and have diverse responsibilities. One of the main responsibilities is to analyze and extract user requirements, such as the Product Manager [115, 201, 246, 312] and the Manager in CodePori [221]. On the other hand, some manager roles are responsible for *task decomposition and allocation* [59, 115, 222]. For example, Rasheed *et al.* [222] proposed a framework in which the Project Planning Agent defines the scope, objectives, and development plan. In Low-code LLM [59], the Planning LLM is tasked with generating a flowchart that systematically breaks down the final task into small steps. Some studies also introduce a CEO role to assist in *software design* [214, 325]. Finally, the Scrum Master is a role specific to the Scrum agile model. In FlowGen_{Scrum} [160], the Scrum Master is responsible for summarizing sprint meetings and extracting user story lists. In AgileCoder [201], the Scrum Master provides feedback to the Product Manager to optimize the requirements list.
- *Requirements Engineers.* Requirements engineers are responsible for understanding user inputs and generating requirement documents [88, 160, 222, 230]. Typically, if managers are already assigned to decompose requirements within the workflow, the requirements engineers will not be introduced.
- *Designers.* Designers are responsible for various aspects of system design. Architectural design is the most common one, where a role such as architect or system Designer is introduced to perform high-level system design based on requirement documents [115, 160, 222, 230, 246, 312]. Prior work also introduces specialized designers for User Experience (UX) and User Interface (UI) design [246].
- *Developers.* Developers are responsible for writing code based on requirements and design specifications. In the collected works, we identify two types of developer agents. The first type is the basic developer, who is responsible for the actual coding process [88, 115, 160, 201, 214, 221, 222, 230, 246, 312, 325]. The second type is the senior developer, who provides feedback to basic developers to ensure code quality. Examples include the Senior Developer in AgileCoder [201] and the CTO in ChatDev [214, 325].
- *Quality Assurance Experts.* QA Experts can be categorized into three main types: (i) Software Testers, who are responsible for generating test code, executing tests, and providing test feedback [88, 115, 160, 201, 214, 222, 230, 246, 312, 325]. (ii) Debuggers, tasked with identifying and resolving software defects [246]. (iii) Reviewers, who enhance code quality by identifying issues through code review [214, 325].
- *Deployment Engineers.* They are responsible for formulating software release strategies, such as the Deployment Plan Agent proposed in [222].
- *Assistants.* Assistants primarily support the output of the aforementioned roles by providing feedback, summarization, and criticism. Examples include the Oracle Agent [246], the QA Agent [222], and the action observer [63].

The detailed categories of roles in existing SE agents are discussed in Section 5.2.1.

Instead of simulating the real-world development teams, some agents design their specialized agent workflow and break down roles accordingly. For example, CodeS [298] decomposes the complex code generation task into the implementation of repository, file, and method layers, and sets up the roles of RepoSketcher, FileSketcher, and SketchFiller. Co-Learning [212] and its subsequent work [213] abstract the code generation process into instruction-response pairs, thus only setting up the roles of instructor and assistant.

Role Creation. The roles in multi-agent end-to-end software development systems are either created in a predefined way or in a task-adaptive way. Most approaches predefine fixed roles and workflows through manual

design [59, 88, 115, 160, 201, 212–215, 221, 222, 230, 298, 312, 325]. In contrast, some agents only predefine a few meta-roles, which discuss and derive the actual agent roles to solve the specific problem. This role-creation approach is typically applied to general-purpose agents to handle various types of tasks, or in scenarios involving multi-turn operations where different roles are utilized in each round. For example, AutoAgents [63] designs a drafting stage that aims at determining the roles of the multi-agent group via the communication between two meta agents: the planner and the agent observer. In AgentVerse [66], a group of different roles is established through an expert recruitment stage, and the roles recruited in each round may vary. Talebirad *et al.* [246] propose a novel framework and enable an agent to spawn additional agents to the system. Such dynamic strategies aim at creating roles in a more diverse and flexible way.

4.7.3 Collaboration Mechanism in Multi-agent. Within the multi-agent systems for end-to-end software development, it is essential to schedule how each agent coordinates with the other. We then discuss the collaboration mode and the communication protocol adopted in existing agents.

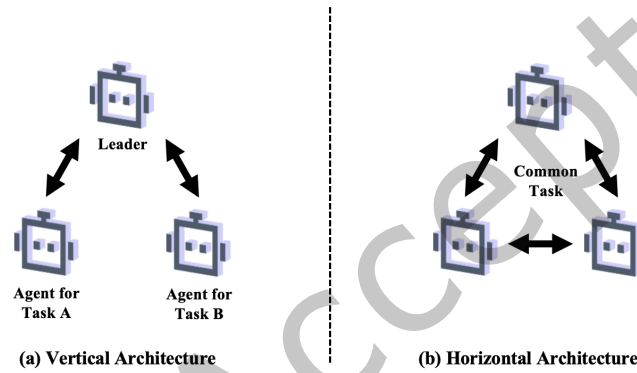


Fig. 14. Vertical and Horizontal Collaboration Architectures

Collaboration Mode. The collaboration mechanism of multiple agents can generally be divided into two types: vertical architecture and horizontal architecture. As illustrated in Figure 14, in the vertical architecture, a leader assigns tasks to different agents for execution and integrates the final results. In the horizontal architecture, the agents are treated as equals and collaborate through discussion to jointly advance the execution of the same task [190]. All the end-to-end software development agents choose to use vertical architecture in the overall pipeline. However, some of these works incorporate horizontal collaboration in some phases.

Vertical Architecture. Previous research [63, 66] suggests that vertical collaboration is preferable for tasks like software development, which produces only the final refined decision. For example, in AgentVerse [66], the Solver agent plays the role of the leader, deciding the final solution by integrating the feedback from other agents. In AutoAgents [63], the Action Observer is predefined as the leader, responsible for assigning tasks, validating the outputs of different agents, and dynamically adjusting the plan. Except for these two agents, all other end-to-end software development agents omit the role of leader, with the output of each phase directly serving as the input for the next, thus forming a linear workflow [88, 115, 201, 212–214, 221, 222, 230, 298, 312, 325].

Vertical + Horizontal Architecture. Some agents in end-to-end software development introduce horizontal collaboration into certain phases. A common approach is to arrange multiple agents at each phase, instructing them to collaboratively solve the tasks through discussion. Depending on the configuration, the collaborating

Table 12. Benchmarks for End-to-end Software Development

Granularity	Benchmarks	# Tasks	Input Scale	Output Scale	Language	Evaluated Agents
Project	SRDD [214]	1,200	Software Description (55 words)	Multiple Files	Python	[212–215, 325]
	CAASD [312]	72	Software Description (50 words)	Multiple Files	Python	[115, 214, 312]
	SoftwareDev [115]	70	Software Description (30 words)	Multiple Files	Python	[66, 115, 214]
	SketchEval [298]	19	README (421 words)	Structured Multiple Files	Python	[214, 298]
	ProjectDev [201]	14	Software Description (262 words)	Multiple Files	Python	[115, 201, 214]
Method	HumanEval [65]	164	Function Description (68 words)	Single Function	Python	[63, 66, 88, 115, 160, 201, 215, 221]
	HumanEval-ET [87]					
	MBPP [48]	974	Function Description (15 words)	Single Function	Python	[88, 115, 160, 201, 221]
	MBPP-ET [87]					

roles may include two [212–215, 222, 325] or more agents [160]. For example, in ChatDev [214], each task is participated in by two agents, who reach a consensus through dialogue. In FlowGen_{scrum} [160], agents with different roles will equally present their opinions in the sprint meeting, and in the end, the Scrum Master will summarize and extract a list of user stories. Another approach to integrating horizontal collaboration is to run multiple pipelines that use vertical collaboration. For instance, CTC [325] sets up different teams of agents to develop the same software in parallel. Teams that contribute low-quality information will be eliminated, and in the end, the software development will be completed based on the high-quality information contributed by different teams.

Communication Protocol. Within the end-to-end software development systems, agents communicate with other agents to exchange information. For those adjacent agents, the most common communication protocol is direct dialogue [63, 66, 88, 160, 214, 215, 221, 222, 312], which leverages natural language to exchange information. This approach allows for flexible expression of intent and is close to human communication. In contrast, some agents (e.g., MetaGPT [115]) structure communication by having agents exchange documents and diagrams instead of relying solely on dialogue, as pure natural language may be insufficient for solving complex tasks due to distortion in multi-turn communication. Likewise, Sami *et al.* [230] proposed a framework that defines the output format of different agents. For example, the requirements engineering agent transcribes user inputs into structured software requirements, which can be downloaded in CSV format, and the architecting agent parses the structured requirements to generate PlantUML [34]. Except for direct communication, some end-to-end software development agents integrate the memory mechanism to establish communication between non-adjacent agents in the pipeline [63, 88, 115, 201, 214, 215, 325]. For example, in MetaGPT [115], all agents have a shared information pool from which they can obtain the required information.

4.7.4 Agent Evaluation. Given the complexity of end-to-end software development, researchers further build diverse benchmarks and metrics for a comprehensive evaluation.

Benchmarks. Table 12 summarizes the benchmarks used for evaluating existing LLM-based agents for end-to-end software development. In particular, we can observe that there are still a large number (*i.e.*, 8) of papers using the function-level code generation benchmarks (e.g., HumanEval [65] or MBPP [48]) for evaluating end-to-end software development. Although these traditional code benchmarks can represent end-to-end software

Table 13. Metrics Used in Evaluating Agents for End-to-end Software Development

Category	Metrics	Used Agents
Execution Validation	Pass Rate	[88, 115, 214]
	PassK	[63, 66, 212]
	Executability	[160, 213, 221]
	# Errors	[201, 215, 325]
Similarity	SketchBLEU [298]	[212, 214, 298]
	Cosine Distance	[213, 215, 325]
Costs	Running Time	[115, 201]
	Token Usage	
	Expenses	
	#Sprints	
Manual Efforts	Human Revision Costs	[115]
Generated Code Scale	Line of Code	[115, 212, 214]
	Code Files	[213, 215, 325]
	Completeness	

development to some extent, they still involve simplified, small-scale development tasks (*i.e.*, input of short function descriptions and output of a single function, as shown in Table 12). In addition, there are five more complicated project-level benchmarks that aim at simulating the end-to-end software development, *i.e.*, SRDD [212–215, 325], CAASD [312], SoftwareDev [115], SketchEval [298], and ProjectDev [201]. The tasks in these benchmarks include more complicated and longer requirement descriptions (*e.g.*, the average length of software description in ProjectDev is 262 words), and their expected outputs are supposed to contain multiple files. In particular, the benchmark SketchEval is built upon the real-world GitHub repositories, and its input descriptions are extracted from the README file of the software. However, although these project-level benchmarks are more complex than function-level benchmarks, their overall complexity remains limited (*e.g.*, most project inputs contain no more than 60 words), failing to capture the intricacies of real-world SE challenges. Moreover, most methods are evaluated on self-constructed benchmarks, which limits the comparability between different approaches. Drawing inspiration from recent high-quality software engineering datasets [128, 293], future research can focus on constructing a high-quality benchmark to advance end-to-end software development techniques, potentially by mining existing GitHub repositories and incorporating manual annotation and filtering.

Metrics. Table 13 summarizes the metrics used for evaluating existing LLM-based agents for end-to-end software development. In fact, given the difficulty of generating a complicated program, it is possible that the generated program cannot perfectly pass the tests. Therefore, in addition to the common metrics (*e.g.*, Pass Rate or Pass@K) that execute the generated program for validation, there are multiple dimensions for assessing how existing agents perform in end-to-end software development. In particular, there are (i) the similarity metrics between the generated program and the ground truth (*e.g.*, SketchBLEU [298] measures the structure similarity), (ii) the costs of executing or generating the program, (iii) the manual efforts to further refine the generated program, and (iv) the scale of the generated program. However, existing evaluation metrics are insufficient for end-to-end software development tasks. For instance, software accuracy is merely assessed based on manually scored executability or code similarity, which may not fully capture the quality of the generated software. Furthermore, some critical evaluation dimensions, such as robustness, security, and cost, remain underexplored, limiting the impact and reliability of these approaches. Future research could focus on these fine-grained metrics. Moreover, key metrics in the agent workflow (*e.g.*, crash rate, valid iteration rounds, etc.) could also be considered to form a more systematic and comprehensive evaluation.

Table 14. Existing LLM-based Agents for End-to-end Software Maintenance

Agents	Multi-Agent	Phases						
		Preprocessing	Issue Reprod.	Issue Localization	Task Decomp.	Patch Generation	Patch Verification	Patch Ranking
MAGIS [249]	✓	×	×	Retrieval-based	×	w/ local context	Code Review	×
AutoCodeRover [316]	✓	×	×	Navigation/Spectrum-based	×	w/ cross-file context	Static Check	×
SWE-agent [288]	×	×	✓	Navigation-based	×	w/ local context	Static Check	×
CodeR [62]	✓	Plan Selection	✓	Spectrum-based	×	w/ cross-file context	Dynamic Check	×
LingmaAgent [187]	✓	Knowledge Graph Const.	×	Simulation	✓	w/ cross-file context	Static Check	×
MASAI [255]	✓	Test Template Generation	✓	Navigation-based	✓	w/ local context	Static/Dynamic Check	✓
Agentless [276]	×	Repository Tree Const.	×	Navigation-based	×	w/ local context	Static/Dynamic Check	✓
SpecRover [229]	✓	×	✓	Navigation-based	×	w/ cross-file context	Static/Dynamic Check	✓
DEIBase [307]	✓	×	×	×	×	×	Static/Dynamic Check	✓

4.7.5 Challenges of LLM-based Agents in End-to-end Software Development. LLM-based agents face several challenges in the end-to-end software development task. First, most existing LLM-based agents still follow a linear, waterfall-style workflow without true iterative or evolutionary mechanisms [88, 115, 214]. This workflow makes it difficult to handle the complexity and dynamism of real-world software development, where requirements often evolve and feedback loops are essential. Second, current agents are predominantly designed for Python or basic Web application development [115, 212, 214, 325], limiting their applicability to a narrow subset of software types. In practice, software systems span a wide range of languages, frameworks, and architectures, many of which involve complex dependencies and integration constraints. As a result, the generalizability of current approaches remains limited. Third, this field lacks standardized benchmarks and evaluation metrics for assessing end-to-end software generation. This is partly due to the open-ended nature of the task, *i.e.*, when the target software is totally unknown in advance, it is difficult to define test cases upfront. Furthermore, important aspects of software quality, such as component reusability or architectural soundness, are challenging to quantify and are currently not well-captured by existing evaluation methods. These limitations highlight the need for more adaptive development workflows, broader language and domain support, and better-defined evaluation standards.

4.8 End-to-end Software Maintenance

Software systems undergo maintenance as requirements continuously change (*i.e.*, adding, deleting, or modifying features) or unexpected software behaviors arise. In practice, users report unsatisfactory behaviors that they encounter; developers then diagnose the reported issues and modify the software to fix them. Such an end-to-end software maintenance process can be time-consuming and labor-intensive in practice, as it involves multiple phases, including understanding user-reported issues, localizing code for maintenance, and precisely editing code to address issues. Recently, there has been an increasing number of multi-agent systems aiming at automatically solving issues of real-world software projects. Table 14 summarizes the characteristics of these agents.

Framework: Figure 15 illustrates the pipeline of existing LLM-based agent systems for end-to-end software maintenance. All of the existing agents follow a common pipeline of three mandatory phases, *i.e.*, *issue localization*, *patch generation*, and *patch verification*, where different agents incorporate various strategies to tackle each phase. In addition, some optional phases can be included in the pipeline to improve performance, *i.e.*, *preprocessing*, *issue reproduction*, *issue localization*, *task decomposition*, *patch generation*, *patch verification*, and *patch ranking*.

4.8.1 Preprocessing. To better understand the whole repository, some agents first perform preprocessing to prepare prior knowledge before the entire procedure. The agent system LingmaAgent [187] constructs a knowledge graph of the entire code repository to facilitate the subsequent process of issue localization. Meanwhile,

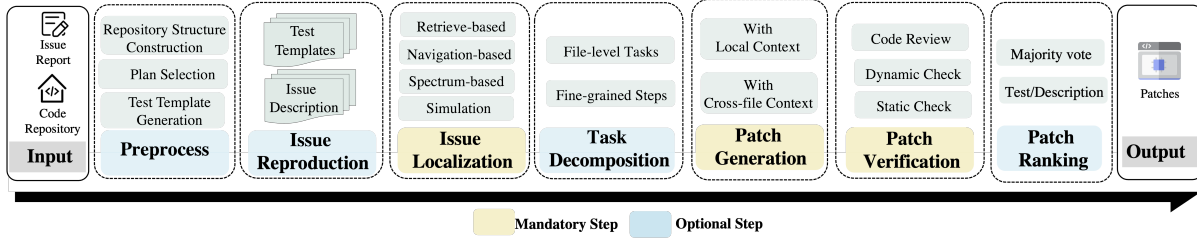


Fig. 15. Pipeline of LLM-based Agents for End-to-end Software Maintenance

Agentless [276], which is simplistic and less agentic, simply turns the whole project into a tree-like structure that demonstrates all directories and files of the repository in a hierarchical format, which facilitates the issue localization phase. In CODER [62], a manager agent first chooses a plan from several workflows pre-defined by human experts. In MASAI [255], the test template generator is used to analyze the testing setup of the repository and generate a test template with the running command, which further serves as an example for the following issue reproduction phase.

4.8.2 Issue Reproduction. A test script that triggers the unexpected behaviors users encounter is essential for issue resolution. It not only helps with issue localization but also serves as the verification criterion for patch correctness. However, in practice, users do not always provide such reproduction tests when they report issues, and such reproduction tests are often added by developers after they fix the buggy software. Therefore, some agents design the issue reproduction phase that aims at generating the test script that can trigger the unexpected behaviors encountered by users. For example, SWE-agent [288], CodeR [62], and SpecRover [229] all directly leverage the agent to generate reproduction tests based on issue descriptions when there is no existing reproduction script in issue descriptions. However, generating reproduction tests can be challenging, as the tests must be executable and ideally should fail on the buggy software version while passing on the fixed version. Therefore, to increase the success rate of issue reproduction, the multi-agent system MASAI [255] includes a two-stage approach for issue reproduction, which first investigates the test framework and existing tests for generating a sample test template (generated in the preprocessing phase) and then uses the template as a demonstration to create the reproduction script.

4.8.3 Issue Localization. Issue localization is one of the most important phases where the agents are supposed to precisely identify the code elements (e.g., classes, methods, or code blocks) that are related to issues and should be edited. Unlike fault localization discussed in Section 4.5.1, which is a task triggered by the developers during the debugging stage and involves test files that reproduce the bug, issue localization is a user-triggered process, typically only accompanied by a natural language description of the issue from the user's perspective and lacking reproducible test scripts. This characteristic poses a unique challenge for issue localization and makes it heavily dependent on the natural language understanding capabilities of LLMs. We then summarize the common localization strategies used in existing LLM-based agents.

Retrieval-based Localization. Retrieval-based localization is the most fundamental approach, where agents identify suspicious code elements based on their similarity to issue descriptions. For example, MAGIS [249] ranks all code files using BM25 [226] and selects the Top-K most relevant ones as potential issue locations. However, retrieval-based methods typically operate at a coarse granularity (e.g., file level) and can not pinpoint the exact functions or lines that need modification. To improve localization accuracy, agents often integrate additional strategies such as navigation, spectrum analysis, or simulation. However, the similarity with the issue description still plays a crucial role in these strategies, for example, by filtering out irrelevant files to narrow the search

space [276], assessing whether the localization path deviates from the description [187], or combining it with other strategies [62].

Navigation-based Localization. The navigation-based localization approach generally equips agents with a series of actions for browsing the directory structure (e.g., listing files within a directory, searching directories or files, scrolling within files, etc.), allowing agents to autonomously explore the project directory and locate specific code snippets [229, 255, 288, 316]. The only exception is Agentless [276], which directly presents the entire repository’s file and directory structure in the prompt to locate the relevant files. Another distinction between different approaches lies in their navigation strategies. For instance, SWE-agent [288] and MASAI [255] both allow the agents to autonomously determine the exploration paths. In contrast, the navigation path in AutoCodeRover [316] and SpecRover [229] is influenced by previously identified relevant code snippets, which are inserted into the prompt as context to guide further exploratory navigation. Agentless [276] employs a hierarchical approach and instructs the LLM to gradually localize files, classes, functions, and concrete edit locations.

Spectrum-based Localization. Some agents integrate spectrum-based fault localization (SBFL) [273], which assigns a suspiciousness score to code elements based on their coverage in passing and failing tests [62, 316]. The difference between these approaches lies in the source of the test suite. AutoCodeRover [316] explores spectrum-based fault localization under ideal conditions, utilizing the developer-written test cases from SWE-bench Lite as the test suite. In contrast, CODER [62] adopts a more practical approach by first generating reproduction tests and then calculating a score based on both the suspiciousness scores and the BM25 scores. The experimental results show that the issue resolution rate increases from 17.00% to 20.33% in AutoCodeRover. However, in CodeR, the localization accuracy of using SBFL alone is lower than that of combining BM25 scores, which might be attributed to differences in test script quality (human-written vs. model-generated).

Simulation-based Localization. Simulation is a special technique adopted by LingmaAgent [187] for issue localization. It applies the classic Monte Carlo Tree Search (MCTS) [58] algorithm. By recursively incorporating nodes with higher similarity with the issue, it evaluates and ranks the most relevant paths in the repository knowledge graph. The collected code is then summarized for issue localization.

4.8.4 Task Decomposition. Before generating patches, some agents decompose the task into more fine-grained sub-tasks. For instance, in MAGIS [249], its manager agent breaks down the issue into file-level tasks and delegates them to a newly-formed development team; similarly, in LingmaAgent [187], its summary agent summarizes the collected code and issue description, and then outlines the fine-grained steps for issue resolution.

4.8.5 Patch Generation. In this phase, the agents generate patches for the localized suspicious code elements. The input context for this phase typically includes the issue/task description and the suspicious code elements to be modified [249, 255, 276, 288]. In addition, some agents (e.g., AutoCodeRover [316], CodeR [62], and LingmaAgent [187]) further refine the input contexts by including relevant cross-file code contexts that are collected by retrieval APIs. Notably, SpecRover [229] provides ancillary function summaries for all collected code snippets, which reflect the high-level intent of related functions and can further assist patch generation.

4.8.6 Patch Verification. Agents further verify the correctness of the generated patches, which is challenging as the reproduction tests are not always available in practice. Therefore, agents incorporate different verification strategies.

Code Review. Some agents design a quality assurance agent to review the quality of generated patches. For example, in MAGIS [249], each developer agent is paired with a QA engineer agent to review the code change and provide timely feedback. SpecRover [229] assigns a reviewer agent to check the correctness of both the patch and reproducer test, which can not only mitigate misjudgments caused by errors in either but also provide comprehensive feedback to assist with iterative modifications.

Static Checking. Some agents (e.g., AutoCodeRover [316], LingmaAgent [187], MASAI [255], Agentless [276], and SWE-agent [288]) use static checking approaches to assess the syntactic correctness, indentation, and compatibility of the generated patch with the repository environment.

Dynamic Checking. Since the static checking cannot find the semantic violation of the patches, some agents (e.g., CodeR [62] and MASAI [255]) further perform dynamic checking by executing the reproduction test on the patch. The patch that passes the reproduction test can be considered as effectively resolving the issue. In particular, existing reproduction tests are reused (if available); otherwise, reproduction tests generated during the issue reproduction phase are used. Agentless [276] also implements a dynamic checking approach by conducting regression testing to filter out incorrect candidate patches. Notably, SpecRover [229] performs both reproduction test and regression test to guarantee the correctness of the final selected patch.

4.8.7 Patch Ranking. In the patch verification phase, code review and static checking strategies are not sufficient for checking the correctness of generated patches. The dynamic checking methods require reproduction or regression test scripts, which are not always available, and may not be able to decide whether the generated patches are correct or not. For example, the generated reproduction test may be incorrect itself, and the correct patch may edit some code snippets that can resolve the issue, but fail the existing regression tests. To tackle these problems, some agents are designed to generate multiple patches and further include a patch ranking phase to identify the optimal patch. For example, in MASAI [255], a ranker agent is responsible for ranking all potential patches based on the issue description and reproduction tests; In Agentless [276], all patches are normalized and re-ranked based on the number of occurrences with the majority voting strategy. In SpecRover [229], patches failing some tests, together with the issue descriptions, are provided to the selection agent, which then deeply analyzes the cause of the issue and chooses the best patch. DEIBase [307] is a framework that integrates multiple expert agents (e.g., Agentless [276], Moatless [33], and Aider [25]). It chooses the optimal patch from candidates generated by existing expert agents to achieve a higher resolve rate. To rerank candidate patches, it assigns an LLM-based code review committee, which takes the issue description, the relevant context, the original code, and the patched code as input and scores each candidate patch based on its analysis and explanation.

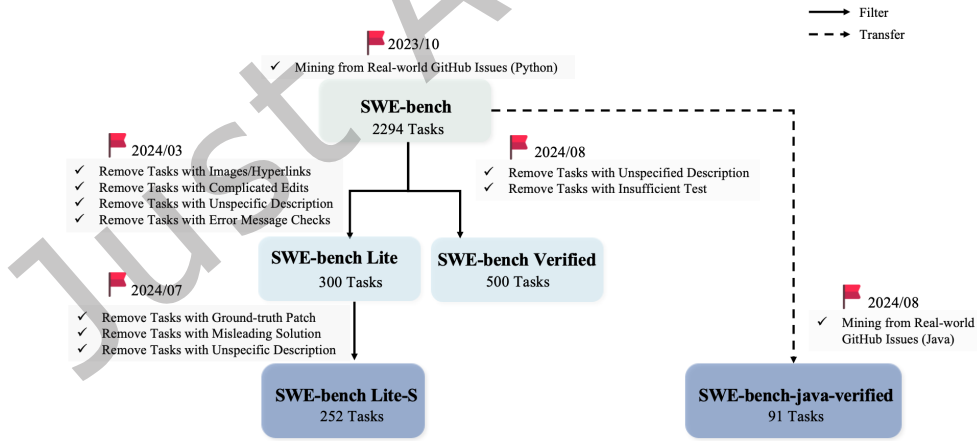


Fig. 16. Benchmark Evolution in Software Maintenance

4.8.8 Agent Evaluation. In this section, we will discuss the benchmarks used for end-to-end software maintenance tasks and analyze the performance of existing approaches.

Benchmarks. Traditional fault localization often relies on Defects4J [134], a dataset that extracts real-world code bugs from software projects, providing pre-fix and post-fix repositories, repair patches, and bug-exposing tests. While Defects4J includes real bugs, it also provides well-filtered test cases with test names, root causes, and stack traces. However, in real-world maintenance scenarios, issues are typically reported by users, who usually provide only descriptions of the problem without reproducible test scripts. Additionally, Defects4J contains only 357 bugs from five open-source projects, with 92.41% of patches modifying only a single file [239]. These limitations create a notable gap between Defects4J and real-world software maintenance in both scale and complexity.

Therefore, to evaluate how LLM-based agents tackle real-world end-to-end software maintenance, researchers build benchmarks by mining user-reported issues from GitHub, including SWE-bench [128], SWE-bench Lite [36], SWE-bench Lite-S [276], SWE-bench Verified [32], and SWE-bench-java-verified [297]. Figure 16 summarizes the evolution timeline and relationships of existing benchmarks for end-to-end software maintenance, with the “Filter” relationship referring to extracting a subset and the “Transfer” relationship referring to adopting a similar construction strategy.

SWE-bench [128] is the first benchmark for end-to-end software maintenance, which consists of 2,294 real-world GitHub issues across 12 popular Python repositories. Each task in SWE-bench includes an original text from a GitHub issue (*i.e.*, the issue description or problem statement), the entire code repository, the execution environment (*i.e.*, Docker environment), and validation tests (*i.e.*, tests that are hidden from the evaluated agents). A typical evaluation process is as follows: the agents under evaluation will take the issue description and the buggy code repository as input and attempt to locate and fix the issue, with passing the validation tests serving as a success indicator. The *resolve rate*, which is defined as the ratio of resolved issues to total issues, is the common metric used to evaluate the agents’ performance on these end-to-end software maintenance datasets.

However, the full SWE-bench benchmark can take too much evaluation costs, and it contains particularly difficult or problematic tasks [276], which can underestimate the evaluation of LLM-based agents. Therefore, researchers have dedicated considerable manual efforts to extract subsets of SWE-bench that feature high-quality tasks with acceptable cost, reasonable difficulty, self-contained information, informative issue descriptions, and sufficient evaluation tests. For example, SWE-bench Lite [36] is a subset of SWE-bench that manually removes tasks requiring complicated edits (*e.g.*, editing more than one file), and the tasks including images or hyperlinks; SWE-bench Lite-S [276] further removes tasks that contain exact patches, misleading solutions, or insufficient information in the issue descriptions; similarly, SWE-bench Verified [32] removes cases with unspecified descriptions or insufficient tests.

In addition to issues in Python projects, some researchers transfer the build process of SWE-bench and construct benchmarks in other popular languages (*e.g.*, Java). SWE-bench-java-verified [297] is constructed by collecting Java projects from GitHub and the Defects4J dataset. Through rigorous validation and filtering, it eventually includes 91 issues across 6 Java projects. Experiments based on the SWE-agent and DeepSeek-Coder achieve the best resolve rate of 9.89%, fixing 9/91 issues.

Performance Comparison. SWE-bench Lite is the most widely used evaluation benchmark in practice. The reported resolve rate data on SWE-bench Lite are illustrated in Figure 17, which are collected from the original papers. It is worth noting that most of these agents are based on the GPT-4 series models, such as GPT-4 [62, 187, 249, 288, 316], and GPT-4o [255, 276, 307]. Therefore, the model capabilities do not significantly contribute to the differences in performance. The only exception is SpecRover [229], which primarily uses the Claude-3.5-Sonnet model [43].

In conjunction with the phases and techniques presented in Table 14, we have drawn some interesting observations:

- Pure autonomous localization methods do not necessarily lead to better performance. A typical example is SWE-agent [288], which allows for fully autonomous issue localization by designing an interaction interface between the agent and the computer. However, it performed the worst among all the agents on SWE-bench Lite.
- Agents that employ dynamic patch verification and patch ranking strategies generally achieve higher resolve rates. Among the top five agents with the highest repair rates, Agentless [276], MASAI [255], SpecRover [229], and DEIBase [307] all adopt both dynamic patch verification (*i.e.*, test execution) and patch ranking strategies. CodeR [62] does not use patch ranking, but it still adopts dynamic code checking.
- Approaches adopting traditional fault localization strategies, such as Agentless [276], have surpassed many agentic approaches, which place higher demands on evaluating the effectiveness of complex agent designs.

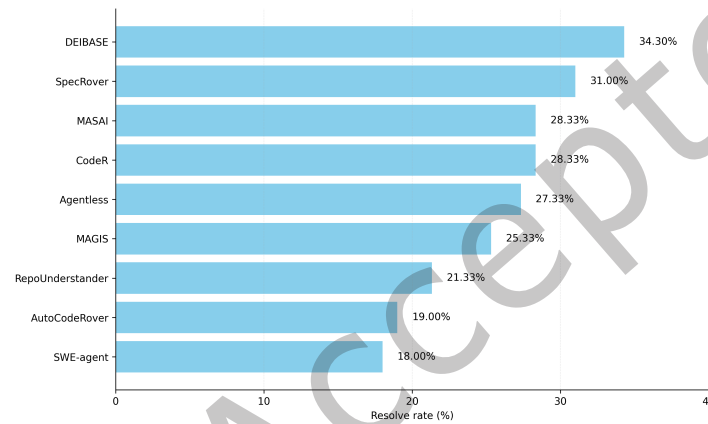


Fig. 17. Resolve Rate of LLM-based Agents on SWE-bench Lite

4.8.9 Challenges of LLM-based Agents in End-to-end Software Maintenance. LLM-based agents encounter several key challenges in the end-to-end software maintenance task. First, issue descriptions submitted by users are often unstructured and may include multimodal content such as links and screenshots. Effectively interpreting such inputs requires multimodal understanding and the ability to extract relevant information from external sources, which are absent in current LLM-based systems [128]. Second, in end-to-end maintenance workflows, the ability to reproduce user-reported issues is critical for both issue localization and patch verification. However, existing approaches lack reliable mechanisms for ensuring issue reproducibility [249, 276, 316]. Third, current LLM-based agents often rely on limited strategies for patch verification, such as static checking or code review heuristics [249, 288, 316], which are insufficient for guaranteeing correctness. While some agents execute tests to validate patches, they still face ambiguity in determining correctness. In some cases, the agent-generated tests may be incorrect; in others, the agent may fail to generate any test [62, 288]. Some LLM-based agents rely on existing regression tests, but they may break due to unrelated code changes, leading to false negatives even when the patch is semantically correct [229, 276]. These challenges highlight the need for improved capabilities in understanding, reproducing, and verifying real-world software issues.

5 Analysis from Agent Perspective

This section organizes the collected papers from the perspective of agents. Specifically, Section 5.1 summarizes the components of existing LLM-based agents for SE; Section 5.2 focuses on existing multi-agent systems for SE by summarizing their roles, collaboration mechanisms, information flows, and real-world applications; and Section 5.3 summarizes how humans coordinate with agents for SE.

5.1 Agent Framework

Based on the common framework of LLM-based agents [107, 261, 275], this section first summarizes the common paradigms of the planning, perception, memory, and action components in existing LLM-based agents for SE, then discusses the impact of different foundation models on the effectiveness of agent systems.

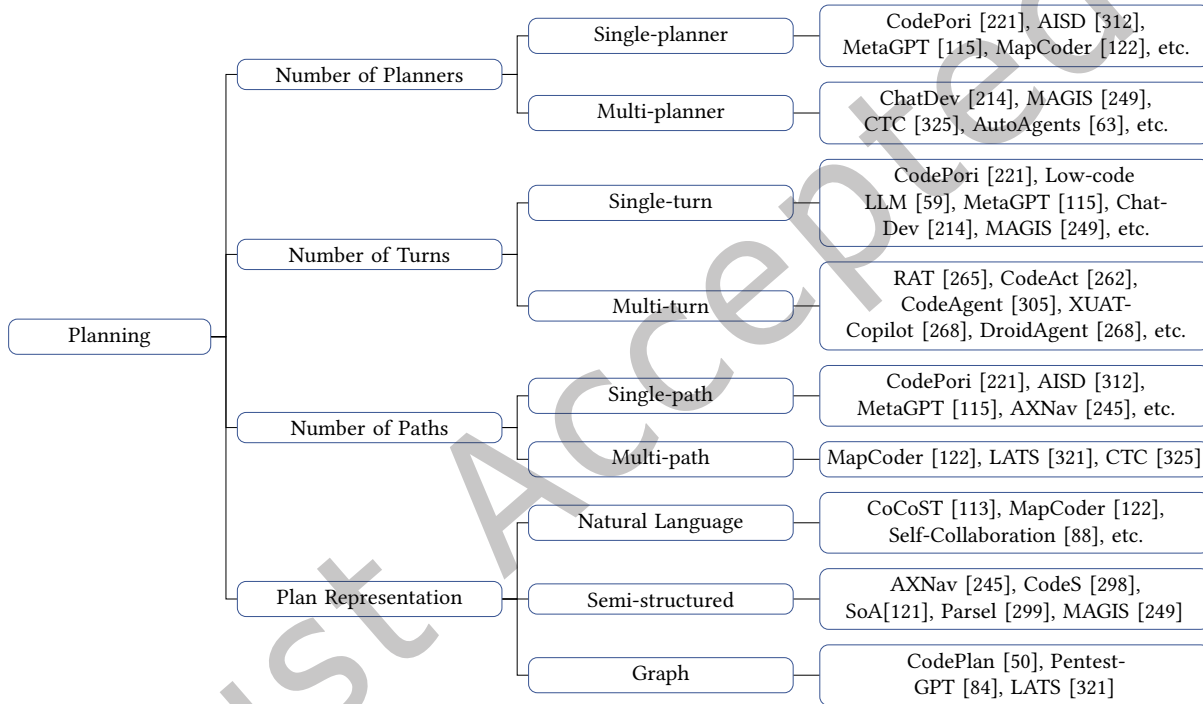


Fig. 18. Taxonomy of Planning Strategies in LLM-based Agents for Software Engineering

5.1.1 Planning. In SE, intricate tasks such as development and maintenance activities necessitate the orchestrated efforts of various agents through multiple iterative cycles. Therefore, planning is an essential component for agent systems by meticulously delineating task sequences and strategically scheduling agents to ensure the seamless progression of the SE process. Figure 18 presents the taxonomy of the planning components in existing LLM-based agents for SE.

Single Planner vs. Multiple Planners. In LLM-based agent systems, planning is typically handled by a specialized agent [59, 88, 97, 115, 122, 132, 221, 245, 292, 299, 312] or as a core responsibility of the sole single agent [95, 113, 262, 265, 266, 268, 305, 321]. Some works use the function-calling interface [16] provided by GPT-3.5 [19] or GPT-4 [20], handing over the planning task to high-performance models [305]. However,

given the pivotal role that planning plays in influencing subsequent action steps, some works incorporate a collaborative approach among several agents to further enhance the accuracy and practicality of the plans formulated [63, 132, 201, 214, 222, 249, 325]. Single-planner strategies incur lower overhead and demonstrate broader applicability, but are more susceptible to the risks of hallucination. In contrast, multi-planner approaches benefit from mutual correction among different agents and the integration of specialized knowledge from diverse perspectives, thus reducing factual errors [91, 267]. However, they also introduce additional token consumption and time overhead [214], making them more suitable for complex tasks such as competitive coding [132], end-to-end software development [63, 201, 214, 222, 325], and end-to-end code maintenance [249].

Single-turn Planning vs. Multi-turn Planning. The fundamental planning strategy is to craft a holistic plan from the very beginning, and then proceed to implement it in successive rounds [59, 88, 95, 97, 113, 115, 122, 132, 201, 214, 221, 222, 249, 265, 298, 299, 305, 312, 325]. In contrast, some SE agents have adopted a ReAct-like [290] architecture, which implements a multi-turn planning mechanism wherein the next-round actions will not be determined until receiving the environmental feedback from the previous round. This form allows for dynamic revision and expansion of the plan, enabling it to adapt to more flexible task scenarios, such as issue resolution [255, 321], iterative code generation [262, 305], mobile app testing [245, 268, 292], among others [266]. Single-turn planning requires as much knowledge as possible prior to planning to alleviate hallucinations. In addition, it faces the challenge of performing fine-grained task decomposition for complex tasks. On the other hand, multi-turn planning allows for plan adjustments based on environmental feedback, thereby improving fault tolerance. However, for complex tasks, the gradually accumulating trajectories may exceed the model's limited context window, leading to hallucinations and forgetting [120]. Consequently, some memory mechanisms are necessary to prevent the agent from deviating from its initial objectives.

Single-path Planning vs. Multi-path Planning. Most LLM-based agents use single-path planning strategies, *i.e.*, they plan and execute tasks in a linear manner [50, 59, 84, 88, 95, 97, 113, 115, 121, 160, 201, 214, 221, 222, 245, 249, 255, 262, 265, 266, 268, 292, 298, 299, 305, 312]. However, agents inherit the randomness from the backbone LLMs, leading to fluctuations in task decomposition. One approach to address this issue is to design a multi-path planning strategy, which instructs the agents to generate or simulate multiple plans, and select [321], switch [122], or aggregate [325] the optimal paths for execution. The single-path strategy is constrained by the inherent stochasticity of LLMs [204], undermining its reliability. In contrast, the multi-path strategy endows the agent with a certain degree of trial-and-error capability, thus increasing the probability of completing the task [122]. However, exploring multiple paths also introduces additional time and token costs [321], limiting its practicality in certain situations.

Plan Representation. The plan can be exhibited in different forms, including natural language descriptions, semi-structured representations, or graphs. Most agents describe the plan in natural language, especially as a list of procedural steps [59, 88, 97, 113, 122, 132, 265] or features to be implemented [115, 160, 201, 312]. Furthermore, some agents implement semi-structured plans [95, 121, 245, 298, 299]. For example, AXNav [245] represents the action list in JSON format. Some code-generating agent systems directly output the code skeleton [95, 121, 298, 299] or present the plan as executable code [249], which can be seen as a special plan form in SE tasks. Graphs can also be implemented as a special plan form, which facilitates the expansion and traceability of execution paths [50, 84, 321].

Challenges in Planning. Although agents applied in software engineering have explored various planning strategies and representations, planning still faces numerous challenges in practical applications:

- *Hallucination.* LLM-based models can suffer from hallucination issues when generating plans, especially in an insufficient context. For example, agents may generate inaccurate plan steps operating non-existent methods or variables. Furthermore, during iterative planning, there is a risk of redundant steps or repetitive sequences, leading to loops in the paths planned by LLMs.

- *Limited reliability in complex tasks.* Current planning primarily relies on the inherent reasoning capability of LLMs, which results in limited reliability in complex tasks. For example, experiments of CoCoST [113] have shown that in scenarios with higher complexity of individual function codes, the contribution of planning significantly decreases. In addition, there is still a gap between agent-generated plans and human-provided plans for complex tasks. For instance, experiments of Flows [132] have demonstrated that providing just a small segment of a human-designed plan can lead to a substantial performance increase in competitive programming tasks (from 26.9% to 74.5% and from 47.5% to 80.8% on novel problems).
- *Lack of fine-grained evaluation.* Although planning plays an important role in agents, its effectiveness is primarily reflected through performance on the task results. The validity of planning itself has not been sufficiently assessed, which is often crucial for task success. Future work should focus on fine-grained evaluation of planning, such as cost and the effectiveness of planning steps.

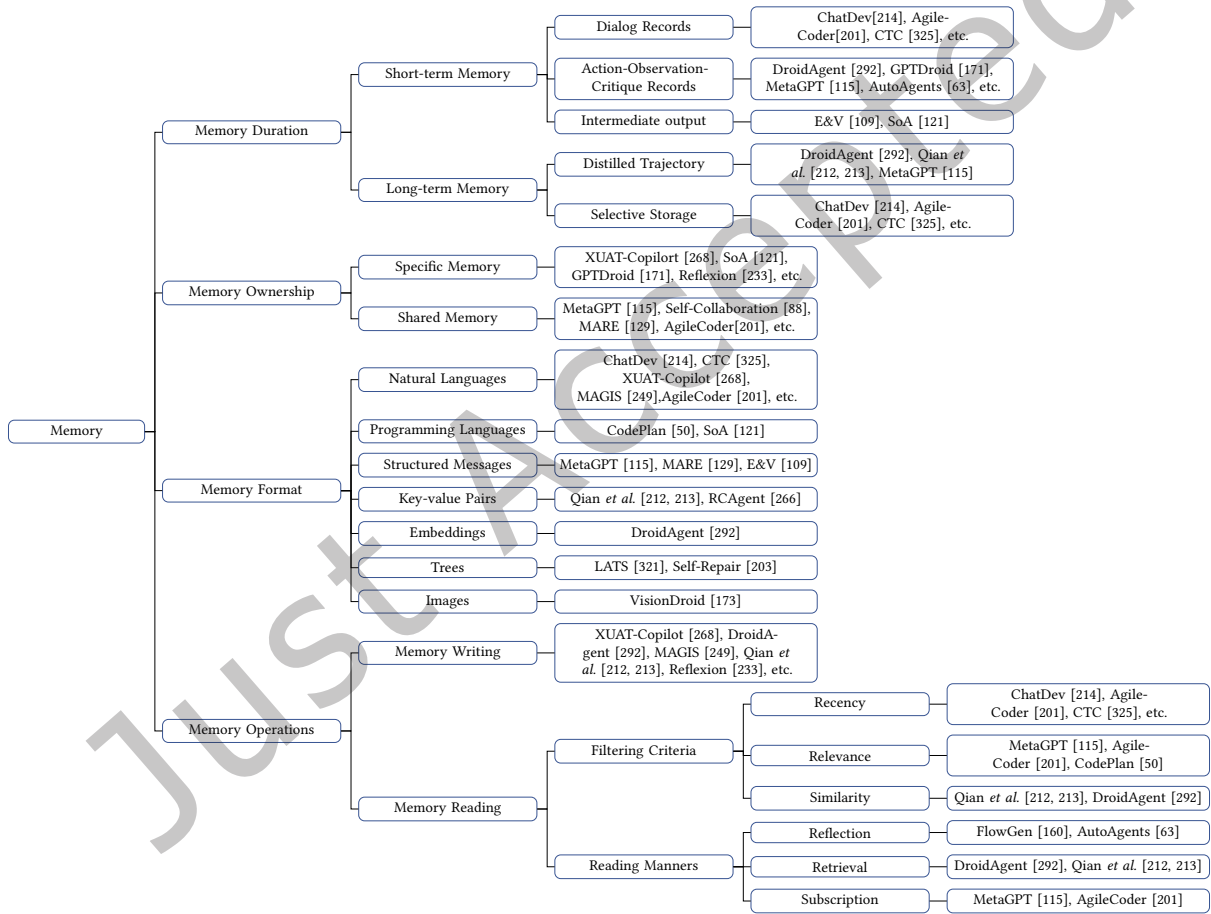


Fig. 19. Taxonomy of Memory Design in LLM-based Agents for SE

5.1.2 Memory. The memory component is a pivotal mechanism responsible for storing the trajectories of historical thoughts, actions, and environmental observations, enabling agents to sustain coherent reasoning

and address intricate tasks. In SE, complex development and maintenance tasks generally necessitate agents conducting iterative revisions, wherein historical intermediate information, *e.g.*, generated code and testing reports, significantly impact integrity and continuity. We then detail the implementation of memory mechanisms in SE from four perspectives: memory duration, ownership, format, and operation. Figure 19 presents the taxonomy of the memory components in existing LLM-based agents for SE.

Memory Duration. Inspired by human memory systems, agent memory can be classified into short-term memory and long-term memory based on the memory duration.

Short-term memory, also known as working memory [105], is integrated into agents to enhance their ability to sustain trajectories of the current ongoing task and is frequently used when multi-turn interactions are involved. Specifically, it enhances the planning module by offering a reference to the historical explorations for the given task, thereby aiding the agent in tracking task progress and preventing the recurrence of the same mistakes through trial-and-error experiences. In SE, short-term memory primarily stores three types of information. The first type is *dialog records*, which is generally used to memorize the pure dialog history among agents and is typically in the form of history summary [63, 268] and multi-turn instruction-response pairs [201, 214, 215, 325]. It is straightforward to implement and can offer a thorough and detailed historical record of the task-solving process. However, the weakness is that the dialog history can be lengthy and contain irrelevant and redundant information. The second type is *Action-Observation-Critique records*. While dialog history concentrates on the thoughts and responses among agents, some works highlight the interaction between agents and the environment by memorizing the action-observation sequences. The critique information is also retained in case certain reflection mechanisms are introduced [292]. This pattern has been adopted in SE tasks that necessitate iterative feedback from the environment, *e.g.*, mobile app testing [171, 173, 268, 292], wherein operations on widgets in each turn should be memorized to facilitate the next-turn decision-making, and iterative code generation [50, 63, 115, 233], wherein the previous editing, execution, or debugging history serves as important information for code revision. The third type is *intermediate outputs*. Some agents merely store the outputs of previous turns in short-term memory to avoid overrunning the limited space, as well as being overly influenced by irrelevant or inaccurate chat history. For example, in E&V [109], only intermediate analysis results are summarized to avoid inconsistency with the previously generated outputs. In SoA [121], to implement a self-organized framework, each agent is equipped with memory that stores the self-generated code and unit tests. These intermediate results allow delayed test execution and code modification for agents in different layers, facilitating hierarchical collaborative code generation.

Long-term memory, on the other hand, is used to memorize valuable experiences of historical tasks. Similar to how humans draw on experience from previously completed tasks, long-term memory contributes to the planning module by providing records of historically relevant or similar tasks, which can serve as a reference for reasoning and solving the current unseen task. However, the entire task execution trajectory may involve extensive context, and given the limited memory space, it can be challenging to store it all completely. As a result, distilling techniques have been proposed, *e.g.*, trajectory summarization [115, 292] and shortcut extraction [212, 213]. These distilled records retain the task execution process in a more concise manner, thereby alleviating the burden on limited memory and prompt windows. Another approach to save long-term memory space is to selectively store vital data of each task, *e.g.*, the final results [63, 201, 214, 215, 325], reflections [63, 233], and action-observations [266, 292, 321]. Compared to complete historical trajectories, these data highlight the pivotal trace information, which can still retain the effects and feedback of previous tasks.

Memory Ownership. In agent systems, the memory module can be designed to serve specific agents or to serve all agents. Based on its ownership, we categorize memory into specific memory and shared memory.

Specific memory is an agent mechanism designed specifically for a limited group of agents. This type of memory has strict pre-defined usage regulations, only storing and serving specific agents in the workflow [50, 63, 109, 115, 121, 122, 171, 173, 201, 212–215, 233, 249, 266, 268, 292, 321, 325]. For example, in SoA [121], each agent is

equipped with individual memory for storing its own generated code fragments and unit tests, which will be used to evaluate the correctness of the final code and provide feedback to the agent for modification.

Shared memory, on the other hand, serves all agents by maintaining the record of their outputs and offering essential historical data. In most cases, shared memory serves as a dynamic information exchange hub in the intricate SE environment, which is akin to the traditional blackboard system [78]. Generally, information stored in the shared memory is the intermediate results of previous phases, hence the agents from subsequent phases can obtain necessary information in a more convenient manner [88, 115, 129, 201, 216]. Representative work like MetaGPT [115] introduces a shared message pool, which saves artifacts from different agent roles, *e.g.*, the product requirement documents from the product manager. Another typical application of shared memory is to store comments in a decentralized debate scenario. Specifically, FlowGen_{scrum} [160] simulates the Sprint Meeting by providing a shared buffer, storing the problem and the discussion comment of all participating agents from which the product manager could extract a list of user stories.

Memory Format. In this section, we elaborate on the format of data stored in the memory. In SE tasks, the most commonly used storage formats include natural languages, program languages, structured messages, key-value pairs, embeddings, and trees.

- *Natural Languages.* LLM-based agents solve tasks specified in natural language, which is thus the most fundamental and prevalent data format in memory [63, 88, 160, 171, 173, 201, 214, 215, 233, 249, 268, 325]. The advantage of raw natural language is that it allows for a more flexible storage of trajectories, thereby enhancing the universality. Moreover, raw natural language can better preserve the integrity of the original dialogue, which minimizes the loss and distortion of essential information.
- *Programming Languages.* Some agents directly store the generated code for subsequent utilization [50, 121]. For example, SoA [121] is a hierarchical code generation framework with each agent focusing on a single function implementation. It stores the generated function code and unit tests in memory for testing, modification, and aggregation.
- *Structured Messages.* In this format, memory is organized as a list of messages with multiple attributes. The strength of this format is that it allows data to be stored in a structured manner, making it more convenient for indexing and processing. Moreover, it can store vital metadata of the message, *e.g.*, task type, message source and destination, so it is easier for agents to trace and subscribe to required messages, making it commonly used in *shared memory*. Representative works like MetaGPT [115] and MARE [129], both wrap the artifacts of each agent as informative messages, involving the original content, instruction, task name, sender, receiver, etc. In E&V [109], intermediate results of previous turns will be summarized and stored in JSON format.
- *Key-value Pairs.* In this format, information received from the agents is stored in an external memory, with a key extracted for the agents to query required history memories [212, 213, 266]. More specifically, in Co-Learning [212], shortcuts are extracted from the trajectories to construct two key-value databases: the solution-to-instruction database for the instructor, and the instruction-to-solution database for the assistant. RCAgent [266] stores the whole observation body in a key-value store, retaining a snapshot key for agents for query details.
- *Embeddings.* In this format, the memory is embedded into a vector, which can help retrieve the most relevant task experiences. Representative work like DroidAgent [292] embeds the textual history into vectors and stores them in an external embedding database. Compared to text similarity retrieval, it can further provide semantic similarity retrieval.
- *Trees.* Some approaches construct a tree or graph for memorizing, especially in scenarios requiring flexible extension or path tracing. For example, in LATS [321], the task-solving process is modeled into a tree with each node representing a state with the instruction, the action, and the observation, and then an extended Monte

Carlo Tree Search algorithm can be integrated. Similarly, Self-Repair [203] proposes a repair tree that stores multiple generation-feedback-repair paths.

- *Images*. Leveraging the capability of multimodal LLM, VisionDroid [173] memorizes the testing history, including both textual descriptions and screenshots.

Memory Operations. We categorize operations on memory into two main sections: memory writing and memory reading.

Memory Writing. The purpose of memory writing is to store essential information in the memory. Information stored in memory is usually the raw task execution trajectories [173, 201, 214, 215, 325]. However, considering that the raw task trajectories might be lengthy, distilling approaches have been proposed to retain a more informative summary in the memory [109, 115, 212, 213, 233, 249, 268, 292]. For instance, in XUAT-Copilot [268], dialog and action history are stored in working memory as summarized texts. Moreover, Co-Learning [212] proposes a novel distilling approach by first constructing a task execution graph and then extracting shortcuts linking non-adjacent solution nodes, which can serve as solution refinement paths for future tasks. These distilling strategies can generally be handled by adding an additional dialogue turn per round [109, 233, 292], which does not introduce significant overhead compared to other multi-turn dialogue processes. On the contrary, previous research suggests that by leveraging the more concise information stored in memory, the agent can retrieve key trajectories using a smaller context window, ultimately helping to reduce token consumption [212, 249]. On the other hand, the limited memory storage and prompt window size result in finite memory records. When overflow occurs, some records must be forgotten. For example, in Reflexion [233], the past experiences are stored in a sliding window with a maximum number of 3 to avoid exceeding the prompt window. Additionally, low-quality and rarely-used data also consume memory storage space. Previous research [212] sets a threshold to filter out experiences with limited information. Further, an elimination mechanism based on the usage frequency is introduced to exclude rarely-used experiences [213]. These memory elimination strategies are all implemented through simple logical scripts rather than agents, which do not introduce significant overhead.

Memory Reading. Memory reading aims at obtaining the required task history and experiences from the memory module. Besides directly providing the raw memory to the agent [88, 115, 171, 173, 201, 214–216, 249, 325], researchers prefer using three methods for obtaining relevant memory: reflection, retrieval, and subscription. *Reflection* refers to extracting pivotal experiences from the extensive trajectory memory [63, 160]. For example, in AutoAgents [63], a dynamic memory mechanism is designed to instruct an agent to extract insights from long-term memory that will serve the current action. In FlowGen_{scrum} [160], the product manager summarizes the comments collected from all agents and extracts a list of user stories to implement. In the *retrieval* manner, the memory is retrieved based on its text or semantic similarity with the current tasks [212, 213, 292]. For example, in Co-Learning [212], the reasoning module uses the prompt as a query to retrieve similar shortcuts from the constructed experience pool, which will serve as examples to facilitate future reasoning. In DroidAgent [292], the past tasks and widgets with similar GUI state embeddings are retrieved by comparing the cosine similarity. Finally, the *subscription* mechanism is chiefly used in shared memory. It permits agents to directly obtain required information according to their roles, without additional interaction costs with other agents, thus improving efficiency. Representative works include MetaGPT [115] and AgileCoder [201], both adopting this kind of publish-subscribe mechanism.

Moreover, to determine whether a historical record should be integrated into the current task, some common filtering criteria have been adopted, including *recency* [50, 115, 171, 201, 214, 249, 268, 292, 325], *relevance* [50, 115, 201], and *similarity* [212, 213, 292]. Agents integrate the most relevant and similar task experiences to provide the best reference for the current task. Additionally, the preference and weight of these factors vary across different works. In DroidAgent [292], the planner agent considers the 20 most *recent* task summaries and the 5 most *similar* task knowledge items. In MAGIS [249], the agent uses the *most recent* summary of a code file to identify

differences. In MetaGPT [115] and AgileCoder [201], agents retrieve only *relevant* messages from shared memory based on their roles.

Challenges in Memory. The memory mechanism helps alleviate the issue of the limited context window in LLMs and is crucial for building a shared knowledge base as well as ensuring consistency in the decision-making process of different agents. However, the practical application of the memory mechanism presents several challenges:

- *Abstraction level of information.* It is non-trivial to determine the abstraction level of the information stored in memory. Storing full trajectories risks excessive context, while overly summarized data may lose crucial details. Moreover, different types of information may require different levels of abstraction. For example, global decisions can be more abstract, whereas implementation requires a more specific code context.
- *Context matching.* This challenge lies in determining the required information in memory and the timing to conduct information retrieval. Excessive inclusion of irrelevant context or omission of necessary context can both significantly impact decision accuracy.
- *Lack of fine-grained evaluation.* Similar to the planning module, the evaluation of the memory mechanism remains inadequate. Except for a few works that verify the effectiveness of its memory mechanisms via ablation experiments [63, 212, 213], most works merely evaluate the final task results. Inspired by the evaluation of general-purpose agents [317], future work could focus on evaluating memory as an independent module with some numerical metrics, such as the accuracy in answering historical questions and memory-related costs.

5.1.3 Perception. Existing LLM-based agents for SE primarily adopt two perception paradigms: textual input perception and visual input perception.

Textual Input. Text can flexibly express the intent, information, and knowledge. In SE, the majority of historical data, *e.g.*, documentation, code, and issues, is stored in the textual form. This alignment with the strengths of LLMs in processing natural language makes textual input the predominant form of perception for agents for SE. Textual input in existing agents can be further categorized into natural language input (*i.e.*, domain-specific instructions and auxiliary information collected from the environment) and programming language input (*i.e.*, the code context). For example, in NL2Code tasks [115, 214], user requirements and function descriptions are provided as the instructions to agents. But in some code-related tasks, *e.g.*, software testing [42, 80] and debugging [57, 144, 216, 309], the target code can also be provided for analysis. Specifically, in repository-level tasks such as issue-resolution [62, 288, 316], repository-level fault localization [216], and code edits [50], only a portion of code snippets are provided due to context length limitations, with further inspections achieved through navigation in the code repository.

Visual Input. Images represent a two-dimensional medium for storing information. In traditional SE scenarios, there is also a portion of data presented in image form, *e.g.*, UML diagrams [139] and UI pages. In current SE agents, visual input is widely used in GUI testing tasks. Traditional GUI testing methods typically rely on text input, *i.e.*, view hierarchy files to extract widgets [155, 172, 241]. However, this method is insufficient, as it may lose the structural semantics of the GUI when converting screenshots to text. In addition, the raw view hierarchy often contains an excessive number of tokens, and its redundancy can interfere with the agent's decision-making and reduce accuracy [173]. On the other hand, using visual input (*i.e.*, screenshots) helps agents locate accessible widgets more precisely. Just as humans use their eyes to capture visual information, these agents integrate visual models to process and understand image data. For example, XUAT-Copilot [268] uses the SegLink++ model [247] for detecting bounding boxes and a ConvNeXts model [174] for text recognition. AXNav [245] utilizes the Screen Recognition model [314] to analyze screenshot pixels from iOS devices and predict bounding boxes, labels, text content, and the clickability of UI elements. In addition, VisionDroid [173] employs multi-modal LLMs to process both the text and image information. By leveraging visual recognition information, these agents can gain a more intuitive understanding of the current GUI state, assess whether the

goal has been achieved, and accordingly predict its next action. However, relying solely on screenshots is also limited due to the overlapping elements and the concise nature of GUI text and icons [173], which makes it difficult for agents to accurately and comprehensively extract information. Moreover, since the output of LLMs remains text-based, subsequent processes still rely on text. As a result, existing works all incorporate both vision input (*i.e.*, the screenshots) and text input (including view hierarchy files, detected elements, instructions, etc.) to achieve better effectiveness [173, 245, 268].

5.1.4 Action. The action component of existing LLM-based agents for SE primarily involves using external tools to extend their capabilities beyond the interactive dialogue typical of standalone LLMs. Figure 20 summarizes the tools used in these agent systems.

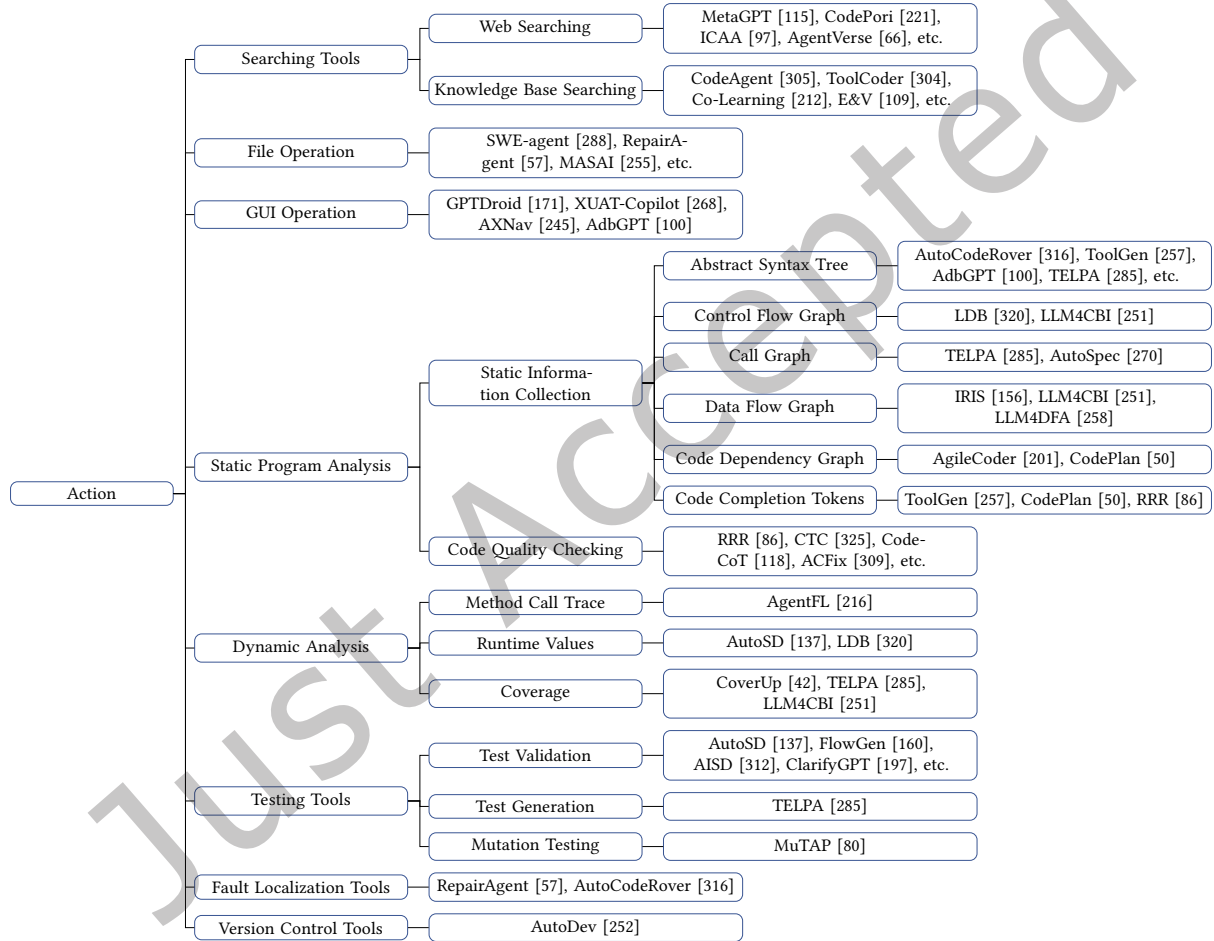


Fig. 20. Taxonomy of Action Components in LLM-based Agents for SE

Searching Tools. In SE, agents frequently use some lightweight RAG techniques to retrieve relevant information (*e.g.*, documentation or code snippets) that can aid in task completion. The retrieved information is either used to enrich the current context (*e.g.*, supplementing API documentation or background knowledge) or to

provide relevant examples for in-context learning. Currently, SE agents primarily acquire information from two sources: the Web and local knowledge bases.

Web Searching. Online search engine tools use community and tutorial websites to offer programmers accurate and practical suggestions based on shared experiences and Q&A. When faced with gaps in specific domain knowledge, programmers distill their needs into a query and use existing search engines (e.g., Google, Bing, WikiSearch) to find the necessary information. Inspired by these practical experiences, some SE agents also retrieve ancillary information from the Web [97, 98, 113, 115, 206, 262, 265, 274, 283, 304, 305]. For example, some agents [304, 305] use DuckDuckGo [2] to search the relevant content, such as APIs. Paranjape *et al.* [206] employ SerpAPI [10] and extract answer box snippets when they are available or combine the Top-2 search result snippets. He *et al.* [113] query Google and then extract pertinent information to construct prompts for LLMs. Information retrieved via web search is generally used to supplement the context rather than serve as in-context learning examples.

Local Knowledge Base Searching. Besides using a web searching tool to externally collect the information from the Web, it is also common for existing agents to retrieve relevant knowledge from the self-established knowledge base, which includes documents [97, 168, 223, 242, 266, 274], codebases [50, 57, 97, 109, 265, 266, 274], or historical experiences [206, 242, 292]. There are various retrieval methods, including similarity-based retrieval, keyword-matching search, and model generation. *Similarity-based retrieval* is the most traditional RAG approach, primarily consisting of *sparse word-bag* and *dense text embedding* methods. Both approaches vectorize code or documents and calculate the similarity between the query and segments in the knowledge base to retrieve relevant information. The sparse word-bag approach (e.g., BM25 [304, 305]) vectorizes text while partially preserving its semantics. Dense text embedding models, such as dual-encoder models, encode text into embedding vectors and compute their cosine similarity [86, 97, 212, 242, 265, 266, 274, 292]. On the other hand, *keyword-matching retrieval* searches relevant context by matching key terms. For example, it may use keywords as keys to query a key-value database [266, 292], directly search for elements in a code repository using keywords [57], or leverage source code analysis tools to match keywords [109]. The final approach is to retrieve knowledge through *model generation* [122, 127]. For example, MapCoder [122] treats the model itself as the data source. It instructs the retrieval agent to generate similar (*problem, plan, code*) examples as in-context learning demonstrations for the planning agent. In summary, different works equip agents with various sources and types of knowledge bases and employ different RAG methods to retrieve relevant context. By leveraging the agent's in-context learning capabilities, these approaches mitigate hallucinations and improve the accuracy of model-generated outputs.

File Operation. As SE activities frequently access massive files, especially for the code repository and documentation, it is common for agent systems [57, 98, 137, 156, 229, 252, 255, 288] to use file operations including shell commands (e.g., Linux shell) or the code utilities (e.g., Python *os* package) for file browsing, file adding, file deleting, and file editing. For example, for file browsing, agents open files based on their paths, scroll through the contents, and jump to specific lines.

GUI Operation. For SE activities related to software with a GUI, it is necessary to enable various GUI interaction operations for agent systems [100, 171–173, 245, 268], including clicking, text input, scrolling, swiping, returning, and termination. In particular, for UI element identification, they use visual and text recognition models (e.g., SegLink++ [247], Screen Recognition [314], and ConvNeXts [174]), dump (e.g., Android UIAutomator [14]), or parse the UI view hierarchy [100, 171]; then they simulate the testing environment using virtual Android devices (e.g., Genymotion [18], VirtualBox [23], and pyvbox [22]) and autonomously execute or reply actions through tools such as Android Debug Bridge [13] to mimic user interactions. These actions enable agent systems to test in various GUI environments.

Static Program Analysis. Static program analysis tools are widely used in agent systems for SE tasks, as they can provide more rigorous code features (e.g., data-flow and control-flow) for LLMs, which can help agents

better understand the program and tackle the relevant tasks. Existing agents primarily collect the following static information.

- *Abstract Syntax Tree (AST)*. AST is a common representation to describe the syntactic structure of the source code and is widely used by agents. In particular, the collected ASTs help agents extract syntactic elements (e.g., class names, method names, and variable names) [42, 50, 50, 86, 100, 109, 201, 255, 257, 270, 285, 305, 316] and identify dependency among these code elements [50, 182, 201, 270, 285]. Tree-sitter [39] and ANTLR [207] are AST parsing tools that are widely used in existing agent systems [50, 57, 201, 216, 255, 305, 309].
- *Control Flow Graph (CFG)*. LDB [320] uses CFG to divide a program into multiple blocks, making it easier to track intermediate variables with the help of the debugger. LLM4CBI [251] calculates the cyclomatic complexity based on a CFG that represents failed tests and accurately identifies high-complexity blocks of the code. These complicated code blocks would be regarded as the targets for program mutation.
- *Call Graph (CG)*. TELPA [285] constructs a method CG, extracting all call sequences that reach uncovered target methods. Based on these sequences, new test cases are generated to ensure comprehensive coverage of previously untested methods. AutoSpec [270] treats loops as nodes as well, constructing an extended call graph, and it then traverses the CG from the bottom up to generate specifications.
- *Data Flow Graph (DFG)*. IRIS [156] constructs a data flow graph to assist taint analysis, which helps detect security vulnerabilities. LLM4CBI [251] performs data flow analysis to output a list of the most complex variables defined and used in the failed test, which guides test generation. LLM4DFA [258] leverages data flow analysis for dataflow-related bug detection.
- *Code Dependency Graph (CDG)*. Agents such as AgileCoder [201] and CodePlan [50] build a Code Dependency Graph for the entire codebase. The graph represents complex relationships between code blocks (e.g., call relationships, inheritance, and import dependencies) and enables the accurate extraction of task-relevant context information (e.g., error traceback path for repair) within constraints of a limited prompt length. In addition, by dynamically maintaining the CDG, agents can perform incremental analysis in a more efficient way.
- *Code Completion Tokens*. In code generation tasks, it is common for agents [50, 86, 161, 182, 257] to use language servers (e.g., Jedi [6] and EclipseJDTLS [3]) to collect candidate tokens at the certain position. In particular, candidate tokens returned by language servers often pass the syntactic violation (e.g., only defined variable names are returned), which can effectively alleviate the hallucinations of standalone LLMs.

Besides, static analysis tools are also widely used by agent systems to check code quality, e.g., syntactic correctness checking, code format checking, code complexity checking, vulnerability detection, and specifications checking. The checked results can then provide feedback or additional hints for agents to further improve code quality. In particular, existing agents [86, 94, 118, 252, 325] use compilers or interpreters (e.g., GCC or Python) for syntactic correctness checking; existing agents [95, 305] use Black [1] and nuXmv [61] for code format checking; the agent in [251] uses OClint [9] and srcSlice [200] for code complexity checking; existing agents [251, 309] use static analysis tools such as Frama-C [4] and Slither [99] to detect vulnerabilities; the agent in [270] uses static tools (e.g., Frama-C) to verify the satisfiability and sufficiency of generated specifications.

Dynamic Analysis. In addition to static analysis, existing agents also use dynamic analysis tools to monitor program execution and collect runtime behaviors for agents. For example, AgentFL [216] uses the *java.lang.instrument* package [31] to record all method call traces during the execution of failed tests, which can facilitate more accurate fault localization. Some agents [137, 320] mimic manual debugging to set breakpoints, so as to capture runtime values for variables. The runtime values can be integrated into the prompt along with requirements to aid in defect localization. Besides, coverage also serves as important feedback for whether each code element is executed by tests or not. For example, prior agents [42, 251, 285] leverage tools such as SlipCover [209], Pynguin [181], and Gcov [17] to collect the coverage information.

Testing Tools. Test cases validate whether the software behaviors violate the specifications, and it is common for agents in SE to invoke testing tools for software validation, test generation, and mutation testing.

- *Software Validation.* Validating the software with test execution frameworks (e.g., PyTest, unittest, or JUnit) can reveal the runtime errors and test failures, which are widely used in existing agent systems [57, 66, 67, 69, 98, 113, 115, 118, 119, 121, 122, 127, 132, 137, 144, 145, 160, 176, 193, 197, 201, 203, 206, 208, 225, 229, 231, 233, 250, 252, 255, 259, 262, 263, 274, 280, 284–286, 289, 296, 305, 312, 316]. The revealed execution violations can serve as feedback for agents to improve programs; otherwise, the absence of execution violations can signal the correctness of programs (e.g., proving that a plausible patch has been found for program repair agents).
- *Test Generation.* Although an LLM itself has promising capabilities of directly generating test code, traditional test generation tools provide complementary benefits as they are good at generating high-coverage tests in a cost-efficient way. For example, some agents [285] use automated test case generation tools (e.g., Pynguin [181]) to generate an initial set of unit test cases. Besides, for some domain-specific languages (DSLs), using existing tools to generate tests is simpler. For example, 3DGen [94] converts informal specifications into executable code (i.e., a binary format parser written in formally verified C code) through a DSL called 3D [243]. To test the generated 3D programs, 3DGen invokes an external tool called SMT-solver Z3 [82] to produce test cases.
- *Mutation Testing.* Some agents [80] use mutation testing tools (e.g., MutPy [411]) to evaluate the sufficiency of test cases, as killing mutants (i.e., exhibiting different behaviors on the mutated program than the original program) indicates the fault detection capabilities of tests. The mutation testing results can further serve as feedback for agents to enhance the tests iteratively.

Fault Localization Tools. Agent systems [57, 316] can invoke traditional fault localization techniques, especially spectrum-based fault localization tools (e.g., GZoltar [60]) to localize suspicious code elements. For example, RepairAgent [57] invokes GZoltar to get the suspiciousness score of each code element (i.e., the probability of being fault).

Version Control Tools. Version control systems manage the changes of various files in a repository, such as changes in code, configuration files, or documentation throughout the software development process. Some agents [164, 182, 252] that manage an entire repository often leverage version control tools.

Overhead of Tools. The overhead of tool invocation is closely related to the tool type and the task complexity. For instance, method-level code generation tasks typically take only a few seconds [304], and thus tend to use code execution tools with lower overhead. In contrast, system-level tasks, such as mobile app testing, fault localization, and end-to-end software development, can incur time costs ranging from minutes to hours [100, 115, 136, 171, 201, 214, 242], and can incorporate more time-consuming tools such as those for static analysis or fault localization. Besides, current studies generally report the overall time overhead of agents without breaking down the overhead of tool invocations [57, 95, 115, 137, 201, 216, 242, 257, 296, 309], which may be attributed to two reasons: first, tool invocations are often integrated into the reasoning process and they are generally treated as a combination; second, the time overhead of commonly used tools, such as code execution and web search tools, is relatively small and can be neglected [320]. For tools that may incur significant time overhead, common practices include limiting their processing time [171, 187, 304] or the number of invocations [187, 245, 288]. For example, ToolCoder [304] limits the search delay to 0.6 seconds; SWE-agent [288] limits the number of search results to 50, and Lingma Agent [187] restricts the number of search iterations to 600 and the maximum search time to 300 seconds in the MCTS-Enhanced Repository Understanding stage.

5.1.5 Foundation LLMs. In this section, we discuss the relationship between the LLM-based agents and their foundation LLMs.

A basic observation is that due to the rapid evolution of foundation LLMs, current agent systems are not tailored to any specific LLMs. Instead, they are built upon a set of abstract capabilities that an LLM is expected to

possess, thereby improving generality and adaptability. In practice, LLM-based agent systems often require the following capabilities from their foundation LLMs:

Basic Instruction-following, Planning, and Multi-turn Dialogue Capabilities. For LLM-based agents that accomplish tasks purely through conversational collaboration [47, 68, 129, 214], the foundation LLMs require to possess fundamental instruction-following, planning, and multi-turn dialogue (*i.e.*, memory) capabilities. Since these capabilities are core features of most LLMs, these LLM-based agents can generally operate with a variety of foundation LLMs. However, GPT-series models remain the predominant choice due to their broad accessibility and consistently strong performance. It is also worth noting that planning can be enhanced through the CoT strategy, which, to some extent, alleviates the demand for the inherent planning ability of LLMs. In addition, some agents explicitly enhance CoT and heavily depend on CoT to boost their performance [118, 265].

Tool-use Capability. In scenarios requiring autonomous tool invocation, the foundation LLM must understand tool specifications, suggest appropriate calls, and incorporate the resulting outputs into its reasoning process [129, 274, 288, 305]. For instance, CodeAgent [305] introduces five programming tools and relies on the foundation LLM to invoke them effectively for interacting with software artifacts.

Open-source Accessibility. Some LLM-based agents require fine-tuning the foundation LLMs to enhance performance, adapt to custom tasks, or intervene in their decoding process [145, 257, 284, 304, 306], which restricts implementation to open-source LLMs. For instance, ToolCoder [304] trains the foundation LLM to generate special tokens for API retrieval, while ToolGen [257] masks non-existent API tokens during model decoding. Such requirements inherently limit these systems to open-source LLMs.

Long Context Window. LLM-based agents solve problems through multi-turn interactions with the environment, which often accumulates substantial contextual information. Although memory management techniques can alleviate the dependence on extremely long context windows [212, 233, 266], the foundation LLMs' context window size and ability to comprehend complex contextual information remain critical for solving complex tasks. For example, in CodeS [298], even the largest available context window (*i.e.*, 200K tokens) might be insufficient for repository-level tasks due to the extensive size of code bases, documentation, and discussions. Other works on LLM-based agents have also reported errors caused by insufficient context length of base LLMs [201, 274, 289].

Stronger Multi-step Reasoning Capability. For tasks that involve deep exploration over multiple turns (*e.g.*, end-to-end software development and maintenance), the foundation LLM must excel at maintaining coherent long-term context and planning subsequent steps based on historical trajectories. A typical example is that all end-to-end software maintenance agents are built on state-of-the-art closed-source models [229, 276, 288], such as GPT-4, GPT-4o, and Claude-3.5-Sonnet, highlighting the strong dependence of such tasks on high-performance foundation LLMs.

In principle, the foundation LLMs can be replaced as long as they meet the agent's fundamental requirements aforementioned. However, although LLM-based agents enhance the capabilities of foundation LLMs, the increasing complexity and variability of the tasks handled by such agents also place new demands and challenges on the foundation LLMs. In some cases, performance gaps between foundation LLMs cannot be fully bridged through the architecture and design of LLM-based agents [98, 113, 188]. For example, Self-Refine [188] performs well with GPT-series models but struggles with Vicuna-13B, which often fails to produce feedback in the required format, repeats previous outputs, or generates hallucinated conversations. Therefore, in practice, it is recommended to employ foundation LLMs with capabilities comparable to those employed in the original experiments.

5.2 Multi-agent System

Based on our statistics, 59.7% of existing agents for SE are multi-agent systems. These systems benefit from the division of specialized roles and coordination among agents, which effectively addresses the complexity of

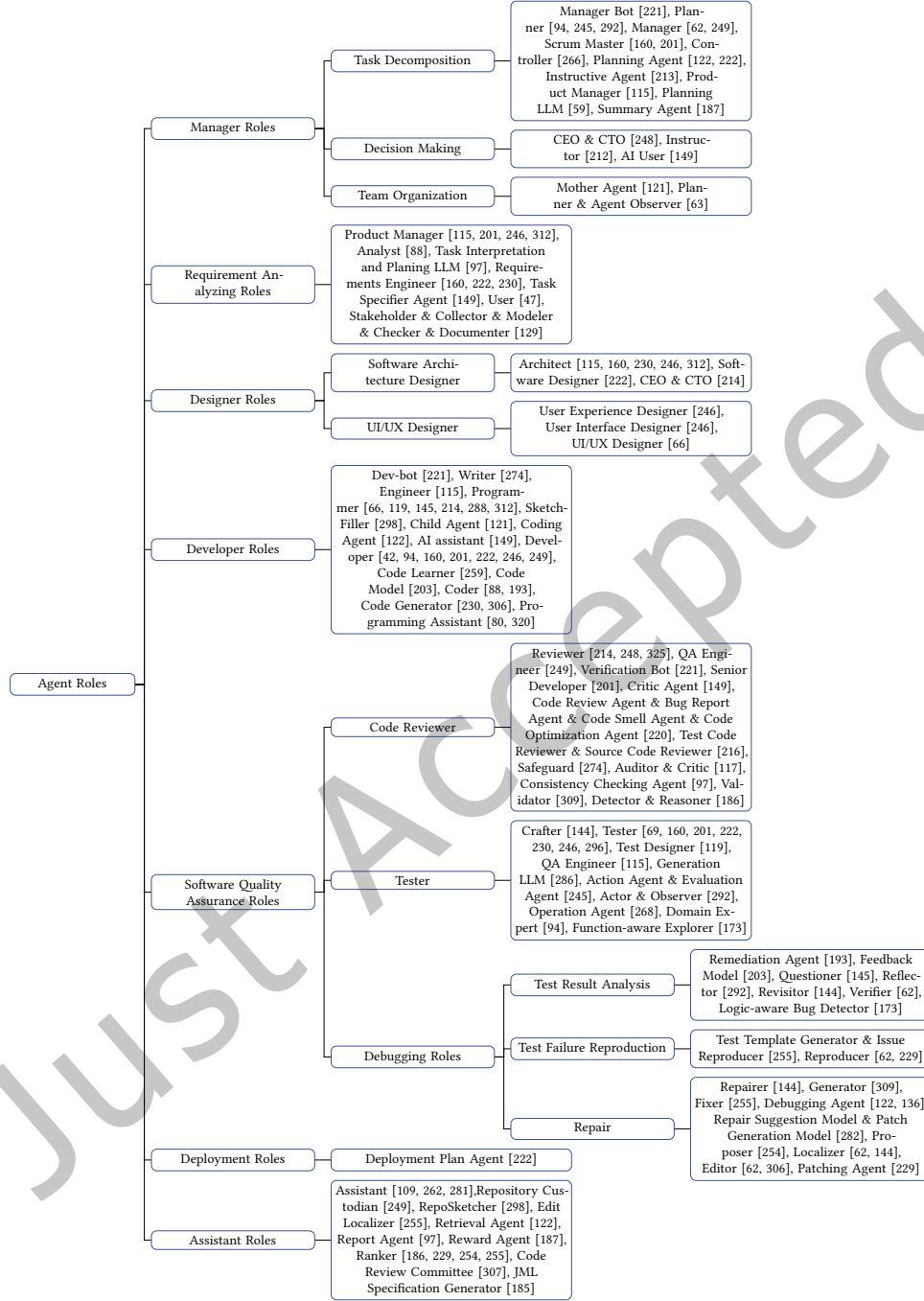


Fig. 21. Taxonomy of Agent Roles in LLM-based Multi-agent Systems for SE

SE tasks, particularly for end-to-end activities spanning multiple phases. This section provides an overview of existing multi-agent systems for SE, with a focus on their agent roles and coordination mechanisms.

5.2.1 Agent Roles. In agent systems, role-playing strategy is commonly used to embed expert personas into the prompts of agents and elicit the relevant professional knowledge. Specifically, role assignment mainly delineates duties, available actions, attributes, and constraints of roles. It enables agents to specialize in their corresponding tasks. Theoretically, both single-agent and multi-agent systems can employ role-playing strategies. However, according to statistics, among the total of 50 single-agent works we have collected, only 12 works have utilized a role-playing prompt [42, 69, 80, 109, 136, 156, 185, 262, 281, 288, 296, 320], which are expressed in a simple task-centered manner (*i.e.*, “You are a/an [TASK] expert/assistant/professional”). Besides, the roles used by these 12 single-agent works are covered by the roles in multi-agent works. Therefore, the roles discussed in this section can encompass all role types of current LLM-based agents in the SE field, whether single-agent or multi-agent. Figure 21 summarizes common agent roles in existing LLM-based agent systems for SE.

Manager Roles. Manager roles (such as CEO, commander, and controller), serve as the leaders of multi-agent teams. These roles are responsible for making decisions, planning, task decomposition and assignment, and overseeing team coordination.

Task Decomposition. To enhance the overall system performance, managers break down a project into manageable sub-tasks and draw up a guiding plan for developers, testers, or repairers to execute [62, 122, 201, 213, 221, 222, 245, 249, 266, 292]. They analyze problem statements, facilitate discussions on issues among various agent roles [160], review design documents submitted by designers [115], summarize the global repository information [187], and incorporate related information gathered by assistants. Subsequently, they produce a specific task list or implementation blueprint, which may be presented in either natural language or as a structured workflow [59].

Decision Making. Another task of managers is to orchestrate team collaboration and provide further guidance for task execution. For example, the CEO and CTO in CodeAgent [248] communicate with staff and make high-level decisions. Similarly, the instructor in Co-Learning [212] and the AI user within the CAMEL system [149] provide instructions to working agents.

Team Organization. Finally, in scenarios involving dynamically derived agents, managers play a role in assembling the team. This role is primarily designed for flexibly deciding the constitution of the agent team, *i.e.*, what roles are included in the team. The main benefits of including such roles are to flexibly optimize costs and better meet project demands. For instance, SoA [121] sets the mother agent, which generates new mother or child agents and designates concrete tasks (*e.g.*, unimplemented functions) to them. AutoAgents [63] includes a planner agent and an observer agent, which collaborate to assemble a team for particular tasks. The planner agent is responsible for assigning existing LLM agent roles or generating new ones, while the observer agent assesses and reviews the relevant roles. These roles are represented in a structured JSON format, encapsulating details such as name, description, available tools, suggestions, and prompts to guide agent behaviors.

Requirement Analyzing Roles. These roles are primarily responsible for analyzing software requirements, such as translating vague and preliminary user concepts into a coherent and structured format. Existing agents [88, 97, 115, 160, 201, 222, 246] include such roles (*e.g.*, product manager [312] or task specifier [149]) to identify key requirement elements and intended objectives for a precise and organized requirement document, which may range from an elaborated task or function description [149] to a formal software requirement specification [115].

In addition, some agents further design more fine-grained roles for requirements analysis. For example, Elicitrion [47] incorporates a set of *User* agents to identify diverse user requirements by mimicking user perspectives and conducting interviews for exploring potential user needs; MARE [129] uses a requirements engineering team (*i.e.*, stakeholder, collector, modeler, checker, and documenter) to produce requirements specifications. The requirements engineering process is segmented into four sub-tasks corresponding to specific roles, seamlessly

transitioning rough user ideas to precise requirement specifications. Sami *et al.* [230] employ a two-stage requirements generation approach by first instructing an agent to generate user requirements and then using another agent to prioritize user stories.

Designer Roles. Designer roles take input information on requirements (such as detailed task descriptions and use cases) and shape the software architecture and system integration. The most typical design roles are the *system architects*, who are responsible for conceptualizing and defining the high-level structure of software, *e.g.*, the *software architect* role in agents [115, 160, 214, 222, 230, 246, 312]. They create a design document that serves as a blueprint for the subsequent stages of development, and the design document can be presented in various forms, including natural language descriptions, structured formats (*e.g.*, JSON for listing project architecture files), and graphical representations (*e.g.*, class diagrams and sequence flowcharts [115]). Moreover, some approaches will assign UI/UX designers to craft the visual and interactive interface [66, 246].

Developer Roles. Developers take a vital role in software development and maintenance activities, which is one of the most common roles (*e.g.*, also called programmer or coder) in existing agents [66, 115, 121, 122, 149, 193, 214, 221, 230, 246, 274, 298] for tasks involving code generation. In accordance with software design schemes, task plans provided by other agents, or user requirements, the developer roles generate or finalize code at various levels (*i.e.*, from function to file and even project levels). In addition, the developer roles also engage in the code refinement process, which refines their previously generated code [88, 119, 145, 160, 203, 249, 259]. Furthermore, the developer roles can be instructed to meet more customized standards, such as elucidating their work through supplementary docstrings or adhering to particular coding criteria [201, 312].

Software Quality Assurance Roles. Agent systems include roles dedicated to software quality assurance, similar to real-world QA teams. These roles typically encompass code reviewers, testers, and debuggers, each focused on checking and improving software quality.

Code reviewers are responsible for identifying potential software quality issues by statically inspecting the software without execution. For example, some agents [66, 214, 221, 248, 249, 309, 325] include such roles to review generated code or patches; AgileCoder [201] and CAMEL [149] include the roles such as senior developer or critic agent to offer suggestions for enhancement; the agent in [220] sets up code review agent, bug report agent, code smell agent, and code optimization agent to access code quality from different aspects; AGENTFL [216] sets test code reviewer and source code reviewer to summarize code behaviour to help fault location; in addition, some agents [97, 117, 274] include such roles (*e.g.*, the auditor agent and the critic agent in GPTLens [117] and the consistency checking agent in [97]) to detect the vulnerability or implementation issues.

Software testers are widely incorporated in multi-agent systems to write corresponding test cases or scripts for software verification [115, 119, 144, 160, 201, 229, 230, 245, 246, 268, 286, 292]. For example, the tester agent in multi-agent systems [119, 144, 160] generates test cases based on relevant code skeletons or patches, requirement documents, existing tests, or rationale for the test. There are two special testing scenarios: one is *GUI testing*, in which the tester agent generates operational action sequences [173, 268, 292], and the other is *reproduction tests* in end-to-end maintenance tasks, where issue reproduction serves as a crucial phase for localizing the issue and checking the correctness of the generated patch [62, 229, 255].

Debuggers help diagnose test failures or unexpected software behaviors. A typical application scenario of debugging roles is to analyze test reports and determine the correctness of the program. [62, 144, 145, 173, 193, 203, 229, 259, 292]. For example, the remediation agent in TGen [193] and the feedback module in Self-Repair [203] are similarly designed to analyze the test failure reports and relevant faulty code to provide explanations and suggestions. Furthermore, debugging roles can directly generate patches based on the detected bugs in some approaches [122, 229, 254, 255, 282, 309].

Deployment Roles. Deployment roles are responsible for creating deployment plans to release the software to the production environment or end-users. Rasheed *et al.* [222] assign a Deployment Plan Agent to conduct this task.

Assistant Roles. Assistant roles primarily provide assistance for other agents. For example, the repository custodian in MAGIS [249], the RepoSketcher in CodeS [298], the edit localizer in MASAI [255], and the Context Retrieval Agent in SpecRover [229] is designed to enhance the comprehension of the target repository architecture for the team; in addition, MapCoder [122] uses the retrieval agent to facilitate memory recall; ICAA [97] introduces the report agent to convert natural language responses into formatted bug reports. The Reward agent [187] measures the possibility that particular code segments contribute to a given issue. Ranker is also a common role category, which is designed to choose the optimal results [229, 254, 255].

5.2.2 Collaboration Mechanism. The collaboration mechanism is essential for multi-agent systems, which can significantly impact the effectiveness and costs of the entire system. In particular, the collaborative mechanisms of existing multi-agent systems for SE tasks can be categorized into four types: layered structure, circular structure, star-like structure, tree-like structure, and mesh structure. Figure 22 illustrates each structure.

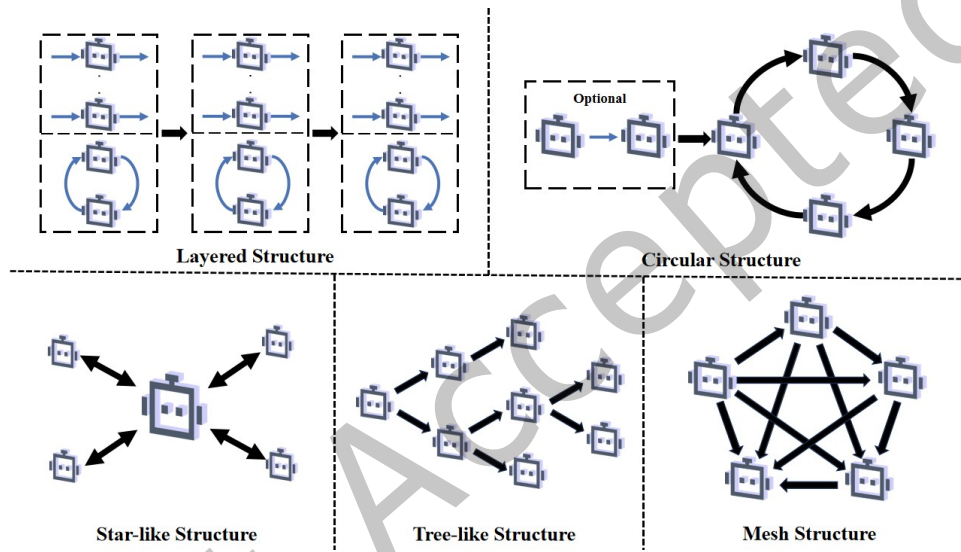


Fig. 22. Multi-agent System Collaboration Mechanisms

Layered Structure. It is a hierarchical structure, where tasks are decomposed into several sub-stages and each is assigned to a specific agent or a group of agents selected from the agent pool. Agents between different stages collaborate sequentially, *i.e.*, they receive intermediate results from agents in the previous stage as input and produce their processed data to agents in the next stage. For example, the workflow within many agents [59, 97, 173, 216, 222, 229, 230, 258, 282, 286, 306, 307, 312, 316] is a simple chain, where each agent focuses on its own sub-task and only interacts with adjacent agents. In addition, agents can also refer to the message produced by the previous non-adjacent agents [115, 122, 298]. In the sequential workflow, each sub-task can also be handled by a group of agents [129, 132]. In [214, 215, 248, 325], each sub-task is solved by the conversation between two agent roles. FlowGen [160] and AgileCoder [201] incorporate even more agents in a single stage. In addition to interactive collaboration, another scenario involves agents within the same layer working in parallel to offer their solutions. These solutions are then combined and passed down to the next layer. For example, GPTLens [117] employs several auditors to present possible vulnerable functions individually in the generation stage. AgentForest [151] incorporates the majority voting mechanism. DyLAN [175] formulates the LLM-agent collaboration structure into a multi-layered feed-forward network.

Circular Structure. This structure typically manifests as multi-turn dialogues between two roles or integrates the feedback mechanism within the overall collaborative processes among multiple agents. The *dual-role* setup is typically implemented as a generation-validation style loop between two agents [119, 145, 149, 186, 189, 193, 203, 212, 259, 309], in which one agent is tasked with the primary function, such as generating code snippets or patches, while the other agent provides validation feedback, including static analysis results, test outcomes, and improvement suggestions. For example, in the INTERVENOR framework [259], the code learner initiates the process by generating the initial code and subsequently engages in iterative repairs guided by the suggestions from the code teacher. In the *multi-role* setup, the pipeline is usually a collaborative loop with iterative feedback and refinement [221, 233, 245, 292]. For example, DroidAgent [292] designs a GUI testing loop including a Planner for task decomposition, an action module with the Actor and the Observer for execution, and a Reflector for providing task reflection and summarization to the Planner.

Star-like Structure. This structure is a centralized structure, where a central agent or system serves as the pivot to interact with other agents. For example, the controller agent in the RCAGENT [266] framework can invoke other expert agents as a kind of tool when necessary. The commander in AutoGen [274] coordinates with the writer and the safeguard separately, to craft code and ensure safety. XUAT-Copilot [268] adopts the operation agent as the core, to receive the judgment from the inspection agent and invoke the parameter selection agent to help the action planning. AutoDev [252] designs a scheduler agent that uses several different kinds of scheduling algorithms (*i.e.*, Round Robin, Token-Based, or Priority-Based) to determine the collaboration order and manner of other agents. MacNet [215] also tests the star topological structures.

Tree-like Structure. In this structure, agents are derived gradually with the breakdown of tasks, with each agent focusing on a single task at a specific level. For example, in SoA [121], the mother agent can dynamically spawn new mother or child agents for code generation, thereby forming a tree-like collaboration structure. In MacNet [215], tree structure is one of the tested topological structures to organize the collaboration of multiple agents.

Mesh Structure. The mesh structure allows communication channels between agents to form a complex network, enabling them to send messages to the target agent along the network and achieve more flexible communication. For example, 3DGen [94] designs an agent system based on the group chat mechanism from AutoGen [274], which allows multiple agents (*i.e.*, the planner agent, 3D developer agent, and domain expert agent) to control the procedure through inter-agent conversation, *e.g.*, choose which agent to send messages to. In MacNet [215], different agents can collaborate in a mesh structure, which means they can communicate with any other agents seamlessly.

Performance Issues and Solutions. The collaboration structures mentioned above may all encounter performance bottlenecks as the scale increases. Currently, most works address this issue by fixing the number of agents to match the task scale. However, for works that can dynamically derive agents [63, 66, 121, 249], increasing the number of agents can lead to significant performance issues. One way to address this challenge is to limit the number of derived agents. For example, AutoAgents [63] and AgentVerse [66], which adopt a star-like structure, can theoretically derive an unlimited number of agents but are limited to deriving at most four agents in actual code-related tasks. In SoA [121], which uses a tree structure, the depth of the tree is limited to 2. Another solution is to limit the task scale. For example, SoA is applied to method-level code generation tasks; MAGIS derives developer agents based on the number of suspicious code files located in SWE-bench, where the average number of files to be modified is 1.7 [288]. Moreover, a prior study [215] has shown that the comprehensive performance on popular natural language understanding and code generation task benchmarks (*e.g.*, MMLU, HumanEval) will reach the performance saturation regardless of collaboration structure. Therefore, balancing the number of agents and the task scale is crucial for designing multi-agent systems.

5.2.3 Information Flow. In this section, we discuss the information flow in multi-agent systems. Building on the aforementioned collaboration structures of multiple agents, we primarily focus on the most fundamental collaboration unit, which is the information exchange between two agents. Generally, there are two patterns for agents to transfer information, which are unidirectional transfer and bidirectional chat. Figure 23 illustrates these two communication patterns.

- *Unidirectional Transfer.* This is a typical data-driven communication pattern, *i.e.*, the next agent will take the data produced by the preceding agent, even without being aware of the preceding agent [47, 59, 62, 63, 97, 117, 119, 121, 122, 137, 144, 151, 173, 175, 186, 187, 193, 201, 221, 229, 230, 245, 246, 254, 255, 258, 259, 268, 277, 282, 286, 298, 299, 307, 309, 312, 316]. This is the most widely used pattern since it allows each agent to maintain individual context, facilitating decoupling and combination. For example, the planner or manager can focus on task decomposition, and the generated plan steps will be iteratively handed over to roles such as Developer and QA Agent, serving as their individual contexts in each round to form the top-down development pipeline [201, 312]. Depending on the implementation, the data produced by the preceding agent can be directly fed into the next agent or stored in a shared space (such as long-term memory [214, 215, 266] or shared memory [88, 115, 129]) for subsequent agents to retrieve.
- *Bidirectional Chat.* In this pattern, agents transfer information through chatting [66, 94, 132, 149, 189, 203, 212, 213, 249, 274]. With short-term memory serving as the carrier of information, these agents can share a common history of dialogues, which is particularly helpful for solving tasks through collaborative discussion.

It is also worth noting that some works combine the two patterns [160, 214, 215, 248, 325]. For example, Chat-Dev [214] divides the end-to-end development tasks into different stages. Agents in different stages transfer information through the intermediate outputs, while agents in the same stage collaborate through chatting.

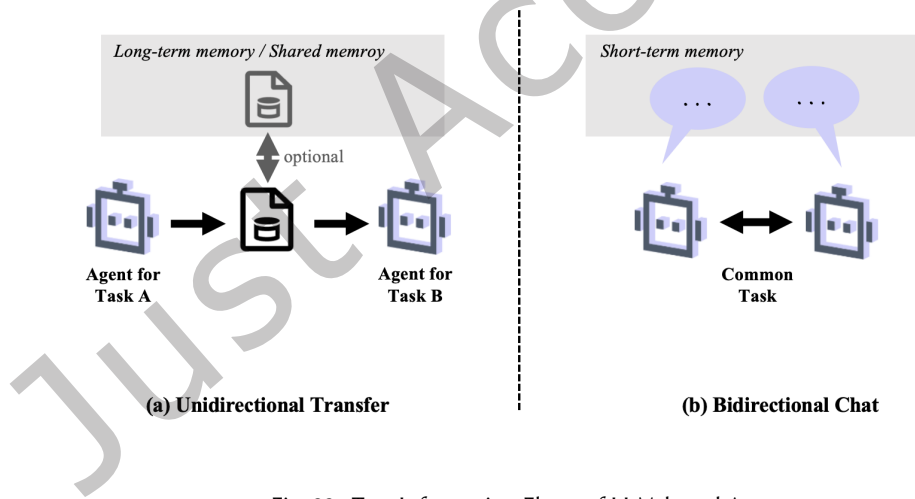


Fig. 23. Two Information Flows of LLM-based Agents

5.2.4 Real-world Applications. In software engineering practice, there are several real-world multi-agent applications available, which are summarized in Table 15.

Most of these products or frameworks primarily function as general-purpose multi-agent systems [15, 26, 35, 102], with software engineering tasks as typical application scenarios. In particular, as an exclusive feature of Devin

Table 15. Real-world Multi-agent Applications

Agent	Organization	Structure	Task	Open Source
MultiDevin [21]	Cognition	Star-like	Mutiple SE tasks	×
Amazon Bedrock [26]	Amazon	Star-like	Universal	×
CrewAI [15]	CrewAI	Layered	Universal	✓
Swarm [35]	OpenAI	Layered	Universal	✓
AgentScope [102]	Alibaba	Layered	Universal	✓

for the enterprise, MultiDevin [21] is fundamentally engineered for software development. It enables parallel task execution using a “manager” Devin and up to ten “worker” Devins. The manager assigns tasks, integrates successful results, and merges them into a single pull request. It is ideal for independent and incremental tasks with measurable success criteria. Besides, Amazon Bedrock [26] is a fully managed generative AI platform for building universal multi-agent systems. It leverages a supervisor agent and specialized sub-agents that collaborate through two core mechanisms: complex task decomposition and intelligent routing based on content-awareness.

Apart from these commercial products, there are several noteworthy open-source multi-agent systems. CrewAI [15] allows developers to assign clear roles, goals, and backstories to individual agents using YAML files and supports both sequential and parallel task execution. Swarm [35] is designed with a lightweight and modular approach, focusing on exploring the basic mechanisms of multi-agent orchestration, such as task handoffs and sequential execution. AgentScope [102] utilizes a message-based communication mechanism and an actor-based distributed architecture, enabling seamless migration from local to distributed environments.

5.3 Human-Agent Collaboration

While most agents aim to achieve maximum automation, where users only need to propose a request and wait for the agents to complete the task, previous studies [59, 312] show that LLM-based agents often encounter bottlenecks during the software development process. Therefore, some agents incorporate the human-agent cooperation paradigm to further align and enhance the agent’s performance with human preferences and expertise. As summarized in Figure 24, existing agents primarily include human participation in four phases: planning, requirements, development, and evaluation.

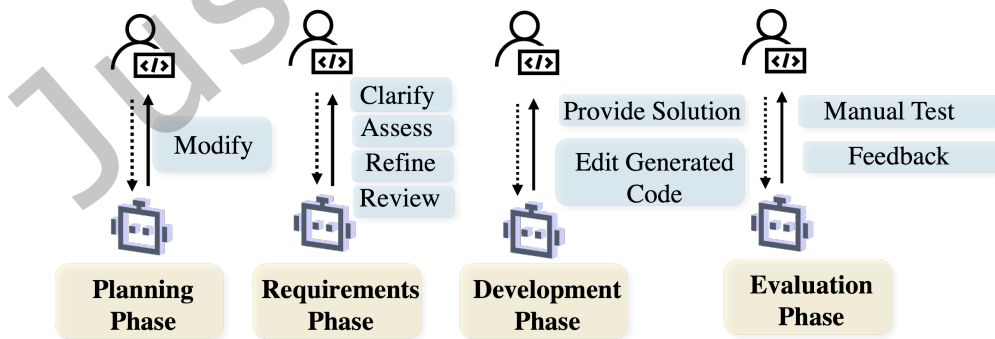


Fig. 24. Human-Agent Collaboration in SE

Planning Phase. Some agents include human intervention in the planning stage of the agent workflow [59, 95, 132, 312]. For example, the low-code LLM platform [59] offers a selection of predefined actions to modify auto-generated workflows, which allows the users to check and revise the workflow before execution. Similarly, Flows [132] explores the impact of having humans provide a brief natural language oracle plan during the planning phase. Experimental results show that this human-AI collaboration outperforms other AI feedback-based approaches. However, revising the system design requires a certain level of expertise, so it is optional in some agents, such as AISD [312] and LLM4PLC [95].

Requirements Phase. The initial requirements (*i.e.*, the task description) provided by users can be ambiguous, which can lead to a gap between the final outputs of the agent system and the user intent. Therefore, it is common for agent systems to further include manual refinement for the requirements. For example, ClarifyGPT [197] detects ambiguous requirements through a code consistency check, generates targeted clarifying questions via an LLM, and refines the requirement based on human responses to produce the final code solution. Similarly, AISD [312] enables users to assess and refine the generated use cases.

Development Phase. Human involvement can be included in the software development phase to correct errors introduced by the agent and guide the next steps of action. In CodeS Extended [298], the entire code repository is generated through a three-layer sketch, allowing users to edit the generated content at each layer to ensure human involvement in refinement and optimization. Similarly, in ART [206], users can enhance agent performance by offering feedback through correcting sub-step outputs(code).

Evaluation Phase. Human participation also serves as a post-evaluation mechanism for the outcomes produced by the agent system, which can further ensure the outputs are aligned with user intent. For example, AISD [312] and Prompt Sapper [75] both include human intervention at the acceptance testing phase. Users can conduct manual testing of the final system, and the test reports help with the necessary refinements.

Discussion. Overall, the exploration of human-agent collaboration in LLM-based agents within the SE domain remains limited. Existing approaches predominantly adopt an agent-centric paradigm, where human involvement is predefined as a fixed stage within the pipeline and is triggered at specific points in each iteration. For instance, in the AISD framework [312], users can revise use cases, adjust plans, and validate the generated software within each development cycle, ensuring that the agent remains aligned with the original intent. Research on human-driven workflows remains an open challenge. Furthermore, current research has not explored human-agent collaboration frameworks designed for multi-person scenarios, which are common in real-world software development teams.

6 Research Opportunities

This section discusses promising research directions and open problems in LLM-based agents for SE.

Evaluation of Agents for SE. Given the emergence of LLM-based agents for SE, it is crucial to develop comprehensive and rigorous evaluation frameworks, including (i) designing more diverse metrics and (ii) constructing higher-quality, more realistic benchmarks.

Metrics. Current evaluations of SE agents primarily focus on their ability to solve specific tasks, such as measuring the success rate of agents on benchmarks such as SWE-bench, without delving into the intermediate states during the agent’s workflow. This lack of fine-grained metrics makes it difficult to assess why agents fail in certain tasks or to what extent they fall short. Given the complexity of SE tasks, failures are common, and without deeper analysis, improving agent performance becomes challenging.

Therefore, the design of fine-grained metrics is necessary, allowing researchers to move beyond “black-box” evaluations and gain insights into the agent’s decision-making process and failure points. These fine-grained metrics have been adopted in many general-purpose agents [149, 166, 184] and embodied agents [234, 235]. For example, researchers design metrics to investigate erroneous states in agents and emphasize assessing progress rates across intermediate sub-stages rather than relying solely on final success rates [184, 234, 235]. Inspired

by these works, more fine-grained metrics can also be introduced in the evaluation of SE agents, which could include error-related metrics (*e.g.*, the ratios of erroneous actions, the average debugging iterations, and the backtracking rate) and progress-related metrics (*e.g.*, the task completion rate of each phase and the overall progress rate). In addition, as we discussed in Section 5.1.1 and Section 5.1.2, it is also vital to incorporate the fine-grained evaluation of individual agent modules, such as the planning module and the memory module. Various methods and metrics have already been employed to evaluate the effectiveness of different modules in general agents [120, 317], *e.g.*, evaluating the effectiveness of the memory mechanism based on the accuracy of answering historical questions, which can serve as references for designing metrics for SE agents.

Additionally, existing metrics heavily emphasize effectiveness, leaving trustworthy requirements such as robustness, security, and fairness underexplored. Given the flexibility and autonomy of LLM-based agents, they may exhibit unstable behavior, which can limit their practical application in real-world SE environments. Evaluating these attributes is essential for building trust in these systems.

Another critical consideration is the cost associated with these agents, particularly as they often involve lengthy workflows, frequent LLM invocations, and the management of large datasets. According to our analysis, only 46.7% of the papers we surveyed have explicitly considered the efficiency of agents in SE tasks, incorporating quantitative analyses of time, token consumption, monetary cost, and feedback loops (*e.g.*, tool invocation frequency or inter-agent discussion frequency). These efficiency and computational costs are particularly important when applying agents to large-scale code repositories, complex documentation, or intricate workflows.

Benchmarks. LLM-based agents significantly extend the capabilities of standalone LLMs, showing great promise in tackling more complex, end-to-end SE tasks. However, existing benchmarks used for evaluating these agents often suffer from quality issues. For instance, prior research [32, 276] has identified that the SWE-bench benchmark includes tasks with vague or incomplete issue descriptions, reducing their relevance and applicability.

Moreover, current benchmarks are far less complex than real-world SE challenges. As outlined in Table 12, the software generated by LLM-based agents for end-to-end development is typically small in scale (*e.g.*, a single function or a few files), which fails to capture the complexity of real-world projects. Furthermore, prior work [32] reports that most tasks in the SWE-bench benchmark (77.8%) can be completed by an experienced engineer within an hour, further underscoring the discrepancy between benchmark tasks and real-world SE demands [219].

To address these shortcomings, future research can focus on creating more realistic, high-quality benchmarks that better reflect the complexity and demands of real-world SE. These improved benchmarks will enable more accurate and meaningful evaluations of LLM-based agents' capabilities and potential.

Human-Agent Collaboration. Software development is inherently a creative process, transforming human requirements into executable software. As such, aligning agent systems with human preferences and intentions is a critical goal. While some existing agents incorporate human participation at various stages of the workflow (as discussed in Section 5.3), there has been limited exploration of how to more thoroughly integrate human involvement throughout the entire software development life cycle. Additionally, the interaction mechanisms between agents and humans remain underexplored.

Currently, agents mainly involve humans in tasks such as requirements clarification, planning adjustments, coding assistance, or evaluation. However, extending human participation to other phases, such as architecture design, test generation, code review, and the end-to-end software maintenance process, remains largely unexplored. A deeper integration of human input across these phases could significantly enhance both the quality and adaptability of the agent's output.

Moreover, designing effective interaction mechanisms is essential for human-agent collaboration. This includes creating user-friendly interfaces for (i) displaying relevant information, such as intermediate outputs from agents, and (ii) collecting user feedback in a streamlined way. Given the complexity of information produced during the agent's workflow, designing such interfaces presents challenges. For instance, when agents are tasked with generating or maintaining a software repository, simply presenting all code files in a flat format would be

resource-intensive and inefficient. Therefore, more sophisticated methods of organizing and representing complex data are required to facilitate effective human-agent interaction.

Perception Modality. Most agents applied to SE tasks primarily rely on textual or visual perception. However, the research on multi-modal approaches is currently insufficient. Text remains the predominant input modality, and the exploration of image inputs is still in its nascent stage, with only one work having utilized a multi-modal LLM (*i.e.*, VisionDroid [173]). This is largely because software development and maintenance activities are heavily associated with processing large volumes of textual data, such as code and documentation. However, there is still significant potential to explore and incorporate more diverse perception modalities into these agents.

For example, in the context of programming assistance, most LLM-powered coding agents predominantly use textual input, such as chat interfaces or Integrated Development Environment (IDE) code contexts. Alternative input formats, such as voice commands or user gestures, remain underutilized. Expanding the range of perception modalities could significantly enhance the flexibility and accessibility of coding assistants, allowing users to interact with agents in ways that better suit their individual workflows and preferences.

Furthermore, exploring diverse perception modalities may shape the future of software development and maintenance, offering new opportunities to streamline interactions and improve the efficiency of agent-driven processes.

Applying Agents for More SE Tasks. While existing agents have been deployed across various SE tasks, several critical phases remain underexplored. As highlighted by our analysis in Section 4, there is a lack of LLM-based agents specifically designed for tasks such as design, verification, and feature maintenance during software development and maintenance.

Developing agent systems tailored to these phases presents unique challenges. Tasks like design and verification require advanced reasoning and comprehension capabilities from the LLM-based agents, extending beyond basic code generation. These tasks demand a deeper understanding of architecture, system logic, and the ability to make informed decisions—skills that traditional LLM-controlled agents may not yet fully possess.

Training Software-oriented LLMs for SE Agents. LLMs are the central component controlling the “brain” of agent systems. Most existing agents for SE rely on LLMs trained on general-purpose data (*e.g.*, ChatGPT [12]) or code-specific data (*e.g.*, Deepseek-Coder [106] and StarCoder [178]). While massive code from GitHub has been leveraged to train LLMs for code, addressing complex SE tasks requires more specialized data. The reason is that software is not just about code. For example, valuable data from the whole software development life cycle, such as design, architecture, developer discussions/communications, historical code changes, and even dynamic runtime information, remain largely untapped. Incorporating such data into training could lead to the development of more powerful LLMs for software (not just for code), better suited for the unique demands of SE. These enhanced models could form the foundation for more advanced and capable agent systems designed to tackle a wider range of SE tasks.

SE Expertise in Building Agents. Incorporating well-established SE expertise into the design of agent systems is crucial. For instance, widely adopted SE techniques can be integrated as tools or sub-components of agent systems. As discussed in Section 5.1.4, some existing agents already leverage SE toolkits and techniques, but many other SE tools and techniques—such as advanced debugging and testing methods—remain underutilized. Further efforts are needed to comprehensively integrate these tools and techniques into agent systems to enhance their functionality.

In addition, SE domain knowledge can guide the workflow of agent systems. As noted in Section 4.7, some agents for end-to-end software development follow traditional software process models, such as the waterfall or agile models. However, many other software process models remain unexplored. Rather than granting agents full autonomy, existing software development and maintenance methodologies can be used to partially control their workflows. For example, as revealed by the recent Agentless study [276] and also further confirmed by OpenAI [32], LLMs using a simplistic workflow based on traditional fault localization and program repair

pipelines can even outperform other, more complex, fully autonomous agents. This suggests that leveraging domain expertise from SE can potentially help improve the effectiveness, robustness, efficiency, interpretability, and replicability of agentic solutions.

Another crucial direction is to leverage the quality assurance techniques in SE to build trustworthy LLM-based agents. Recently, concerns about the trustworthiness of AI have been raised [169], including but not limited to issues of privacy [103], security [70], fairness [73], and robustness [256]. Quality assurance techniques in SE (*e.g.*, testing and debugging) have been effectively used to evaluate and assist in addressing trustworthiness issues [40, 72, 74], holding significant potential for building trustworthy LLM-based agents. However, LLMs offer diverse functionalities across various domains, which necessitate task-specific designs for trustworthiness assurance. Moreover, unlike standalone LLMs, trustworthiness issues in LLM-based agents can stem from the backbone LLMs, attached modules (*e.g.*, memory, action), and the underlying software systems or frameworks. These characteristics pose unique challenges for designing trustworthiness assurance techniques for LLM-based agents, requiring consideration of both individual modules and the overall system across diverse tasks.

Priorities of different Research Directions. We then discuss the priorities of the aforementioned research directions. Overall, we believe that establishing standardized and high-quality benchmarks and metrics for specific tasks should be given higher priority. On the one hand, the absence of standardized benchmarks in current works limits meaningful comparisons (especially for the end-to-end software development task); on the other hand, benchmarks play a crucial role in driving advancements in related fields [238], as evidenced by SWE-bench, which has facilitated the development of end-to-end maintenance agents. In addition, considering the technical development in general-purpose agents [76, 261, 275], it is feasible to explore richer human-agent collaboration patterns, more diverse perception modalities, a broader range of applicable tasks, and the integration of SE expertise into agent design. In contrast, training software-oriented LLMs for SE agents is likely a long-term endeavor, as it requires extensive data accumulation across the software engineering life cycle, which requires significant human effort and shifts in development paradigms (*e.g.*, documenting in more diverse formats). Additionally, it necessitates well-designed data formats and effective training methodologies. However, despite these challenges, this direction holds significant promise for realizing a general-purpose SE agent.

7 Discussion

7.1 Disparity of LLM-based Agents and Standalone LLMs in SE Tasks

LLMs serve as the central reasoning engine in LLM-based agents, enabling core tasks such as inference, analysis, and planning. Prior work has demonstrated that standalone LLMs can achieve promising results when directly applied to various SE tasks [96, 116]. However, compared to LLM-based agents, standalone LLMs lack the ability to perceive environmental changes and to dynamically adjust their plans and actions, thus suffering from higher susceptibility to hallucination and non-deterministic outputs. These limitations undermine their performance in SE tasks, particularly in two dimensions: effectiveness and practicality.

From the **effectiveness** perspective, although standalone LLMs achieve satisfactory results on many single-phase SE tasks (*e.g.*, code generation, software testing, and program repair), their limited capabilities in handling generation hallucination and stochasticity often lead to inferior performance than LLM-based agents. For example, studies [124, 305] have shown that LLM-based agents consistently achieve higher pass@1 than their foundation LLMs across code generation benchmarks of various complexity, including method-level (*e.g.*, HumanEval [65]), competition-level (*e.g.*, LiveCodeBench [124]), and repository-level (*e.g.*, CodeAgentBench [305]) tasks. Similar findings have also been observed in other SE tasks, such as software testing [80, 296], debugging [136, 137, 216], and static code checking [117, 248].

In terms of **practicality**, the plug-and-play nature of standalone LLMs makes them easier to adopt, but this simplicity comes at the cost of task coverage. Without the ability to interact with dynamic environments, standalone LLMs fall short in handling complex, environment-dependent tasks [96, 116], such as IT operations [313, 323], end-to-end software development [115, 298], and end-to-end software maintenance [276, 288]. In contrast, LLM-based agents have filled this gap and been successfully applied to broader SE scenarios.

In summary, although standalone LLMs are effective for many SE tasks, their limitations in adaptability, controllability, and iterative reasoning often lead to suboptimal outcomes and narrower applications. LLM-based agents, by integrating planning, feedback, and environmental interaction, offer stronger performance and wider applicability in real-world software engineering.

7.2 Threats to Validity

One potential threat to the validity of our survey arises from the manual paper inspection process. Despite having two authors independently review each paper and involving a third author to resolve disagreements, the subjective judgment inherent in manual screening may still lead to relevant papers being inadvertently excluded. Such omissions could affect the comprehensiveness of our survey and potentially bias the final findings.

Another threats to the validity of our conclusions stem from the publication status of the collected papers. In particular, some strategies or agent configurations are supported primarily by preprints or unpublished manuscripts, which have not undergone peer review, as follows:

Multi-Agent Strategy in Requirements Engineering Lacks Sufficient Validation. As we mentioned in Section 4.1, multi-agent collaboration has emerged as a commonly adopted strategy in the requirements engineering domain. Among the four works we surveyed in this section, three employed multi-agent architectures to handle complex interactions and task decomposition [45, 47, 129]. However, two works have not been published in peer-reviewed venues yet. Therefore, further empirical and peer-reviewed studies are needed to substantiate the effectiveness of multi-agent strategies in requirements engineering.

Uncertain Efficacy of Knowledge-Enhanced Bug Detection Methods. Similarly, for the bug detection task described in Section 4.3.1, the use of additional knowledge from tool execution is an intriguing strategy, but it is supported by one published work [168] out of four [97, 168, 206, 242]. The approach might still be in an exploratory phase, and more future effort should be dedicated to assessing its robustness and generalizability.

Weak Evidence for Iterative Coverage Improvements in Unit Testing. In the unit testing task, iterative refinement to fix compilation or execution errors and enhance fault detection is not only more widely adopted but also more frequently published, suggesting stronger community endorsement. However, iterative refinement to increase coverage is supported by fewer studies and has lower publication rates (one [42] published out of three [42, 252, 285]). It indicates that this strategy, while promising, still requires more empirical evidence to support its effectiveness.

Limited Validation of Visual Input in LLM-based Agents for SE. As discussed in Section 5.1.3, the integration of visual input into LLM-based agents for software engineering tasks has shown potential for enhancing contextual understanding and multi-modal reasoning. However, among the three works surveyed [173, 245, 268], one [245] has been published. The benefits and applicability of visual input in this domain remain insufficiently validated and require further systematic investigation.

Insufficient Evidence on the Effectiveness of Specific Memory Formats. As outlined in Section 5.1.2, memory format plays a critical role in the coordination and long-term reasoning abilities of LLM-based agents, but the effectiveness of some memory formats has not been sufficiently validated. For example, structured messages have been adopted in MetaGPT [115], E&V [109], and MARE [129], but only MetaGPT [115] has undergone peer-reviewed publication. Similarly, storing images in the memory has only been explored in VisionDroid [173],

which has not been published. These gaps highlight the need for more empirical and peer-reviewed studies to substantiate the effectiveness of different memory formats in LLM-based SE agents.

Overall, the uneven publication status across different tasks and strategies introduces uncertainty into our assessment of the effectiveness and generality of various LLM-agent techniques. Future research with more rigorous evaluation and wider peer-reviewed dissemination will be essential to strengthen the reliability of conclusions in this emerging field.

8 Conclusion

In this paper, we have presented a comprehensive and systematic survey of 124 papers on LLM-based agents for SE. We analyzed the current research from both the SE and agent perspectives. From the SE perspective, we analyzed how LLM-based agents are applied across different software development and maintenance activities. From the agent perspective, we focus on the design of components in LLM-based agents for SE. In addition, we discussed open challenges and future directions in this critical domain.

Acknowledgement

After drafting the initial version, we contacted the authors of the collected papers to verify the accuracy and comprehensiveness of the survey. We extend our sincere gratitude to those authors who generously provided valuable comments and feedback on the earlier draft of this paper.

References

- [1] [n. d.]. *Black*. <https://github.com/psf/black>.
- [2] [n. d.]. *DuckDuckGo*. <https://duckduckgo.com/>.
- [3] [n. d.]. *EclipseJDTLS*. <https://github.com/eclipse-jdtls/eclipse-jdt.ls>.
- [4] [n. d.]. *Frama-C*. <https://www.frama-c.com/>.
- [5] [n. d.]. *International Dataset of Flaky tests*. <https://github.com/TestingResearchIllinois/idoft>.
- [6] [n. d.]. *Jedi*. <https://github.com/davidhalter/jedi/>.
- [7] [n. d.]. *The LLVM Compiler Infrastructure*. <https://llvm.org>.
- [8] [n. d.]. *Metasploit Framework*. <https://www.metasploit.com/>.
- [9] [n. d.]. *OCLint*. <https://github.com/oclint/>.
- [10] [n. d.]. *SerpApi*. <https://serpapi.com/>.
- [11] 2021. *Python Wrapper of Android UiAutomator Test Tool*. <https://github.com/xiaocong/uiautomator>.
- [12] 2022. *OpenAI: Introducing ChatGPT*. <https://openai.com/blog/chatgpt/>.
- [13] 2023. *Android Debug Bridge (adb) - Android Developers*. <https://developer.android.com/studio/command-line/adb/>.
- [14] 2023. *Android UiAutomator2 Python Wrapper*. <https://github.com/openatx/uiautomator2/>.
- [15] 2023. *CrewAI*. <https://www.crewai.com/>.
- [16] 2023. *Function Calling and Other API Updates*. <https://openai.com/index/function-calling-and-other-api-updates/>.
- [17] 2023. *Gcov*. <https://gcc.gnu.org/onlinedocs/gcc/Gcov.html>.
- [18] 2023. *Genymotion - Android Emulator for App Testing*. <https://www.genymotion.com/>.
- [19] 2023. *GPT-3.5*. <https://platform.openai.com/docs/models/gpt-3-5-turbo>.
- [20] 2023. *GPT-4*. <https://platform.openai.com/docs/models/gpt-4-and-gpt-4-turbo>.
- [21] 2023. *MultiDevin*. <https://devin.ai/>.
- [22] 2023. *pyvbox*. <https://pypi.org/project/pyvbox/>.
- [23] 2023. *virtualbox*. <https://www.virtualbox.org/>.
- [24] 2024. 7 million publications. <https://blog.dblp.org/2024/01/01/7-million-publications/>.
- [25] 2024. *Aider*. <https://aider.chat/>.
- [26] 2024. *Amazon Bedrock*. <https://aws.amazon.com/bedrock/>.
- [27] 2024. *arXiv*. <https://arxiv.org/>.
- [28] 2024. *Clang*. <https://clang.llvm.org/>.
- [29] 2024. *codeparrot/github-jupyter*. <https://huggingface.co/datasets/codeparrot/github-jupyter>.
- [30] 2024. *DBLP*. <https://dblp.org>.

- [31] 2024. *GPT-4*. <https://docs.oracle.com/javase/8/docs/api/java/lang/instrument/package-summary.html>.
- [32] 2024. *Introducing SWE-bench Verified*. <https://openai.com/index/introducing-swe-bench-verified/>.
- [33] 2024. *Moatless tools*. <https://github.com/aorwall/moatless-tools>.
- [34] 2024. *PlantUML*. <https://plantuml.com/>.
- [35] 2024. *Swarm*. <https://github.com/openai/swarm>.
- [36] 2024. *SWE-bench Lite*. <https://www.swebench.com/lite.html>.
- [37] 2024. *syzbot*. <https://syzkaller.appspot.com/upstream>.
- [38] 2024. *Syzkaller*. <https://github.com/google/syzkaller/>.
- [39] 2024. *tree-sitter*. <https://github.com/tree-sitter/tree-sitter/>.
- [40] Mohit Kumar Ahuja, Mohamed-Bachir Belaid, Pierre Bernabé, Mathieu Collet, Arnaud Gotlieb, Chhagan Lal, Dusica Marijan, Sagar Sen, Aizaz Sharif, and Helge Spieker. 2020. Opening the Software Engineering Toolbox for the Assessment of Trustworthy AI. In *Proceedings of the First International Workshop on New Foundations for Human-Centered AI (NeHuAI) co-located with 24th European Conference on Artificial Intelligence (ECAI 2020), Santiago de Compostella, Spain, September 4, 2020 (CEUR Workshop Proceedings, Vol. 2659)*, Alessandro Saffiotti, Luciano Serafini, and Paul Lukowicz (Eds.). CEUR-WS.org, 67–70. <https://ceur-ws.org/Vol-2659/ahuja.pdf>
- [41] Miltiadis Allamanis, Earl T. Barr, Premkumar T. Devanbu, and Charles Sutton. 2018. A Survey of Machine Learning for Big Code and Naturalness. *ACM Comput. Surv.* 51, 4 (2018), 81:1–81:37. <https://doi.org/10.1145/3212695>
- [42] Juan Altmayer Pizzorno and Emery D. Berger. 2025. CoverUp: Effective High Coverage Test Generation for Python. *Proc. ACM Softw. Eng.* 2, FSE, Article FSE128 (June 2025), 23 pages. <https://doi.org/10.1145/3729398>
- [43] Anthropic. 2024. *Claude 3.5 Sonnet Model Card Addendum*. https://www-cdn.anthropic.com/fed9cc193a14b84131812372d8d5857f8f304c52/Model_Card_Claude_3_Addendum.pdf.
- [44] Mikio Aoyama. 1997. Agile Software Process Model. In *21st International Computer Software and Applications Conference (COMPSAC '97), 11-15 August 1997, Washington, DC, USA*. IEEE Computer Society, 454–459. <https://doi.org/10.1109/CMPSAC.1997.625042>
- [45] Chetan Arora, John Grundy, and Mohamed Abdelrazek. 2024. *Advancing Requirements Engineering Through Generative AI: Assessing the Role of LLMs*. Springer Nature Switzerland, Cham, 129–148. https://doi.org/10.1007/978-3-031-55642-5_6
- [46] Ellen Arteca, Sebastian Harner, Michael Pradel, and Frank Tip. 2022. Nessie: Automatically Testing JavaScript APIs with Asynchronous Callbacks. In *44th IEEE/ACM 44th International Conference on Software Engineering, ICSE 2022, Pittsburgh, PA, USA, May 25-27, 2022*. ACM, 1494–1505. <https://doi.org/10.1145/3510003.3510106>
- [47] Mohammadmehdi Ataei, Hyunmin Cheong, Daniele Grandi, Ye Wang, Nigel Morris, and Alexander Tessier. 2025. Elicitron: A Large Language Model Agent-Based Simulation Framework for Design Requirements Elicitation. *Journal of Computing and Information Science in Engineering* 25, 2 (01 2025), 021012. <https://doi.org/10.1115/1.4067388> arXiv:https://asmedigitalcollection.asme.org/computingengineering/article-pdf/25/2/021012/7423688/jcise_25_2_021012.pdf
- [48] Jacob Austin, Augustus Odena, Maxwell I. Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie J. Cai, Michael Terry, Quoc V. Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. *CoRR* abs/2108.07732 (2021). arXiv:2108.07732 <https://arxiv.org/abs/2108.07732>
- [49] Pavel Avgustinov, Oege de Moor, Michael Peyton Jones, and Max Schäfer. 2016. QL: Object-oriented Queries on Relational Data. In *30th European Conference on Object-Oriented Programming, ECOOP 2016, July 18-22, 2016, Rome, Italy (LIPIcs, Vol. 56)*, Shriram Krishnamurthi and Benjamin S. Lerner (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2:1–2:25. <https://doi.org/10.4230/LIPICS.ECOOP.2016.2>
- [50] Ramakrishna Bairi, Atharv Sonwane, Aditya Kanade, Vageesh D. C., Arun Iyer, Suresh Parthasarathy, Sriram K. Rajamani, Balasubramanyan Ashok, and Shashank Shet. 2024. CodePlan: Repository-Level Coding using LLMs and Planning. *Proc. ACM Softw. Eng.* 1, FSE (2024), 675–698. <https://doi.org/10.1145/3643757>
- [51] Saikat Barua. 2024. Exploring Autonomous Agents through the Lens of Large Language Models: A Review. *CoRR* abs/2404.04442 (2024). <https://doi.org/10.48550/ARXIV.2404.04442> arXiv:2404.04442
- [52] Samuel Benton, Xia Li, Yiling Lou, and Lingming Zhang. 2020. On the Effectiveness of Unified Debugging: An Extensive Study on 16 Program Repair Systems. In *35th IEEE/ACM International Conference on Automated Software Engineering, ASE 2020, Melbourne, Australia, September 21-25, 2020*. IEEE, 907–918. <https://doi.org/10.1145/3324884.3416566>
- [53] Samuel Benton, Xia Li, Yiling Lou, and Lingming Zhang. 2022. Evaluating and Improving Unified Debugging. *IEEE Trans. Software Eng.* 48, 11 (2022), 4692–4716. <https://doi.org/10.1109/TSE.2021.3125203>
- [54] Barry W. Boehm. 2006. A View of 20th and 21st Century Software Engineering. In *28th International Conference on Software Engineering (ICSE 2006), Shanghai, China, May 20-28, 2006*, Leon J. Osterweil, H. Dieter Rombach, and Mary Lou Soffa (Eds.). ACM, 12–29. <https://doi.org/10.1145/1134285.1134288>
- [55] Tim Boland and Paul E. Black. 2012. Juliet 1.1 C/C++ and Java Test Suite. *Computer* 45, 10 (2012), 88–90. <https://doi.org/10.1109/MC.2012.345>
- [56] Grady Booch, Ivar Jacobson, James Rumbaugh, et al. 1996. The Unified Modeling Language. *Unix Review* 14, 13 (1996), 5.

- [57] Islem Bouzenia, Premkumar T. Devanbu, and Michael Pradel. 2025. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. In *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 - May 6, 2025*. IEEE, 2188–2200. <https://doi.org/10.1109/ICSE55347.2025.00157>
- [58] Cameron Browne, Edward Jack Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavenor, Diego Perez Liebana, Spyridon Samothrakis, and Simon Colton. 2012. A Survey of Monte Carlo Tree Search Methods. *IEEE Trans. Comput. Intell. AI Games* 4, 1 (2012), 1–43. <https://doi.org/10.1109/TCIAIG.2012.2186810>
- [59] Yuzhe Cai, Shaoguang Mao, Wenshan Wu, Zehua Wang, Yaobo Liang, Tao Ge, Chenfei Wu, Wang You, Ting Song, Yan Xia, Nan Duan, and Furu Wei. 2024. Low-code LLM: Graphical User Interface over Large Language Models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies: System Demonstrations, NAACL 2024, Mexico City, Mexico, June 16-21, 2024*, Kai-Wei Chang, Annie Lee, and Nazneen Rajani (Eds.). Association for Computational Linguistics, 12–25. <https://doi.org/10.18653/V1/2024.NAACL-DEMO.2>
- [60] José Campos, André Ribeiro, Alexandre Perez, and Rui Abreu. 2012. GZoltar: An Eclipse Plug-in for Testing and Debugging. In *IEEE/ACM International Conference on Automated Software Engineering, ASE’12, Essen, Germany, September 3-7, 2012*, Michael Goedicke, Tim Menzies, and Motoshi Saeki (Eds.). ACM, 378–381. <https://doi.org/10.1145/2351676.2351752>
- [61] Roberto Cavada, Alessandro Cimatti, Michele Dorigatti, Alberto Griggio, Alessandro Mariotti, Andrea Micheli, Sergio Mover, Marco Roveri, and Stefano Tonetta. 2014. The nuXmv Symbolic Model Checker. In *Computer Aided Verification - 26th International Conference, CAV 2014, Held as Part of the Vienna Summer of Logic, VSL 2014, Vienna, Austria, July 18-22, 2014. Proceedings (Lecture Notes in Computer Science, Vol. 8559)*, Armin Biere and Roderick Bloem (Eds.). Springer, 334–342. https://doi.org/10.1007/978-3-319-08867-9_22
- [62] Dong Chen, Shaoxin Lin, Muhan Zeng, Daoguang Zan, Jian-Gang Wang, Anton Cheshkov, Jun Sun, Hao Yu, Guoliang Dong, Artem Aliev, Jie Wang, Xiao Cheng, Guangtai Liang, Yuchi Ma, Pan Bian, Tao Xie, and Qianxiang Wang. 2024. CodeR: Issue Resolving with Multi-Agent and Task Graphs. *CoRR* abs/2406.01304 (2024). <https://doi.org/10.48550/ARXIV.2406.01304> arXiv:2406.01304
- [63] Guangyao Chen, Siwei Dong, Yu Shu, Ge Zhang, Jaward Sesay, Börje Karlsson, Jie Fu, and Yemin Shi. 2024. AutoAgents: A Framework for Automatic Agent Generation. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI 2024, Jeju, South Korea, August 3-9, 2024*. ijcai.org, 22–30. <https://www.ijcai.org/proceedings/2024/3>
- [64] Junjie Chen, Jibesh Patra, Michael Pradel, Yingfei Xiong, Hongyu Zhang, Dan Hao, and Lu Zhang. 2021. A Survey of Compiler Testing. *ACM Comput. Surv.* 53, 1 (2021), 4:1–4:36. <https://doi.org/10.1145/3363562>
- [65] Mark Chen, Jerry Twarek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgren Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. *CoRR* abs/2107.03374 (2021). arXiv:2107.03374 <https://arxiv.org/abs/2107.03374>
- [66] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. 2024. AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. <https://openreview.net/forum?id=EHg5GDnyq1>
- [67] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2024. Teaching Large Language Models to Self-Debug. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. <https://openreview.net/forum?id=KuPixIqPiq>
- [68] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2024. ChatUniTest: A Framework for LLM-Based Test Generation. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering, FSE 2024, Porto de Galinhas, Brazil, July 15-19, 2024*, Marcelo d’Amorim (Ed.). ACM, 572–576. <https://doi.org/10.1145/3663529.3663801>
- [69] Yang Chen and Reyhaneh Jabbarvand. 2024. Neurosymbolic Repair of Test Flakiness. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024, Vienna, Austria, September 16-20, 2024*, Maria Christakis and Michael Pradel (Eds.). ACM, 1402–1414. <https://doi.org/10.1145/3650212.3680369>
- [70] Yuchen Chen, Weisong Sun, Chunrong Fang, Zhenpeng Chen, Yifei Ge, Tingxu Han, Qunjun Zhang, Yang Liu, Zhenyu Chen, and Baowen Xu. 2025. Security of Language Models for Code: A Systematic Literature Review. *ACM Trans. Softw. Eng. Methodol.* (2025). <https://doi.org/10.1145/3735554>
- [71] Zimin Chen, Steve Kommrusch, Michele Tufano, Louis-Noël Pouchet, Denys Poshyvanyk, and Martin Monperrus. 2021. SequenceR: Sequence-to-Sequence Learning for End-to-End Program Repair. *IEEE Trans. Software Eng.* 47, 9 (2021), 1943–1959. <https://doi.org/10.1109/TSE.2019.2940179>
- [72] Zhenpeng Chen, Xinyue Li, Jie M. Zhang, Weisong Sun, Ying Xiao, Tianlin Li, Yiling Lou, and Yang Liu. 2025. Software Fairness Dilemma: Is Bias Mitigation a Zero-Sum Game? *Proc. ACM Softw. Eng.* 2, FSE (2025), 1780–1801.

- [73] Zhenpeng Chen, Sheng Shen, Ziniu Hu, Xuan Lu, Qiaozhu Mei, and Xuanzhe Liu. 2019. Emoji-Powered Representation Learning for Cross-Lingual Sentiment Classification. In *The World Wide Web Conference, WWW 2019*. 251–262.
- [74] Zhenpeng Chen, Jie M. Zhang, Max Hort, Mark Harman, and Federica Sarro. 2024. Fairness Testing: A Comprehensive Survey and Analysis of Trends. *ACM Trans. Softw. Eng. Methodol.* 33, 5 (2024), 137:1–137:59. <https://doi.org/10.1145/3652155>
- [75] Yu Cheng, Jieshan Chen, Qing Huang, Zhenchang Xing, Xiwei Xu, and Qinghua Lu. 2024. Prompt Sapper: A LLM-Empowered Production Tool for Building AI Chains. *ACM Trans. Softw. Eng. Methodol.* 33, 5 (2024), 124:1–124:24. <https://doi.org/10.1145/3638247>
- [76] Yuheng Cheng, Ceyao Zhang, Zhengwen Zhang, Xiangrui Meng, Sirui Hong, Wenhao Li, Zihao Wang, Zekai Wang, Feng Yin, Junhua Zhao, and Xiuqiang He. 2024. Exploring Large Language Model based Intelligent Agents: Definitions, Methods, and Prospects. *CoRR abs/2401.03428* (2024). <https://doi.org/10.48550/ARXIV.2401.03428> arXiv:2401.03428
- [77] David R. Cok. 2011. OpenJML: JML for Java 7 by Extending OpenJDK. In *NASA Formal Methods - Third International Symposium, NFM 2011, Pasadena, CA, USA, April 18-20, 2011. Proceedings (Lecture Notes in Computer Science, Vol. 6617)*, Mihaela Gheorghiu Bobaru, Klaus Havelund, Gerard J. Holzmann, and Rajeev Joshi (Eds.). Springer, 472–479. https://doi.org/10.1007/978-3-642-20398-5_35
- [78] Iain D. Craig. 1988. Blackboard Systems. *Artif. Intell. Rev.* 2, 2 (1988), 103–118. <https://doi.org/10.1007/BF00140399>
- [79] Chris Cummins, Volker Seeker, Dejan Grubisic, Baptiste Roziere, Jonas Gehring, Gabriel Synnaeve, and Hugh Leather. 2025. LLM Compiler: Foundation Language Models for Compiler Optimization. In *Proceedings of the 34th ACM SIGPLAN International Conference on Compiler Construction (Las Vegas, NV, USA) (CC '25)*. Association for Computing Machinery, New York, NY, USA, 141–153. <https://doi.org/10.1145/3708493.3712691>
- [80] Arghavan Moradi Dakhel, Amin Nikanjam, Vahid Majdinasab, Foutse Khomh, and Michel C. Desmarais. 2024. Effective Test Generation Using Pre-Trained Large Language Models and Mutation Testing. *Inf. Softw. Technol.* 171 (2024), 107468. <https://doi.org/10.1016/j.infsof.2024.107468>
- [81] Leonardo Mendonça de Moura and Nikolaj S. Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems, 14th International Conference, TACAS 2008, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2008, Budapest, Hungary, March 29-April 6, 2008. Proceedings (Lecture Notes in Computer Science, Vol. 4963)*, C. R. Ramakrishnan and Jakob Rehof (Eds.). Springer, 337–340. https://doi.org/10.1007/978-3-540-78800-3_24
- [82] Leonardo Mendonça de Moura and Nikolaj S. Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems, 14th International Conference, TACAS 2008, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2008, Budapest, Hungary, March 29-April 6, 2008. Proceedings (Lecture Notes in Computer Science, Vol. 4963)*, C. R. Ramakrishnan and Jakob Rehof (Eds.). Springer, 337–340. https://doi.org/10.1007/978-3-540-78800-3_24
- [83] Alix Decrop, Gilles Perrouin, Mike Papadakis, Xavier Devroey, and Pierre-Yves Schobbens. 2024. You Can REST Now: Automated Specification Inference and Black-Box Testing of RESTful APIs with Large Language Models. *CoRR abs/2402.05102* (2024). <https://doi.org/10.48550/ARXIV.2402.05102> arXiv:2402.05102
- [84] Gelei Deng, Yi Liu, Victor Mayoral Vilches, Peng Liu, Yuekang Li, Yuan Xu, Martin Pinzger, Stefan Rass, Tianwei Zhang, and Yang Liu. 2024. PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing. In *33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14-16, 2024*, Davide Balzarotti and Wenyuan Xu (Eds.). USENIX Association. <https://www.usenix.org/conference/usenixsecurity24/presentation/deng>
- [85] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. 2023. Large Language Models Are Zero-Shot Fuzzers: Fuzzing Deep-Learning Libraries via Large Language Models. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2023, Seattle, WA, USA, July 17-21, 2023*, René Just and Gordon Fraser (Eds.). ACM, 423–435. <https://doi.org/10.1145/3597926.3598067>
- [86] Ajinkya Deshpande, Anmol Agarwal, Shashank Shet, Arun Iyer, Aditya Kanade, Ramakrishna Bairi, and Suresh Parthasarathy. 2024. Natural Language to Class-level Code Generation by Iterative Tool-augmented Reasoning over Repository. In *ICML 2024 Workshop on Data-Centric Machine Learning Research*. <https://openreview.net/forum?id=yqjr7ojVYa>
- [87] Yihong Dong, Jiazheng Ding, Xue Jiang, Ge Li, Zhuo Li, and Zhi Jin. 2025. CodeScore: Evaluating Code Generation by Learning Code Execution. *ACM Trans. Softw. Eng. Methodol.* 34, 3 (2025), 77:1–77:22. <https://doi.org/10.1145/3695991>
- [88] Yihong Dong, Xue Jiang, Zhi Jin, and Ge Li. 2024. Self-Collaboration Code Generation via ChatGPT. *ACM Trans. Softw. Eng. Methodol.* 33, 7 (2024), 189:1–189:38. <https://doi.org/10.1145/3672459>
- [89] Xueying Du, Geng Zheng, Kaixin Wang, Jiayi Feng, Wentai Deng, Mingwei Liu, Bihuan Chen, Xin Peng, Tao Ma, and Yiling Lou. 2024. Vul-RAG: Enhancing LLM-based Vulnerability Detection via Knowledge-level RAG. *CoRR abs/2406.11147* (2024). <https://doi.org/10.48550/ARXIV.2406.11147> arXiv:2406.11147
- [90] Xueying Du, Geng Zheng, Kaixin Wang, Jiayi Feng, Wentai Deng, Mingwei Liu, Bihuan Chen, Xin Peng, Tao Ma, and Yiling Lou. 2024. Vul-RAG: Enhancing LLM-based Vulnerability Detection via Knowledge-level RAG. *CoRR abs/2406.11147* (2024). <https://doi.org/10.48550/ARXIV.2406.11147> arXiv:2406.11147
- [91] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2024. Improving Factuality and Reasoning in Language Models through Multiagent Debate. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net. <https://openreview.net/forum?id=zj7YuTE4t8>

- [92] Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. 2007. The Daikon System for Dynamic Detection of Likely Invariants. *Sci. Comput. Program.* 69, 1-3 (2007), 35–45. <https://doi.org/10.1016/J.SCICO.2007.01.015>
- [93] Khashayar Etemadi, Bardia Mohammadi, Zhendong Su, and Martin Monperrus. 2025. Mokav: Execution-driven Differential Testing with LLMs. *Journal of Systems and Software* 230 (2025), 112571. <https://doi.org/10.1016/j.jss.2025.112571>
- [94] Sarah Fakhoury, Markus Kuppe, Shuvendu K. Lahiri, Tahina Ramananandro, and Nikhil Swamy. 2025. 3DGen: AI-Assisted Generation of Provably Correct Binary Format Parsers. In *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 - May 6, 2025*. IEEE, 2535–2547. <https://doi.org/10.1109/ICSE55347.2025.00173>
- [95] Mohamad Fakihi, Rahul Dharmaji, Yasamin Moghaddas, Gustavo Quiros, Oluwatosin Ogundare, and Mohammad Abdullah Al Faruque. 2024. LLM4PLC: Harnessing Large Language Models for Verifiable Programming of PLCs in Industrial Control Systems. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice, ICSE-SEIP 2024, Lisbon, Portugal, April 14-20, 2024*. ACM, 192–203. <https://doi.org/10.1145/3639477.3639743>
- [96] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. 2023. Large Language Models for Software Engineering: Survey and Open Problems. In *IEEE/ACM International Conference on Software Engineering: Future of Software Engineering, ICSE-FoSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 31–53. <https://doi.org/10.1109/ICSE-FOSE59343.2023.00008>
- [97] Gang Fan, Xiaoheng Xie, Xunjin Zheng, Yinan Liang, and Peng Di. 2023. Static Code Analysis in the AI Era: An In-depth Exploration of the Concept, Function, and Potential of Intelligent Code Analysis Agents. *CoRR abs/2310.08837* (2023). <https://doi.org/10.48550/ARXIV.2310.08837> arXiv:2310.08837
- [98] Richard Fang, Rohan Bindu, Akul Gupta, and Daniel Kang. 2024. LLM Agents can Autonomously Exploit One-day Vulnerabilities. *CoRR abs/2404.08144* (2024). <https://doi.org/10.48550/ARXIV.2404.08144> arXiv:2404.08144
- [99] Josselin Feist, Gustavo Grieco, and Alex Groce. 2019. Slither: A Static Analysis Framework for Smart Contracts. In *Proceedings of the 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain, WETSEB@ICSE 2019, Montreal, QC, Canada, May 27, 2019*. IEEE / ACM, 8–15. <https://doi.org/10.1109/WETSEB.2019.00008>
- [100] Sidong Feng and Chunyang Chen. 2024. Prompting Is All You Need: Automated Android Bug Replay with Large Language Models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14-20, 2024*. ACM, 67:1–67:13. <https://doi.org/10.1145/3597503.3608137>
- [101] Cormac Flanagan and K. Rustan M. Leino. 2001. Houdini, An Annotation Assistant for ESC/Java. In *FME 2001: Formal Methods for Increasing Software Productivity, International Symposium of Formal Methods Europe, Berlin, Germany, March 12-16, 2001, Proceedings (Lecture Notes in Computer Science, Vol. 2021)*, José Nuno Oliveira and Pamela Zave (Eds.). Springer, 500–517. https://doi.org/10.1007/3-540-45251-6_29
- [102] Dawei Gao, Zitao Li, Weirui Kuang, Xuchen Pan, Daoyuan Chen, Zhijian Ma, Bingchen Qian, Liuyi Yao, Lin Zhu, Chen Cheng, Hongzhu Shi, Yaliang Li, Bolin Ding, and Jingren Zhou. 2024. AgentScope: A Flexible yet Robust Multi-Agent Platform. *CoRR abs/2402.14034* (2024). <https://doi.org/10.48550/ARXIV.2402.14034> arXiv:2402.14034
- [103] Wei Gao, Shangwei Guo, Tianwei Zhang, Han Qiu, Yonggang Wen, and Yang Liu. 2021. Privacy-Preserving Collaborative Learning With Automatic Transformation Search. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2021*. 114–123.
- [104] Luca Gazzola, Daniela Micucci, and Leonardo Mariani. 2019. Automatic Software Repair: A Survey. *IEEE Trans. Software Eng.* 45, 1 (2019), 34–67. <https://doi.org/10.1109/TSE.2017.2755013>
- [105] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. 2021. Transformer Feed-Forward Layers Are Key-Value Memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*, Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.). Association for Computational Linguistics, 5484–5495. <https://doi.org/10.18653/V1/2021.EMNLP-MAIN.446>
- [106] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming - The Rise of Code Intelligence. *CoRR abs/2401.14196* (2024). <https://doi.org/10.48550/ARXIV.2401.14196> arXiv:2401.14196
- [107] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. 2024. Large Language Model based Multi-agents: A Survey of Progress and Challenges. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI 2024, Jeju, South Korea, August 3-9, 2024*. ijcai.org, 8048–8057. <https://www.ijcai.org/proceedings/2024/890>
- [108] Yaoqi Guo, Zhenpeng Chen, Jie M. Zhang, Yang Liu, and Yun Ma. 2025. Personality-Guided Code Generation Using Large Language Models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Vienna, Austria, 1068–1080. <https://doi.org/10.18653/v1/2025.acl-long.54>
- [109] Yu Hao, Weiteng Chen, Ziqiao Zhou, and Weidong Cui. 2023. E&V: Prompting Large Language Models to Perform Static Analysis by Pseudo-code Execution and Verification. *CoRR abs/2312.08477* (2023). <https://doi.org/10.48550/ARXIV.2312.08477> arXiv:2312.08477

- [110] Deepti Singh² Harish Chandra Maurya³, Ankit Thakur¹. 2015. A Survey on Incremental Software Development Life Cycle Model. *International Journal of Engineering Technology and Computer Research* 3, 2 (Apr. 2015). <https://ijetcr.org/index.php/ijetcr/article/view/144>
- [111] Konrad Halas. 2019. Mutpy: A Mutation Testing Tool for Python 3.x Source Code. <https://github.com/mutpy/mutpy>.
- [112] Junda He, Christoph Treude, and David Lo. 2025. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Trans. Softw. Eng. Methodol.* 34, 5, Article 124 (May 2025), 30 pages. <https://doi.org/10.1145/3712003>
- [113] Xinyi He, Jiaru Zou, Yun Lin, Mengyu Zhou, Shi Han, Zejian Yuan, and Dongmei Zhang. 2024. CoCoST: Automatic Complex Code Generation with Online Searching and Correctness Testing. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, November 12-16, 2024*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, 19433–19451. <https://doi.org/10.18653/V1/2024.EMNLP-MAIN.1082>
- [114] Dávid Hidvégi, Khashayar Etemadi, Sofia Bobadilla, and Martin Monperrus. 2024. CigaR: Cost-efficient Program Repair with LLMs. *CoRR* abs/2402.06598 (2024). <https://doi.org/10.48550/ARXIV.2402.06598> arXiv:2402.06598
- [115] Sirui Hong, Mingchen Zhuze, Jonathan Chen, Xiaowu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. <https://openreview.net/forum?id=VtmBAGCN7o>
- [116] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Trans. Softw. Eng. Methodol.* 33, 8 (2024), 220:1–220:79. <https://doi.org/10.1145/3695988>
- [117] Sihao Hu, Tiansheng Huang, Fatih Ilhan, Selim Furkan Tekin, and Ling Liu. 2023. Large Language Model-Powered Smart Contract Vulnerability Detection: New Perspectives. In *5th IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications, TPS-ISA 2023, Atlanta, GA, USA, November 1-4, 2023*. IEEE, 297–306. <https://doi.org/10.1109/TPS-ISA58951.2023.00044>
- [118] Dong Huang, Qingwen Bu, and Heming Cui. 2023. CodeCoT and Beyond: Learning to Program and Test like a Developer. *CoRR* abs/2308.08784 (2023). <https://doi.org/10.48550/ARXIV.2308.08784> arXiv:2308.08784
- [119] Dong Huang, Qingwen Bu, Jie M. Zhang, Michael Luck, and Heming Cui. 2023. AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation. *CoRR* abs/2312.13010 (2023). <https://doi.org/10.48550/ARXIV.2312.13010> arXiv:2312.13010
- [120] Xu Huang, Weiwen Liu, Xiaolong Chen, Xingmei Wang, Hao Wang, Defu Lian, Yasheng Wang, Ruiming Tang, and Enhong Chen. 2024. Understanding the Planning of LLM Agents: A Survey. *CoRR* abs/2402.02716 (2024). <https://doi.org/10.48550/ARXIV.2402.02716> arXiv:2402.02716
- [121] Yoichi Ishibashi and Yoshimasa Nishimura. 2024. Self-Organized Agents: A LLM Multi-Agent Framework toward Ultra Large-Scale Code Generation and Optimization. *CoRR* abs/2404.02183 (2024). <https://doi.org/10.48550/ARXIV.2404.02183> arXiv:2404.02183
- [122] Md. Ashrafur Islam, Mohammed Eunus Ali, and Md. Rizwan Parvez. 2024. MapCoder: Multi-Agent Code Generation for Competitive Problem Solving. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, 4912–4944. <https://doi.org/10.18653/V1/2024.ACL-LONG.269>
- [123] Ivar Jacobson, Grady Booch, and James Rumbaugh. 1999. The Unified Process. *Ieee Software* 16, 3 (1999), 96.
- [124] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2025. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net. <https://openreview.net/forum?id=chfJJYC3iL>
- [125] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2025. A Survey on Large Language Models for Code Generation. *ACM Trans. Softw. Eng. Methodol.* (July 2025). <https://doi.org/10.1145/3747588> Just Accepted.
- [126] Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. 2023. Impact of Code Language Models on Automated Program Repair. In *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 1430–1442. <https://doi.org/10.1109/ICSE48619.2023.00125>
- [127] Shuyang Jiang, Yuhao Wang, and Yu Wang. 2023. SelfEvolve: A Code Evolution Framework via Large Language Models. *CoRR* abs/2306.02907 (2023). <https://doi.org/10.48550/ARXIV.2306.02907> arXiv:2306.02907
- [128] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. <https://openreview.net/forum?id=VTF8yNQm66>
- [129] Dongming Jin, Zhi Jin, Xiaohong Chen, and Chunhui Wang. 2024. MARE: Multi-Agents Collaboration Framework for Requirements Engineering. *CoRR* abs/2405.03256 (2024). <https://doi.org/10.48550/ARXIV.2405.03256> arXiv:2405.03256
- [130] Michael Johnson, Robert Rosebrugh, and RJ Wood. 2002. Entity-relationship-attribute Designs and Sketches. *Theory and Applications of Categories* 10, 3 (2002), 94–112.

- [131] Harshit Joshi, José Pablo Cambronero Sánchez, Sumit Gulwani, Vu Le, Gust Verbruggen, and Ivan Radicek. 2023. Repair Is Nearly Generation: Multilingual Program Repair with LLMs. In *Thirty-Seventh AAAI Conference on Artificial Intelligence, AAAI 2023, Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence, IAAI 2023, Thirteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2023, Washington, DC, USA, February 7-14, 2023*, Brian Williams, Yiling Chen, and Jennifer Neville (Eds.). AAAI Press, 5131–5140. <https://doi.org/10.1609/AAAI.V37I4.25642>
- [132] Martin Josifoski, Lars Henning Klein, Maxime Peyrard, Yifei Li, Saibo Geng, Julian Paul Schnitzler, Yuxing Yao, Jiheng Wei, Debjit Paul, and Robert West. 2023. Flows: Building Blocks of Reasoning and Collaborating AI. *CoRR* abs/2308.01285 (2023). <https://doi.org/10.48550/ARXIV.2308.01285> arXiv:2308.01285
- [133] Charles Lee Isbell Jr., Christian R. Shelton, Michael J. Kearns, Satinder Singh, and Peter Stone. 2001. A Social Reinforcement Learning Agent. In *Proceedings of the Fifth International Conference on Autonomous Agents, AGENTS 2001, Montreal, Canada, May 28 - June 1, 2001*, Elisabeth André, Sandip Sen, Claude Frasson, and Jörg P. Müller (Eds.). ACM, 377–384. <https://doi.org/10.1145/375735.376334>
- [134] René Just, Darioush Jalali, and Michael D. Ernst. 2014. Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs. In *International Symposium on Software Testing and Analysis, ISSTA '14, San Jose, CA, USA - July 21 - 26, 2014*, Corina S. Pasareanu and Darko Marinov (Eds.). ACM, 437–440. <https://doi.org/10.1145/2610384.2628055>
- [135] Leslie Pack Kaelbling, Michael L. Littman, and Andrew W. Moore. 1996. Reinforcement Learning: A Survey. *J. Artif. Intell. Res.* 4 (1996), 237–285. <https://doi.org/10.1613/JAIR.301>
- [136] Sungmin Kang, Gabin An, and Shin Yoo. 2024. A Quantitative and Qualitative Evaluation of LLM-Based Explainable Fault Localization. *Proc. ACM Softw. Eng.* 1, FSE (2024), 1424–1446. <https://doi.org/10.1145/3660771>
- [137] Sungmin Kang, Bei Chen, Shin Yoo, and Jian-Guang Lou. 2025. Explainable Automated Debugging via Large Language Model-driven Scientific Debugging. *Empir. Softw. Eng.* 30, 2 (2025), 45. <https://doi.org/10.1007/S10664-024-10594-X>
- [138] Taghi M. Khoshgoftaar and Naeem Seliya. 2004. Comparative Assessment of Software Quality Classification Techniques: An Empirical Case Study. *Empir. Softw. Eng.* 9, 3 (2004), 229–257. <https://doi.org/10.1023/B:EMSE.0000027781.18360.9B>
- [139] Hatice Koç, Ali Mert Erdoğan, Yousef Barjakly, and Serhat Peker. 2021. UML Diagrams in Software Engineering Research: A Systematic Literature Review. In *Proceedings*, Vol. 74. MDPI, 13.
- [140] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large Language Models are Zero-Shot Reasoners. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh (Eds.). http://papers.nips.cc/paper_files/paper/2022/hash/8bb0d291acd4acf06ef112099c16f326-Abstract-Conference.html
- [141] Madhava Krishna, Bhagesh Gaur, Arsh Verma, and Pankaj Jalote. 2024. Using LLMs in Software Requirements Specifications: An Empirical Evaluation. In *32nd IEEE International Requirements Engineering Conference, RE 2024, Reykjavik, Iceland, June 24-28, 2024*, Grischä Liebel, Irit Hadar, and Paola Spoletini (Eds.). IEEE, 475–483. <https://doi.org/10.1109/RE59067.2024.00056>
- [142] Philip A Laplante, Naoufel Werghi, Christopher Lee Kuzmavl, Chris Verhof, Brian Henderson-Sellers, Joseph L Ganley, Ian Sommerville, Amos R Omondi, Ling Guan, Marco Gori, et al. 2017. *Dictionary of Computer Science, Engineering and Technology*. CRC Press.
- [143] Hung Le, Hailin Chen, Amrita Saha, Akash Gokul, Doyen Sahoo, and Shafiq Joty. 2024. CodeChain: Towards Modular Code Generation Through Chain of Self-revisions with Representative Sub-modules. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. <https://openreview.net/forum?id=vYhglxSj8j>
- [144] Cheryl Lee, Chunqiu Steven Xia, Jen-tse Huang, Zhouruixin Zhu, Lingming Zhang, and Michael R. Lyu. 2024. A Unified Debugging Approach via LLM-Based Multi-Agent Synergy. *CoRR* abs/2404.17153 (2024). <https://doi.org/10.48550/ARXIV.2404.17153> arXiv:2404.17153
- [145] Bin Lei, Yuchen Li, and Qiuwu Chen. 2024. AutoCoder: Enhancing Code Large Language Model with AIEV-Instruct. *CoRR* abs/2405.14906 (2024). <https://doi.org/10.48550/ARXIV.2405.14906> arXiv:2405.14906
- [146] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. 2023. CodaMosa: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models. In *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 919–931. <https://doi.org/10.1109/ICSE48619.2023.00085>
- [147] Chuanyi Li, Liguang Huang, Jidong Ge, Bin Luo, and Vincent Ng. 2018. Automatically Classifying User Requests in Crowdsourcing Requirements Engineering. *J. Syst. Softw.* 138 (2018), 108–123. <https://doi.org/10.1016/J.JSS.2017.12.028>
- [148] Chengpeng Li, Chenguang Zhu, Wenxi Wang, and August Shi. 2022. Repairing Order-Dependent Flaky Tests via Test Generation. In *44th IEEE/ACM 44th International Conference on Software Engineering, ICSE 2022, Pittsburgh, PA, USA, May 25-27, 2022*. ACM, 1881–1892. <https://doi.org/10.1145/3510003.3510173>
- [149] Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. CAMEL: Communicative Agents for “Mind” Exploration of Large Language Model Society. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (Eds.). http://papers.nips.cc/paper_files/paper/2023/hash/a3621ee907def47c1b952ade25c67698-Abstract-Conference.html

- [150] Haonan Li, Yu Hao, Yizhuo Zhai, and Zhiyun Qian. 2024. Enhancing Static Analysis for Practical Bug Detection: An LLM-Integrated Approach. *Proc. ACM Program. Lang.* 8, OOPSLA1 (2024), 474–499. <https://doi.org/10.1145/3649828>
- [151] Junyou Li, Qin Zhang, Yangbin Yu, Qiang Fu, and Deheng Ye. 2024. More Agents Is All You Need. *Trans. Mach. Learn. Res.* 2024 (2024). <https://openreview.net/forum?id=bgzUSZ8aeg>
- [152] Jia Li, Yunfei Zhao, Yongmin Li, Ge Li, and Zhi Jin. 2023. Towards Enhancing In-Context Learning for Code Generation. *CoRR abs/2303.17780* (2023). <https://doi.org/10.48550/ARXIV.2303.17780> arXiv:2303.17780
- [153] Lingwei Li, Li Yang, Huaxi Jiang, Jun Yan, Tiejian Luo, Zihan Hua, Geng Liang, and Chun Zuo. 2022. AUGER: Automatically Generating Review Comments with Pre-training Models. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022, Singapore, Singapore, November 14–18, 2022*, Abhik Roychoudhury, Cristian Cadar, and Miryung Kim (Eds.). ACM, 1009–1021. <https://doi.org/10.1145/3540250.3549099>
- [154] Xia Li, Wei Li, Yuqun Zhang, and Lingming Zhang. 2019. DeepFL: Integrating Multiple Fault Diagnosis Dimensions for Deep Fault Localization. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2019, Beijing, China, July 15–19, 2019*, Dongmei Zhang and Anders Möller (Eds.). ACM, 169–180. <https://doi.org/10.1145/3293882.3330574>
- [155] Yuanchun Li, Ziyue Yang, Yao Guo, and Xiangqun Chen. 2019. Humanoid: A Deep Learning-Based Approach to Automated Black-box Android App Testing. In *34th IEEE/ACM International Conference on Automated Software Engineering, ASE 2019, San Diego, CA, USA, November 11–15, 2019*. IEEE, 1070–1073. <https://doi.org/10.1109/ASE.2019.00104>
- [156] Ziyang Li, Saikat Dutta, and Mayur Naik. 2025. IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24–28, 2025*. OpenReview.net. <https://openreview.net/forum?id=9LdJDU7E91>
- [157] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Yawei Zhu, and Zhaoxuan Chen. 2022. SySeVR: A Framework for Using Deep Learning to Detect Software Vulnerabilities. *IEEE Trans. Dependable Secur. Comput.* 19, 4 (2022), 2244–2258. <https://doi.org/10.1109/TDSC.2021.3051525>
- [158] Bin Lin, Nathan Cassee, Alexander Serebrenik, Gabriele Bavota, Nicole Novielli, and Michele Lanza. 2022. Opinion Mining for Software Development: A Systematic Literature Review. *ACM Trans. Softw. Eng. Methodol.* 31, 3 (2022), 38:1–38:41. <https://doi.org/10.1145/3490388>
- [159] Derrick Lin, James Koppel, Angela Chen, and Armando Solar-Lezama. 2017. QuixBugs: A Multi-lingual Program Repair Benchmark Set Based on the Quixey Challenge. In *Proceedings Companion of the 2017 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity, SPLASH 2017, Vancouver, BC, Canada, October 23 – 27, 2017*, Gail C. Murphy (Ed.). ACM, 55–56. <https://doi.org/10.1145/3135932.3135941>
- [160] Feng Lin, Dong Jae Kim, and Tse-Hsun Chen. 2025. SOEN-101: Code Generation by Emulating Software Process Models Using Large Language Model Agents. In *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 – May 6, 2025*. IEEE, 1527–1539. <https://doi.org/10.1109/ICSE55347.2025.00140>
- [161] Junwei Liu, Yixuan Chen, Mingwei Liu, Xin Peng, and Yiling Lou. 2024. STALL+: Boosting LLM-based Repository-level Code Completion with Static Analysis. *CoRR abs/2406.10018* (2024). <https://doi.org/10.48550/ARXIV.2406.10018> arXiv:2406.10018
- [162] Jiawei Liu, Jinkun Lin, Fabian Ruffy, Cheng Tan, Jinyang Li, Aurojit Panda, and Lingming Zhang. 2023. NNSmith: Generating Diverse and Valid Test Cases for Deep Learning Compilers. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2023, Vancouver, BC, Canada, March 25–29, 2023*, Tor M. Aamodt, Natalie D. Enright Jerger, and Michael M. Swift (Eds.). ACM, 530–543. <https://doi.org/10.1145/3575693.3575707>
- [163] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 – 16, 2023*, Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (Eds.). http://papers.nips.cc/paper_files/paper/2023/hash/43e9d647ccd3e4b7b5baab53f0368686-Abstract-Conference.html
- [164] Junwei Liu, Chen Xu, Chong Wang, Tong Bai, Weitong Chen, Kaseng Wong, Yiling Lou, and Xin Peng. 2025. EvoDev: An Iterative Feature-Driven Framework for End-to-End Software Development with LLM-based Agents. *arXiv preprint arXiv:2511.02399* (2025).
- [165] Kaibo Liu, Zhenpeng Chen, Yiyang Liu, Jie M. Zhang, Mark Harman, Yudong Han, Yun Ma, Yihong Dong, Ge Li, and Gang Huang. 2025. LLM-Powered Test Case Generation for Detecting Bugs in Plausible Programs. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 430–440. <https://doi.org/10.18653/v1/2025.acl-long.20>
- [166] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. 2024. AgentBench: Evaluating LLMs as Agents. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7–11, 2024*. OpenReview.net. <https://openreview.net/forum?id=zAdUB0aCTQ>
- [167] Yue Liu, Sin Kit Lo, Qinghua Lu, Liming Zhu, Dehai Zhao, Xiwei Xu, Stefan Harrer, and Jon Whittle. 2025. Agent Design Pattern Catalogue: A Collection of Architectural Patterns for Foundation Model based Agents. *J. Syst. Softw.* 220 (2025), 112278. <https://doi.org/10.1016/J.JSS.2024.112278>

- [168] Ye Liu, Yue Xue, Daoyuan Wu, Yuqiang Sun, Yi Li, Miaolei Shi, and Yang Liu. 2025. PropertyGPT: LLM-driven Formal Verification of Smart Contracts through Retrieval-Augmented Property Generation. In *32nd Annual Network and Distributed System Security Symposium, NDSS 2025, San Diego, California, USA, February 24-28, 2025*. The Internet Society. <https://www.ndss-symposium.org/ndss-paper/propertygpt-llm-driven-formal-verification-of-smart-contracts-through-retrieval-augmented-property-generation/>
- [169] Yang Liu, Yuanshun Yao, Jean-Francois Ton, Xiaoying Zhang, Ruocheng Guo, Hao Cheng, Yegor Klochkov, Muhammad Faaiz Taufiq, and Hang Li. 2023. Trustworthy LLMs: A Survey and Guideline for Evaluating Large Language Models' Alignment. *CoRR* abs/2308.05374 (2023). <https://doi.org/10.48550/ARXIV.2308.05374> arXiv:2308.05374
- [170] Zhe Liu, Chunyang Chen, Junjie Wang, Xing Che, Yuekai Huang, Jun Hu, and Qing Wang. 2023. Fill in the Blank: Context-aware Automated Text Input Generation for Mobile GUI Testing. In *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 1355–1367. <https://doi.org/10.1109/ICSE48619.2023.00119>
- [171] Zhe Liu, Chunyang Chen, Junjie Wang, Mengzhuo Chen, Boyu Wu, Xing Che, Dandan Wang, and Qing Wang. 2024. Make LLM a Testing Expert: Bringing Human-like Interaction to Mobile GUI Testing via Functionality-aware Decisions. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14-20, 2024*. ACM, 100:1–100:13. <https://doi.org/10.1145/3597503.3639180>
- [172] Zhe Liu, Chunyang Chen, Junjie Wang, Mengzhuo Chen, Boyu Wu, Zhilin Tian, Yuekai Huang, Jun Hu, and Qing Wang. 2024. Testing the Limits: Unusual Text Inputs Generation for Mobile App Crash Detection with Large Language Model. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14-20, 2024*. ACM, 137:1–137:12. <https://doi.org/10.1145/3597503.3639118>
- [173] Zhe Liu, Cheng Li, Chunyang Chen, Junjie Wang, Boyu Wu, Yawen Wang, Jun Hu, and Qing Wang. 2024. Vision-driven Automated Mobile GUI Testing via Multimodal Large Language Model. *CoRR* abs/2407.03037 (2024). <https://doi.org/10.48550/ARXIV.2407.03037> arXiv:2407.03037
- [174] Zhuang Liu, Hanzhi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. 2022. A ConvNet for the 2020s. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022*. IEEE, 11966–11976. <https://doi.org/10.1109/CVPR52688.2022.01167>
- [175] Zijun Liu, Yanzhe Zhang, Peng Li, Yang Liu, and Diyi Yang. 2024. A Dynamic LLM-Powered Agent Network for Task-Oriented Agent Collaboration. In *First Conference on Language Modeling*. <https://openreview.net/forum?id=XII0Wp1XA9>
- [176] Vadim Liventsev, Anastasiia Grishina, Aki Härmä, and Leon Moonen. 2023. Fully Autonomous Programming with Large Language Models. In *Proceedings of the Genetic and Evolutionary Computation Conference, GECCO 2023, Lisbon, Portugal, July 15-19, 2023*, Sara Silva and Luis Paquete (Eds.). ACM, 1146–1155. <https://doi.org/10.1145/3583131.3590481>
- [177] Yiling Lou, Ali Ghanbari, Xia Li, Lingming Zhang, Haotian Zhang, Dan Hao, and Lu Zhang. 2020. Can Automated Program Repair Refine Fault Localization? A Unified Debugging Approach. In *ISSTA '20: 29th ACM SIGSOFT International Symposium on Software Testing and Analysis, Virtual Event, USA, July 18-22, 2020*, Sarfraz Khurshid and Corina S. Pasareanu (Eds.). ACM, 75–87. <https://doi.org/10.1145/3395363.3397351>
- [178] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, Tianyang Liu, Max Tian, Denis Kocetkov, Arthur Zucker, Younes Belkada, Zijian Wang, Qian Liu, Dmitry Abul'khanov, Indraneil Paul, Zhuang Li, Wen-Ding Li, Megan Risdal, Jia Li, Jian Zhu, Terry Yue Zhuo, Evgenii Zheltonozhskii, Nii Osae Osae Dade, Wenhao Yu, Lucas Krauß, Naman Jain, Yixuan Su, Xuanli He, Manan Dey, Edoardo Abati, Yekun Chai, Niklas Muennighoff, Xiangru Tang, Muhtasham Oblokulov, Christopher Akiki, Marc Marone, Chenghao Mou, Mayank Mishra, Alex Gu, Binyuan Hui, Tri Dao, Armel Zebaze, Olivier Dehaene, Nicolas Patry, Canwen Xu, Julian J. McAuley, Han Hu, Torsten Scholak, Sébastien Paquet, Jennifer Robinson, Carolyn Jane Anderson, Nicolas Chapados, and et al. 2024. StarCoder 2 and The Stack v2: The Next Generation. *CoRR* abs/2402.19173 (2024). <https://doi.org/10.48550/ARXIV.2402.19173> arXiv:2402.19173
- [179] Lechanceux Luhunu and Eugene Syriani. 2017. Survey on Template-based Code Generation. In *Proceedings of MODELS 2017 Satellite Event: Workshops (ModComp, ME, EXE, COMMiMDE, MRT, MULTI, GEMOC, MoDeVva, MDETools, FlexMDE, MDEbug), Posters, Doctoral Symposium, Educator Symposium, ACM Student Research Competition, and Tools and Demonstrations co-located with ACM/IEEE 20th International Conference on Model Driven Engineering Languages and Systems (MODELS 2017), Austin, TX, USA, September, 17, 2017 (CEUR Workshop Proceedings, Vol. 2019)*, Loli Burgueño, Jonathan Corley, Nelly Bencomo, Peter J. Clarke, Philippe Collet, Michalis Famelis, Sudipto Ghosh, Martin Gogolla, Joel Greenyer, Esther Guerra, Sahar Kokaly, Alfonso Pierantonio, Julia Rubin, and Davide Di Ruscio (Eds.). CEUR-WS.org, 468–471. https://ceur-ws.org/Vol-2019/posters_3.pdf
- [180] Dipeeka Luitel, Shabnam Hassani, and Mehrdad Sabetzadeh. 2024. Improving Requirements Completeness: Automated Assistance through Large Language Models. *Requir. Eng.* 29, 1 (2024), 73–95. <https://doi.org/10.1007/S00766-024-00416-3>
- [181] Stephan Lukasczyk and Gordon Fraser. 2022. Pynguin: Automated Unit Test Generation for Python. In *44th IEEE/ACM International Conference on Software Engineering: Companion Proceedings, ICSE Companion 2022, Pittsburgh, PA, USA, May 22-24, 2022*. ACM/IEEE, 168–172. <https://doi.org/10.1145/3510454.3516829>
- [182] Qinyu Luo, Yining Ye, Shihao Liang, Zhong Zhang, Yujia Qin, Yaxi Lu, Yesai Wu, Xin Cong, Yankai Lin, Yingli Zhang, Xiaoyin Che, Zhiyuan Liu, and Maosong Sun. 2024. RepoAgent: An LLM-Powered Open-Source Framework for Repository-level Code

- Documentation Generation. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, Delia Irazu Hernandez Farias, Tom Hope, and Manling Li (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 436–464. <https://doi.org/10.18653/v1/2024.emnlp-demo.46>
- [183] Xianchang Luo, Yinxing Xue, Zhenchang Xing, and Jiamou Sun. 2022. PRCBERT: Prompt Learning for Requirement Classification using BERT-based Pretrained Language Models. In *37th IEEE/ACM International Conference on Automated Software Engineering, ASE 2022, Rochester, MI, USA, October 10–14, 2022*. ACM, 75:1–75:13. <https://doi.org/10.1145/3551349.3560417>
- [184] Chang Ma, Junlei Zhang, Zhihao Zhu, Cheng Yang, Yujiu Yang, Yaohui Jin, Zhenzhong Lan, Lingpeng Kong, and Junxian He. 2024. AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 – 15, 2024*, Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang (Eds.). http://papers.nips.cc/paper_files/paper/2024/hash/877b40688e330a0e2a3fc24084208dfa-Abstract-Datasets_and_Benchmarks_Track.html
- [185] Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. 2025. SpecGen: Automated Generation of Formal Program Specifications via Large Language Models. In *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 – May 6, 2025*. IEEE, 16–28. <https://doi.org/10.1109/ICSE55347.2025.00129>
- [186] Wei Ma, Daoyuan Wu, Yuqiang Sun, Tianwen Wang, Shangqing Liu, Jian Zhang, Yue Xue, and Yang Liu. 2025. Combining Fine-Tuning and LLM-Based Agents for Intuitive Smart Contract Auditing with Justifications. In *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 – May 6, 2025*. IEEE, 1742–1754. <https://doi.org/10.1109/ICSE55347.2025.00027>
- [187] Yingwei Ma, Qingping Yang, Rongyu Cao, Binhua Li, Fei Huang, and Yongbin Li. 2024. Alibaba LingmaAgent: Improving Automated Issue Resolution via Comprehensive Repository Exploration. *CoRR* abs/2406.01422 (2024). <https://doi.org/10.48550/ARXIV.2406.01422> arXiv:2406.01422
- [188] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 – 16, 2023*, Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (Eds.). http://papers.nips.cc/paper_files/paper/2023/hash/91edff07232fb1b55a505a9e9f6c0ff3-Abstract-Conference.html
- [189] Zhenyu Mao, Jialong Li, Dongming Jin, Munan Li, and Kenji Tei. 2024. Multi-Role Consensus Through LLMs Discussions for Vulnerability Detection. In *24th IEEE International Conference on Software Quality, Reliability, and Security, QRS - Companion, Cambridge, United Kingdom, July 1–5, 2024*. IEEE, 1318–1319. <https://doi.org/10.1109/QRS-C63300.2024.00173>
- [190] Tula Masterman, Sandi Besen, Mason Sawtell, and Alex Chao. 2024. The Landscape of Emerging AI Agent Architectures for Reasoning, Planning, and Tool Calling: A Survey. *CoRR* abs/2404.11584 (2024). <https://doi.org/10.48550/ARXIV.2404.11584> arXiv:2404.11584
- [191] Antonio Mastropaolo, Simone Scalabrino, Nathan Cooper, David Nader-Palacio, Denys Poshyvanyk, Rocco Oliveto, and Gabriele Bavota. 2021. Studying the Usage of Text-To-Text Transfer Transformer to Support Code-Related Tasks. In *43rd IEEE/ACM International Conference on Software Engineering, ICSE 2021, Madrid, Spain, 22–30 May 2021*. IEEE, 336–347. <https://doi.org/10.1109/ICSE43902.2021.00041>
- [192] George Mathew, Amritanshu Agrawal, and Tim Menzies. 2023. Finding Trends in Software Research. *IEEE Trans. Software Eng.* 49, 4 (2023), 1397–1410. <https://doi.org/10.1109/TSE.2018.2870388>
- [193] Noble Saji Mathews and Meiyappan Nagappan. 2024. Test-Driven Development and LLM-based Code Generation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE 2024, Sacramento, CA, USA, October 27 – November 1, 2024*, Vladimir Filkov, Baishakhi Ray, and Minghui Zhou (Eds.). ACM, 1583–1594. <https://doi.org/10.1145/3691620.3695527>
- [194] Sjouke Mauw and Martijn Oostdijk. 2005. Foundations of Attack Trees. In *Information Security and Cryptology - ICISC 2005, 8th International Conference, Seoul, Korea, December 1–2, 2005, Revised Selected Papers (Lecture Notes in Computer Science, Vol. 3935)*, Dongho Won and Seungjoo Kim (Eds.). Springer, 186–198. https://doi.org/10.1007/11734727_17
- [195] Grégoire Mialon, Roberto Dessì, Maria Lomeli, Christoforos Nalmpantis, Ramakanth Pasunuru, Roberta Raileanu, Baptiste Rozière, Timo Schick, Jane Dwivedi-Yu, Asli Celikyilmaz, Edouard Grave, Yann LeCun, and Thomas Scialom. 2023. Augmented Language Models: A Survey. *Trans. Mach. Learn. Res.* 2023 (2023). <https://openreview.net/forum?id=jh7wH2AzKK>
- [196] Marvin Minsky. 1961. Steps Toward Artificial Intelligence. *Proceedings of the IRE* 49, 1 (1961), 8–30.
- [197] Fangwen Mu, Lin Shi, Song Wang, Zhuohao Yu, Binqun Zhang, ChenXue Wang, Shichao Liu, and Qing Wang. 2024. ClarifyGPT: A Framework for Enhancing LLM-Based Code Generation via Requirements Clarification. *Proc. ACM Softw. Eng.* 1, FSE, Article 103 (July 2024), 23 pages. <https://doi.org/10.1145/3660810>
- [198] Peter Naur and Brian Randell. 1969. *Software Engineering: Report of a conference sponsored by the NATO Science Committee, Garmisch, Germany, 7–11 Oct. 1968, Brussels, Scientific Affairs Division, NATO*.
- [199] Farhana Nazir, Wasi Haider Butt, Muhammad Waseem Anwar, and Muazzam Ali Khan Khattak. 2017. The Applications of Natural Language Processing (NLP) for Software Requirement Engineering - A Systematic Literature Review. In *Information Science and Applications 2017 - ICISA 2017, Macau, China, 20–23 March 2017 (Lecture Notes in Electrical Engineering, Vol. 424)*, Kuinam Kim and

- Nikolai Joukov (Eds.). Springer, 485–493. https://doi.org/10.1007/978-981-10-4154-9_56
- [200] Christian D. Newman, Tessandra Sage, Michael L. Collard, Hakam W. Alomari, and Jonathan I. Maletic. 2016. srcSlice: A Tool For Efficient Static Forward Slicing. In *Proceedings of the 38th International Conference on Software Engineering, ICSE 2016, Austin, TX, USA, May 14-22, 2016 - Companion Volume*, Laura K. Dillon, Willem Visser, and Laurie A. Williams (Eds.). ACM, 621–624. <https://doi.org/10.1145/2889160.2889173>
- [201] Minh Huynh Nguyen, Thang Phan Chau, Phong X. Nguyen, and Nghi D. Q. Bui. 2025. AgileCoder: Dynamic Collaborative Agents for Software Development based on Agile Methodology. In *2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge)*. IEEE Computer Society, Los Alamitos, CA, USA, 156–167. <https://doi.org/10.1109/Forge66646.2025.00026>
- [202] Bashar Nuseibeh and Steve M. Easterbrook. 2000. Requirements Engineering: A Roadmap. In *22nd International Conference on Software Engineering, Future of Software Engineering Track, ICSE 2000, Limerick Ireland, June 4-11, 2000*, Anthony Finkelstein (Ed.). ACM, 35–46. <https://doi.org/10.1145/336512.336523>
- [203] Theo X. Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. 2024. Is Self-Repair a Silver Bullet for Code Generation?. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. <https://openreview.net/forum?id=y0GJXRungR>
- [204] Kaan Ozkara, Tao Yu, and Youngsuk Park. 2025. Stochastic Rounding for LLM Training: Theory and Practice. CoRR abs/2502.20566 (2025). <https://doi.org/10.48550/ARXIV.2502.20566> arXiv:2502.20566
- [205] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: A Method for Automatic Evaluation of Machine Translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics, July 6-12, 2002, Philadelphia, PA, USA*. ACL, 311–318. <https://doi.org/10.3115/1073083.1073135>
- [206] Bhargavi Paranjape, Scott M. Lundberg, Sameer Singh, Hannaneh Hajishirzi, Luke Zettlemoyer, and Marco Túlio Ribeiro. 2023. ART: Automatic Multi-step Reasoning and Tool-use for Large Language Models. CoRR abs/2303.09014 (2023). <https://doi.org/10.48550/ARXIV.2303.09014> arXiv:2303.09014
- [207] Terence John Parr and Russell W. Quong. 1995. ANTLR: A Predicated- $LL(k)$ Parser Generator. *Softw. Pract. Exp.* 25, 7 (1995), 789–810. <https://doi.org/10.1002/SPE.4380250705>
- [208] Sanyogita Piya and Allison Sullivan. 2024. LLM4TDD: Best Practices for Test Driven Development Using Large Language Models. In *LLM4CODE@ICSE*. 14–21. <https://doi.org/10.1145/3643795.3648382>
- [209] Juan Altmayer Pizzorno and Emery D. Berger. 2023. SlipCover: Near Zero-Overhead Code Coverage for Python. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2023, Seattle, WA, USA, July 17-21, 2023*, René Just and Gordon Fraser (Eds.). ACM, 1195–1206. <https://doi.org/10.1145/3597926.3598128>
- [210] Klaus Pohl. 1996. *Requirements Engineering: An Overview*. RWTH, Fachgruppe Informatik Aachen.
- [211] Russell A Poldrack, Thomas Lu, and Gasper Begus. 2023. AI-assisted Coding: Experiments with GPT-4. CoRR abs/2304.13187 (2023). <https://doi.org/10.48550/ARXIV.2304.13187> arXiv:2304.13187
- [212] Chen Qian, Yufan Dang, Jiahao Li, Wei Liu, Zihao Xie, Yifei Wang, Weize Chen, Cheng Yang, Xin Cong, Xiaoyin Che, Zhiyuan Liu, and Maosong Sun. 2024. Experiential Co-Learning of Software-Developing Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, 5628–5640. <https://doi.org/10.18653/V1/2024.ACL-LONG.305>
- [213] Chen Qian, Jiahao Li, Yufan Dang, Wei Liu, Yifei Wang, Zihao Xie, Weize Chen, Cheng Yang, Yingli Zhang, Zhiyuan Liu, and Maosong Sun. 2024. Iterative Experience Refinement of Software-Developing Agents. CoRR abs/2405.04219 (2024). <https://doi.org/10.48550/ARXIV.2405.04219> arXiv:2405.04219
- [214] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ChatDev: Communicative Agents for Software Development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, 15174–15186. <https://doi.org/10.18653/V1/2024.ACL-LONG.810>
- [215] Chen Qian, Zihao Xie, Yifei Wang, Wei Liu, Kunlun Zhu, Hanchen Xia, Yufan Dang, Zhuoyun Du, Weize Chen, Cheng Yang, Zhiyuan Liu, and Maosong Sun. 2025. Scaling Large Language Model-based Multi-Agent Collaboration. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net. <https://openreview.net/forum?id=K3n5jPkrU6>
- [216] Yihao Qin, Shangwen Wang, Yiling Lou, Jinhao Dong, Kaixin Wang, Xiaoling Li, and Xiaoguang Mao. 2025. SoapFL: A Standard Operating Procedure for LLM-Based Method-Level Fault Localization. *IEEE Trans. Software Eng.* 51, 4 (2025), 1173–1187. <https://doi.org/10.1109/TSE.2025.3543187>
- [217] Aida Radu and Sarah Nadi. 2019. A Dataset of Non-functional Bugs. In *Proceedings of the 16th International Conference on Mining Software Repositories, MSR 2019, 26-27 May 2019, Montreal, Canada*, Margaret-Anne D. Storey, Bram Adams, and Sonia Haiduc (Eds.). IEEE / ACM, 399–403. <https://doi.org/10.1109/MSR.2019.00066>

- [218] Alejandro Rago, Claudia A. Marcos, and J. Andrés Díaz Pace. 2013. Uncovering Quality-attribute Concerns in Use Case Specifications via Early Aspect Mining. *Requir. Eng.* 18, 1 (2013), 67–84. <https://doi.org/10.1007/S00766-011-0142-Z>
- [219] Alfin Wijaya Rahardja, Junwei Liu, Weitong Chen, Zhenpeng Chen, and Yiling Lou. 2025. Can Agents Fix Agent Issues?. In *Advances in Neural Information Processing Systems 39: Annual Conference on Neural Information Processing Systems 2025, NeurIPS 2025*.
- [220] Zeeshan Rasheed, Malik Abdul Sami, Muhammad Waseem, Kai-Kristian Kemell, Xiaofeng Wang, Anh Nguyen, Kari Systä, and Pekka Abrahamsson. 2024. AI-powered Code Review with LLMs: Early Results. *CoRR abs/2404.18496* (2024). <https://doi.org/10.48550/ARXIV.2404.18496> arXiv:2404.18496
- [221] Zeeshan Rasheed, Muhammad Waseem, Mika Saari, Kari Systä, and Pekka Abrahamsson. 2024. CodePori: Large Scale Model for Autonomous Software Development by Using Multi-Agents. *CoRR abs/2402.01411* (2024). <https://doi.org/10.48550/ARXIV.2402.01411> arXiv:2402.01411
- [222] Zeeshan Rasheed, Muhammad Waseem, Malik Abdul Sami, Kai-Kristian Kemell, Aakash Ahmad, Anh Nguyen-Duc, Kari Systä, and Pekka Abrahamsson. 2024. Autonomous Agents in Software Development: A Vision Paper. In *Agile Processes in Software Engineering and Extreme Programming - Workshops - XP 2024 Workshops, Bozen-Bolzano, Italy, June 4-7, 2024, Revised Selected Papers (Lecture Notes in Business Information Processing, Vol. 524)*, Lodovica Marchesi, Alfredo Goldman, Maria Ilaria Lunesu, Adam Przybyłek, Ademar Aguiar, Lorraine Morgan, Xiaofeng Wang, and Andrea Pinna (Eds.). Springer, 15–23. https://doi.org/10.1007/978-3-031-72781-8_2
- [223] Xiaoxue Ren, Xinyuan Ye, Dehai Zhao, Zhenchang Xing, and Xiaohu Yang. 2023. From Misuse to Mastery: Enhancing Code Generation with Knowledge-Driven AI Chaining. In *38th IEEE/ACM International Conference on Automated Software Engineering, ASE 2023, Luxembourg, September 11-15, 2023*. IEEE, 976–987. <https://doi.org/10.1109/ASE56229.2023.00143>
- [224] Carlos H. C. Ribeiro. 2002. Reinforcement Learning Agents. *Artif. Intell. Rev.* 17, 3 (2002), 223–250. <https://doi.org/10.1023/A:1015008417172>
- [225] Tal Ridnik, Dedy Kredo, and Itamar Friedman. 2024. Code Generation with AlphaCodium: From Prompt Engineering to Flow Engineering. *CoRR abs/2401.08500* (2024). <https://doi.org/10.48550/ARXIV.2401.08500> arXiv:2401.08500
- [226] Stephen E. Robertson and Steve Walker. 1994. Some Simple Effective Approximations to the 2-Poisson Model for Probabilistic Weighted Retrieval. In *Proceedings of the 17th Annual International ACM-SIGIR Conference on Research and Development in Information Retrieval, Dublin, Ireland, 3-6 July 1994 (Special Issue of the SIGIR Forum)*, W. Bruce Croft and C. J. van Rijsbergen (Eds.). ACM/Springer, 232–241. https://doi.org/10.1007/978-1-4471-2099-5_24
- [227] Krishna Ronanki, Beatriz Cabrero Daniel, and Christian Berger. 2023. ChatGPT as a Tool for User Story Quality Evaluation: Trustworthy Out of the Box?. In *Agile Processes in Software Engineering and Extreme Programming - Workshops - XP 2022 Workshops, Copenhagen, Denmark, June 13-17, 2022, and XP 2023 Workshops, Amsterdam, The Netherlands, June 13-16, 2023, Revised Selected Papers (Lecture Notes in Business Information Processing, Vol. 489)*, Philippe Kruchten and Peggy Gregory (Eds.). Springer, 173–181. https://doi.org/10.1007/978-3-031-48550-3_17
- [228] W. W. Royce. 1987. Managing the Development of Large Software Systems: Concepts and Techniques. In *Proceedings, 9th International Conference on Software Engineering, Monterey, California, USA, March 30 - April 2, 1987*, William E. Riddle, Robert M. Balzer, and Kouichi Kishida (Eds.). ACM Press, 328–339. <http://dl.acm.org/citation.cfm?id=41801>
- [229] Haifeng Ruan, Yuntong Zhang, and Abhik Roychoudhury. 2025. SpecRover: Code Intent Extraction via LLMs. In *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 - May 6, 2025*. IEEE, 963–974. <https://doi.org/10.1109/ICSE55347.2025.00080>
- [230] Malik Abdul Sami, Muhammad Waseem, Zeeshan Rasheed, Mika Saari, Kari Systä, and Pekka Abrahamsson. 2024. Experimenting with Multi-Agent Software Development: Towards a Unified Platform. *CoRR abs/2406.05381* (2024). <https://doi.org/10.48550/ARXIV.2406.05381> arXiv:2406.05381
- [231] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2024. An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. *IEEE Trans. Software Eng.* 50, 1 (2024), 85–105. <https://doi.org/10.1109/TSE.2023.3334955>
- [232] Freda Shi, Mirac Suzgun, Markus Freitag, Xuezhi Wang, Suraj Srivats, Soroush Vosoughi, Hyung Won Chung, Yi Tay, Sebastian Ruder, Denny Zhou, Dipanjan Das, and Jason Wei. 2023. Language Models Are Multilingual Chain-of-thought Reasoners. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net. <https://openreview.net/forum?id=fR3wGCK-IXp>
- [233] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (Eds.). http://papers.nips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html
- [234] Mohit Shridhar, Jesse Thomason, Daniel Gordon, Yonatan Bisk, Winson Han, Roozbeh Mottaghi, Luke Zettlemoyer, and Dieter Fox. 2020. ALFRED: A Benchmark for Interpreting Grounded Instructions for Everyday Tasks. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2020, Seattle, WA, USA, June 13-19, 2020*. Computer Vision Foundation / IEEE, 10737–10746. <https://doi.org/10.1109/CVPR42600.2020.01075>

- [235] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew J. Hausknecht. 2021. ALFWorld: Aligning Text and Embodied Environments for Interactive Learning. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net. <https://openreview.net/forum?id=0IOX0YcCdTn>
- [236] Vivek Shukla, Dharendra Pandey, and Raj Shree. 2015. Requirements Engineering: A Survey. *Communications on Applied Electronics* 3, 5 (2015), 28–31.
- [237] Alexander Shypula, Aman Madaan, Yimeng Zeng, Uri Alon, Jacob R. Gardner, Yiming Yang, Milad Hashemi, Graham Neubig, Parthasarathy Ranganathan, Osbert Bastani, and Amir Yazdanbakhsh. 2024. Learning Performance-Improving Code Edits. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. <https://openreview.net/forum?id=ix7rLVHXyY>
- [238] Susan Elliott Sim, Steve M. Easterbrook, and Richard C. Holt. 2003. Using Benchmarking to Advance Research: A Challenge to Software Engineering. In *Proceedings of the 25th International Conference on Software Engineering, May 3-10, 2003, Portland, Oregon, USA*, Lori A. Clarke, Laurie Dillon, and Walter F. Tichy (Eds.). IEEE Computer Society, 74–83. <https://doi.org/10.1109/ICSE.2003.1201189>
- [239] Victor Sobreira, Thomas Durieux, Fernanda Madeiral, Martin Monperrus, and Marcelo de Almeida Maia. 2018. Dissection of a Bug Dataset: Anatomy of 395 Patches from Defects4J. In *25th International Conference on Software Analysis, Evolution and Reengineering, SANER 2018, Campobasso, Italy, March 20-23, 2018*, Rocco Oliveto, Massimiliano Di Penta, and David C. Shepherd (Eds.). IEEE Computer Society, 130–140. <https://doi.org/10.1109/SANER.2018.8330203>
- [240] Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shoeb, Abubakar Abid, Adam Fisch, Adam R. Brown, Adam Santoro, Aditya Gupta, Adrià Garriga-Alonso, Agnieszka Kluska, Aitor Lewkowycz, Akshat Agarwal, and etc. 2023. Beyond the Imitation Game: Quantifying and Extrapolating the Capabilities of Language Models. *Trans. Mach. Learn. Res.* 2023 (2023). <https://openreview.net/forum?id=uyTL5Bvosj>
- [241] Ting Su, Guozhu Meng, Yuting Chen, Ke Wu, Weiming Yang, Yao Yao, Geguang Pu, Yang Liu, and Zhendong Su. 2017. Guided, Stochastic Model-based GUI Testing of Android Apps. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering, ESEC/FSE 2017, Paderborn, Germany, September 4-8, 2017*, Eric Bodden, Wilhelm Schäfer, Arie van Deursen, and Andrea Zisman (Eds.). ACM, 245–256. <https://doi.org/10.1145/3106237.3106298>
- [242] Yuqiang Sun, Daoyuan Wu, Yue Xue, Han Liu, Wei Ma, Lyuye Zhang, Miaolei Shi, and Yang Liu. 2024. LLM4Vuln: A Unified Evaluation Framework for Decoupling and Enhancing LLMs' Vulnerability Reasoning. *CoRR* abs/2401.16185 (2024). <https://doi.org/10.48550/ARXIV.2401.16185>
- [243] Nikhil Swamy, Tahina Ramananandro, Aseem Rastogi, Irina Spiridonova, Haobin Ni, Dmitry Malloy, Juan Vazquez, Michael Tang, Omar Cardona, and Arti Gupta. 2022. Hardening Attack Surfaces with Formally Proven Binary Format Parsers. In *PLDI '22: 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation, San Diego, CA, USA, June 13 - 17, 2022*, Ranjit Jhala and Isil Dillig (Eds.). ACM, 31–45. <https://doi.org/10.1145/3519939.3523708>
- [244] Eugene Syriani, Lechanceux Luhunu, and Houari A. Sahraoui. 2018. Systematic Mapping Study of Template-based Code Generation. *Comput. Lang. Syst. Struct.* 52 (2018), 43–62. <https://doi.org/10.1016/J.CL.2017.11.003>
- [245] Maryam Taeb, Amanda Swearngin, Eldon Schoop, Ruijia Cheng, Yue Jiang, and Jeffrey Nichols. 2024. AXNav: Replaying Accessibility Tests from Natural Language. In *Proceedings of the CHI Conference on Human Factors in Computing Systems, CHI 2024, Honolulu, HI, USA, May 11-16, 2024*, Florian 'Floyd' Mueller, Penny Kyburz, Julie R. Williamson, Corina Sas, Max L. Wilson, Phoebe O. Touns Dugas, and Irina Shklovski (Eds.). ACM, 962:1–962:16. <https://doi.org/10.1145/3613904.3642777>
- [246] Yashar Talebirad and Amirhossein Nadiri. 2023. Multi-Agent Collaboration: Harnessing the Power of Intelligent LLM Agents. *CoRR* abs/2306.03314 (2023). <https://doi.org/10.48550/ARXIV.2306.03314>
- [247] Jun Tang, Zhibo Yang, Yongpan Wang, Qi Zheng, Yongchao Xu, and Xiang Bai. 2019. SegLink++: Detecting Dense and Arbitrary-shaped Scene Text by Instance-aware Component Grouping. *Pattern Recognit.* 96 (2019). <https://doi.org/10.1016/J.PATCOG.2019.06.020>
- [248] Xunzhu Tang, Kisub Kim, Yewei Song, Cedric Lothritz, Bei Li, Saad Ezzini, Haoye Tian, Jacques Klein, and Tegawendé F. Bissyandé. 2024. CodeAgent: Autonomous Communicative Agents for Code Review. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, November 12-16, 2024*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, 11279–11313. <https://doi.org/10.18653/V1/2024.EMNLP-MAIN.632>
- [249] Wei Tao, Yucheng Zhou, Yanlin Wang, Wenqiang Zhang, Hongyu Zhang, and Yu Cheng. 2024. MAGIS: LLM-Based Multi-Agent Framework for GitHub Issue Resolution. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang (Eds.). http://papers.nips.cc/paper_files/paper/2024/hash/5d1f02132ef51602adf07000ca5b6138-Abstract-Conference.html
- [250] Zhao Tian, Junjie Chen, and Xiangyu Zhang. 2025. Fixing Large Language Models' Specification Misunderstanding for Better Code Generation. In *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 - May 6, 2025*. IEEE, 1514–1526. <https://doi.org/10.1109/ICSE55347.2025.00108>
- [251] Haoxin Tu, Zhide Zhou, He Jiang, Imam Nur Bani Yusuf, Yuxian Li, and Lingxiao Jiang. 2024. Isolating Compiler Bugs by Generating Effective Witness Programs With Large Language Models. *IEEE Trans. Software Eng.* 50, 7 (2024), 1768–1788. <https://doi.org/10.1109/>

- TSE.2024.3397822
- [252] Michele Tufano, Anisha Agarwal, Jinu Jang, Roshanak Zilouchian Moghaddam, and Neel Sundaresan. 2024. AutoDev: Automated AI-Driven Development. *CoRR* abs/2403.08299 (2024). <https://doi.org/10.48550/ARXIV.2403.08299> arXiv:2403.08299
 - [253] Muhammad Aminu Umar and Kevin Lano. 2024. Advances in Automated Support for Requirements Engineering: A Systematic Literature Review. *Requir. Eng.* 29, 2 (2024), 177–207. <https://doi.org/10.1007/S00766-023-00411-0>
 - [254] Nalin Wadhwa, Jui Pradhan, Atharv Sonwane, Surya Prakash Sahu, Nagarajan Natarajan, Aditya Kanade, Suresh Parthasarathy, and Sriram K. Rajamani. 2024. CORE: Resolving Code Quality Issues using LLMs. *Proc. ACM Softw. Eng.* 1, FSE (2024), 789–811. <https://doi.org/10.1145/3643762>
 - [255] Nalin Wadhwa, Atharv Sonwane, Daman Arora, Abhav Mehrotra, Saiteja Utpala, Ramakrishna B Bairi, Aditya Kanade, and Nagarajan Natarajan. 2024. MASAI: Modular Architecture for Software-engineering AI Agents. In *NeurIPS 2024 Workshop on Open-World Agents*. <https://openreview.net/forum?id=NSInt8ILYB>
 - [256] Chong Wang, Zhenpeng Chen, Tianlin Li, Yilun Zhang, and Yang Liu. 2025. Towards Trustworthy LLMs for Code: A Data-Centric Synergistic Auditing Framework. In *47th IEEE/ACM International Conference on Software Engineering: New Ideas and Emerging Results, ICSE 2025 - NIER*. 56–60.
 - [257] Chong Wang, Jian Zhang, Yebo Feng, Tianlin Li, Weisong Sun, Yang Liu, and Xin Peng. 2025. Teaching Code LLMs to Use Autocompletion Tools in Repository-Level Code Generation. *ACM Trans. Softw. Eng. Methodol.* (Jan. 2025). <https://doi.org/10.1145/3714462> Just Accepted.
 - [258] Chengpeng Wang, Wuqi Zhang, Zian Su, Xiangzhe Xu, Xiaoheng Xie, and Xiangyu Zhang. 2024. LLMDFa: Analyzing Dataflow in Code with Large Language Models. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang (Eds.). http://papers.nips.cc/paper_files/paper/2024/hash/ed9dcde1eb9c597f68c1d375bbe3fc-Abstract-Conference.html
 - [259] Hanbin Wang, Zhenghao Liu, Shuo Wang, Ganqu Cui, Ning Ding, Zhiyuan Liu, and Ge Yu. 2024. INTERVENOR: Prompting the Coding Ability of Large Language Models with the Interactive Chain of Repair. In *Findings of the Association for Computational Linguistics, ACL 2024, Bangkok, Thailand and virtual meeting, August 11-16, 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, 2081–2107. <https://doi.org/10.18653/V1/2024.FINDINGS-ACL.124>
 - [260] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software Testing With Large Language Models: Survey, Landscape, and Vision. *IEEE Trans. Software Eng.* 50, 4 (2024), 911–936. <https://doi.org/10.1109/TSE.2024.3368208>
 - [261] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. 2024. A Survey on Large Language Model based Autonomous Agents. *Frontiers Comput. Sci.* 18, 6 (2024), 186345. <https://doi.org/10.1007/S11704-024-40231-1>
 - [262] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. 2024. Executable Code Actions Elicit Better LLM Agents. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net. <https://openreview.net/forum?id=jj9BoXAffa>
 - [263] Xingyao Wang, Zihan Wang, Jiateng Liu, Yangyi Chen, Lifan Yuan, Hao Peng, and Heng Ji. 2024. MINT: Evaluating LLMs in Multi-turn Interaction with Tools and Language Feedback. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. <https://openreview.net/forum?id=jp3gWrMuLZ>
 - [264] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net. <https://openreview.net/forum?id=1PL1NIMMrw>
 - [265] Zihao Wang, Anji Liu, Haowei Lin, Jiaqi Li, Xiaojian Ma, and Yitao Liang. 2024. RAT: Retrieval Augmented Thoughts Elicit Context-Aware Reasoning in Long-Horizon Generation. *CoRR* abs/2403.05313 (2024). <https://doi.org/10.48550/ARXIV.2403.05313> arXiv:2403.05313
 - [266] Zefan Wang, Zichuan Liu, Yingying Zhang, Aoxiao Zhong, Jihong Wang, Fengbin Yin, Lunting Fan, Lingfei Wu, and Qingsong Wen. 2024. RAgent: Cloud Root Cause Analysis by Autonomous Agents with Tool-Augmented Large Language Models. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management, CIKM 2024, Boise, ID, USA, October 21-25, 2024*, Edoardo Serra and Francesca Spezzano (Eds.). ACM, 4966–4974. <https://doi.org/10.1145/3627673.3680016>
 - [267] Zhenhailong Wang, Shaoguang Mao, Wenshan Wu, Tao Ge, Furu Wei, and Heng Ji. 2024. Unleashing the Emergent Cognitive Synergy in Large Language Models: A Task-Solving Agent through Multi-Persona Self-Collaboration. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), NAACL 2024, Mexico City, Mexico, June 16-21, 2024*, Kevin Duh, Helena Gómez-Adorno, and Steven Bethard (Eds.). Association for Computational Linguistics, 257–279. <https://doi.org/10.18653/V1/2024.NAACL-LONG.15>
 - [268] Zhitao Wang, Wei Wang, Zirao Li, Long Wang, Can Yi, Xinjie Xu, Luyang Cao, Hanjing Su, Shouzhi Chen, and Jun Zhou. 2024. XUAT-Copilot: Multi-Agent Collaborative System for Automated User Acceptance Testing with Large Language Model. *CoRR* abs/2401.02705 (2024). <https://doi.org/10.48550/ARXIV.2401.02705> arXiv:2401.02705

- [269] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh (Eds.). http://papers.nips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html
- [270] Cheng Wen, Jialun Cao, Jie Su, Zhiwu Xu, Shengchao Qin, Mengda He, Haokun Li, Shing-Chi Cheung, and Cong Tian. 2024. Enchanting Program Specification Synthesis by Large Language Models Using Static Analysis and Program Verification. In *Computer Aided Verification - 36th International Conference, CAV 2024, Montreal, QC, Canada, July 24-27, 2024, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 14682)*, Arie Gurfinkel and Vijay Ganesh (Eds.). Springer, 302–328. https://doi.org/10.1007/978-3-031-65630-9_16
- [271] Hao Wen, Hongming Wang, Jiaxuan Liu, and Yuanchun Li. 2023. DroidBot-GPT: GPT-powered UI Automation for Android. *CoRR* abs/2304.07061 (2023). <https://doi.org/10.48550/ARXIV.2304.07061> arXiv:2304.07061
- [272] Ratnadira Widayarsi, Sheng Qin Sim, Camellia Lok, Haodi Qi, Jack Phan, Qijin Tay, Constance Tan, Fiona Wee, Jodie Ethelda Tan, Yuheng Yieh, Brian Goh, Ferdian Thung, Hong Jin Kang, Thong Hoang, David Lo, and Eng Lieh Ouh. 2020. BugsInPy: A Database of Existing Bugs in Python Programs to Enable Controlled Testing and Debugging Studies. In *ESEC/FSE '20: 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual Event, USA, November 8-13, 2020*, Prem Devanbu, Myra B. Cohen, and Thomas Zimmermann (Eds.). ACM, 1556–1560. <https://doi.org/10.1145/3368089.3417943>
- [273] W. Eric Wong, Ruizhi Gao, Yihao Li, Rui Abreu, and Franz Wotawa. 2016. A Survey on Software Fault Localization. *IEEE Trans. Software Eng.* 42, 8 (2016), 707–740. <https://doi.org/10.1109/TSE.2016.2521368>
- [274] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations. In *First Conference on Language Modeling*. <https://openreview.net/forum?id=BAaY1hNKS>
- [275] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, Rui Zheng, Xiaoran Fan, Xiao Wang, Limao Xiong, Yuhao Zhou, Weiran Wang, Changhao Jiang, Yicheng Zou, Xiangyang Liu, Zhangyue Yin, Shihan Dou, Rongxiang Weng, Wenjuan Qin, Yongyan Zheng, Xipeng Qiu, Xuanjing Huang, Qi Zhang, and Tao Gui. 2025. The Rise and Potential of Large Language Model Based Agents: A Survey. *Sci. China Inf. Sci.* 68, 2 (2025). <https://doi.org/10.1007/S11432-024-4222-0>
- [276] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Demystifying LLM-Based Software Engineering Agents. *Proc. ACM Softw. Eng.* 2, FSE, Article FSE037 (June 2025), 24 pages. <https://doi.org/10.1145/3715754>
- [277] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2024. Fuzz4All: Universal Fuzzing with Large Language Models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14-20, 2024*. ACM, 126:1–126:13. <https://doi.org/10.1145/3597503.3639121>
- [278] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated Program Repair in the Era of Large Pre-trained Language Models. In *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 1482–1494. <https://doi.org/10.1109/ICSE48619.2023.00129>
- [279] Chunqiu Steven Xia and Lingming Zhang. 2022. Less Training, More Repairing Please: Revisiting Automated Program Repair via Zero-shot Learning. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022, Singapore, Singapore, November 14-18, 2022*, Abhik Roychoudhury, Cristian Cadar, and Miryung Kim (Eds.). ACM, 959–971. <https://doi.org/10.1145/3540250.3549101>
- [280] Chunqiu Steven Xia and Lingming Zhang. 2023. Conversational Automated Program Repair. *CoRR* abs/2301.13246 (2023). <https://doi.org/10.48550/ARXIV.2301.13246> arXiv:2301.13246
- [281] Chunqiu Steven Xia and Lingming Zhang. 2024. Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for \$0.42 Each using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024, Vienna, Austria, September 16-20, 2024*, Maria Christakis and Michael Pradel (Eds.). ACM, 819–831. <https://doi.org/10.1145/3650212.3680323>
- [282] Jiahong Xiang, Xiaoyang Xu, Fanchu Kong, Mingyuan Wu, Haotian Zhang, and Yuqun Zhang. 2024. How Far Can We Go with Practical Function-Level Program Repair? *CoRR* abs/2404.12833 (2024). <https://doi.org/10.48550/ARXIV.2404.12833> arXiv:2404.12833
- [283] Binfeng Xu, Xukun Liu, Hua Shen, Zeyu Han, Yuhao Li, Murong Yue, Zhiyuan Peng, Yuchen Liu, Ziyu Yao, and Dongkuan Xu. 2023. Gentopia.AI: A Collaborative Platform for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023 - System Demonstrations, Singapore, December 6-10, 2023*, Yansong Feng and Els Lefever (Eds.). Association for Computational Linguistics, 237–245. <https://doi.org/10.18653/V1/2023.EMNLP-DEMO.20>
- [284] Yiheng Xu, Hongjin Su, Chen Xing, Boyu Mi, Qian Liu, Weijia Shi, Binyuan Hui, Fan Zhou, Yitao Liu, Tianbao Xie, Zhoujun Cheng, Siheng Zhao, Lingpeng Kong, Bailin Wang, Caiming Xiong, and Tao Yu. 2024. Lemur: Harmonizing Natural Language and Code for Language Agents. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. <https://openreview.net/forum?id=hNhwSmtXRh>
- [285] Chen Yang, Junjie Chen, Bin Lin, Jianyi Zhou, and Ziqi Wang. 2024. Enhancing LLM-based Test Generation for Hard-to-Cover Branches via Program Analysis. *CoRR* abs/2404.04966 (2024). <https://doi.org/10.48550/ARXIV.2404.04966> arXiv:2404.04966

- [286] Chenyuan Yang, Yinlin Deng, Runyu Lu, Jiayi Yao, Jiawei Liu, Reyhaneh Jabbarvand, and Lingming Zhang. 2024. WhiteFox: White-Box Compiler Fuzzing Empowered by Large Language Models. *Proc. ACM Program. Lang.* 8, OOPSLA2 (2024), 709–735. <https://doi.org/10.1145/3689736>
- [287] Chenyuan Yang, Zijie Zhao, and Lingming Zhang. 2025. KernelGPT: Enhanced Kernel Fuzzing via Large Language Models. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2025, Rotterdam, Netherlands, 30 March 2025 - 3 April 2025*, Lieven Eeckhout, Georgios Smaragdakis, Katai Liang, Adrian Sampson, Martha A. Kim, and Christopher J. Rossbach (Eds.). ACM, 560–573. <https://doi.org/10.1145/3676641.3716022>
- [288] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang (Eds.). http://papers.nips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html
- [289] John Yang, Akshara Prabhakar, Karthik Narasimhan, and Shunyu Yao. 2023. InterCode: Standardizing and Benchmarking Interactive Coding with Execution Feedback. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (Eds.). http://papers.nips.cc/paper_files/paper/2023/hash/4b175d846fb008d540d233c188379ff9-Abstract-Datasets_and_Benchmarks.html
- [290] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net. https://openreview.net/pdf?id=WE_vluYUL-X
- [291] Burak Yetistiren, Isik Özsoy, Miray Ayerdem, and Eray Tüzün. 2023. Evaluating the Code Quality of AI-Assisted Code Generation Tools: An Empirical Study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT. *CoRR* abs/2304.10778 (2023). <https://doi.org/10.48550/ARXIV.2304.10778> arXiv:2304.10778
- [292] Juyeon Yoon, Robert Feldt, and Shin Yoo. 2024. Intent-Driven Mobile GUI Testing with Autonomous Large Language Model Agents. In *IEEE Conference on Software Testing, Verification and Validation, ICST 2024, Toronto, ON, Canada, May 27-31, 2024*. IEEE, 129–139. <https://doi.org/10.1109/ICST60714.2024.00020>
- [293] Hao Yu, Bo Shen, Dezhi Ran, Jiaxin Zhang, Qi Zhang, Yuchi Ma, Guangtai Liang, Ying Li, Qianxiang Wang, and Tao Xie. 2024. CoderEval: A Benchmark of Pragmatic Code Generation with Generative Pre-trained Models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14-20, 2024*. ACM, 37:1–37:12. <https://doi.org/10.1145/3597503.3623316>
- [294] Zhiqiang Yuan, Weitong Chen, Hanlin Wang, Kai Yu, Xin Peng, and Yiling Lou. 2024. TRANSAGENT: An LLM-Based Multi-Agent System for Code Translation. *CoRR* abs/2409.19894 (2024). <https://doi.org/10.48550/ARXIV.2409.19894> arXiv:2409.19894
- [295] Zhiqiang Yuan, Junwei Liu, Qiancheng Zi, Mingwei Liu, Xin Peng, and Yiling Lou. 2023. Evaluating Instruction-Tuned Large Language Models on Code Comprehension and Generation. *CoRR* abs/2308.01240 (2023). <https://doi.org/10.48550/ARXIV.2308.01240> arXiv:2308.01240
- [296] Zhiqiang Yuan, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, Xin Peng, and Yiling Lou. 2024. Evaluating and Improving ChatGPT for Unit Test Generation. *Proc. ACM Softw. Eng.* 1, FSE (2024), 1703–1726. <https://doi.org/10.1145/3660783>
- [297] Daoguang Zan, Zhirong Huang, Ailun Yu, Shaoxin Lin, Yifan Shi, Wei Liu, Dong Chen, Zongshuai Qi, Hao Yu, Lei Yu, Dezhi Ran, Muhan Zeng, Bo Shen, Pan Bian, Guangtai Liang, Bei Guan, Pengjie Huang, Tao Xie, Yongji Wang, and Qianxiang Wang. 2024. SWE-bench-java: A GitHub Issue Resolving Benchmark for Java. *CoRR* abs/2408.14354 (2024). <https://doi.org/10.48550/ARXIV.2408.14354> arXiv:2408.14354
- [298] Daoguang Zan, Ailun Yu, Wei Liu, Dong Chen, Bo Shen, Wei Li, Yafen Yao, Yongshun Gong, Xiaolin Chen, Bei Guan, Zhiguang Yang, Yongji Wang, Qianxiang Wang, and Lizhen Cui. 2024. CodeS: Natural Language to Code Repository via Multi-Layer Sketch. *CoRR* abs/2403.16443 (2024). <https://doi.org/10.48550/ARXIV.2403.16443> arXiv:2403.16443
- [299] Eric Zelikman, Qian Huang, Gabriel Poesia, Noah D. Goodman, and Nick Haber. 2023. Parsel: Algorithmic Reasoning with Language Models by Composing Decompositions. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (Eds.). http://papers.nips.cc/paper_files/paper/2023/hash/6445dd8ebb9a6a3afa0b126ad87fe41-Abstract-Conference.html
- [300] Andreas Zeller. 2009. *Why Programs Fail - A Guide to Systematic Debugging, 2nd Edition*. Academic Press. <http://store.elsevier.com/product.jsp?isbn=9780123745156&pagename=search>
- [301] Yizhuo Zhai, Yu Hao, Hang Zhang, Daimeng Wang, Chengyu Song, Zhiyun Qian, Mohsen Lesani, Srikanth V. Krishnamurthy, and Paul L. Yu. 2020. UBITest: A Precise and Scalable Method to Detect Use-before-initialization Bugs in Linux Kernel. In *ESEC/FSE '20: 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual Event, USA, November 8-13, 2020*, Prem Devanbu, Myra B. Cohen, and Thomas Zimmermann (Eds.). ACM, 221–232. <https://doi.org/10.1145/3368089.3409686>

- [302] Jianzhang Zhang, Yiyang Chen, Chuang Liu, Nan Niu, and Yinglin Wang. 2023. Empirical Evaluation of ChatGPT on Requirements Information Retrieval Under Zero-shot Setting. In *2023 International Conference on Intelligent Computing and Next Generation Networks (ICNGN)*. IEEE, 1–6.
- [303] Jie M. Zhang, Mark Harman, Lei Ma, and Yang Liu. 2022. Machine Learning Testing: Survey, Landscapes and Horizons. *IEEE Trans. Software Eng.* 48, 2 (2022), 1–36. <https://doi.org/10.1109/TSE.2019.2962027>
- [304] Kechi Zhang, Ge Li, Jia Li, Zhuo Li, and Zhi Jin. 2023. ToolCoder: Teach Code Generation Models to Use API Search Tools. *CoRR* abs/2305.04032 (2023). <https://doi.org/10.48550/ARXIV.2305.04032> arXiv:2305.04032
- [305] Kechi Zhang, Jia Li, Ge Li, Xianjie Shi, and Zhi Jin. 2024. CodeAgent: Enhancing Code Generation with Tool-Integrated Agent Systems for Real-World Repo-level Coding Challenges. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, *ACL 2024, Bangkok, Thailand, August 11-16, 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, 13643–13658. <https://aclanthology.org/2024.acl-long.737>
- [306] Kechi Zhang, Zhuo Li, Jia Li, Ge Li, and Zhi Jin. 2023. Self-Edit: Fault-Aware Code Editor for Code Generation. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, *ACL 2023, Toronto, Canada, July 9-14, 2023*, Anna Rogers, Jordan L. Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, 769–787. <https://doi.org/10.18653/V1/2023.ACL-LONG.45>
- [307] Kexun Zhang, Weiran Yao, Zuxin Liu, Yihao Feng, Zhiwei Liu, Rithesh R. N., Tian Lan, Lei Li, Renze Lou, Jiacheng Xu, Bo Pang, Yingbo Zhou, Shelby Heinecke, Silvio Savarese, Huan Wang, and Caiming Xiong. 2025. Diversity Empowers Intelligence: Integrating Expertise of Software Engineering Agents. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net. <https://openreview.net/forum?id=cKlZKs3Nnb>
- [308] Lingzhe Zhang, Tong Jia, Mengxi Jia, Yifan Wu, Aiwei Liu, Yong Yang, Zhonghai Wu, Xuming Hu, Philip Yu, and Ying Li. 2025. A Survey of AIOps in the Era of Large Language Models. *ACM Comput. Surv.* (June 2025). <https://doi.org/10.1145/3746635> Just Accepted.
- [309] Lyuye Zhang, Kaixuan Li, Kairan Sun, Daoyuan Wu, Ye Liu, Haoye Tian, and Yang Liu. 2025. ACFix: Guiding LLMs with Mined Common RBAC Practices for Context-Aware Repair of Access Control Vulnerabilities in Smart Contracts. *IEEE Transactions on Software Engineering* (2025), 1–21. <https://doi.org/10.1109/TSE.2025.3590108>
- [310] Li Zhang, Jia-Hao Tian, Jing Jiang, Yi-Jun Liu, Meng-Yuan Pu, and Tao Yue. 2018. Empirical Research in Software Engineering - A Literature Survey. *J. Comput. Sci. Technol.* 33, 5 (2018), 876–899. <https://doi.org/10.1007/S11390-018-1864-X>
- [311] Peilun Zhang, Yanjie Jiang, Anjiang Wei, Victoria Stodden, Darko Marinov, and August Shi. 2021. Domain-Specific Fixes for Flaky Tests with Wrong Assumptions on Underdetermined Specifications. In *43rd IEEE/ACM International Conference on Software Engineering, ICSE 2021, Madrid, Spain, 22-30 May 2021*. IEEE, 50–61. <https://doi.org/10.1109/ICSE43902.2021.00018>
- [312] Simiao Zhang, Jiaping Wang, Guoliang Dong, Jun Sun, Yueling Zhang, and Geguang Pu. 2024. Experimenting a New Programming Practice with LLMs. *CoRR* abs/2401.01062 (2024). <https://doi.org/10.48550/ARXIV.2401.01062> arXiv:2401.01062
- [313] Wei Zhang, Hongcheng Guo, Jian Yang, Zhoujin Tian, Yi Zhang, Chaoran Yan, Zhoujun Li, Tongliang Li, Xu Shi, Liangfan Zheng, and Bo Zhang. 2024. mABC: Multi-Agent Blockchain-inspired Collaboration for Root Cause Analysis in Micro-Services Architecture. In *Findings of the Association for Computational Linguistics: EMNLP 2024, Miami, Florida, USA, November 12-16, 2024*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, 4017–4033. <https://aclanthology.org/2024.findings-emnlp.232>
- [314] Xiaoyi Zhang, Lilian de Greef, Amanda Swearngin, Samuel White, Kyle I. Murray, Lisa Yu, Qi Shan, Jeffrey Nichols, Jason Wu, Chris Fleizach, Aaron Everitt, and Jeffrey P. Bigham. 2021. Screen Recognition: Creating Accessibility Metadata for Mobile Applications from Pixels. In *CHI '21: CHI Conference on Human Factors in Computing Systems, Virtual Event / Yokohama, Japan, May 8-13, 2021*, Yoshifumi Kitamura, Aaron Quigley, Katherine Isbister, Takeo Igarashi, Pernille Bjørn, and Steven Mark Drucker (Eds.). ACM, 275:1–275:15. <https://doi.org/10.1145/3411764.3445186>
- [315] Yue Zhang, Yafu Li, Leyang Cui, Deng Cai, Lemao Liu, Tingchen Fu, Xinting Huang, Enbo Zhao, Yu Zhang, Yulong Chen, Longyue Wang, Anh Tuan Luu, Wei Bi, Freda Shi, and Shuming Shi. 2023. Siren’s Song in the AI Ocean: A Survey on Hallucination in Large Language Models. *CoRR* abs/2309.01219 (2023). <https://doi.org/10.48550/ARXIV.2309.01219> arXiv:2309.01219
- [316] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. AutoCodeRover: Autonomous Program Improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024, Vienna, Austria, September 16-20, 2024*, Maria Christakis and Michael Pradel (Eds.). ACM, 1592–1604. <https://doi.org/10.1145/3650212.3680384>
- [317] Zeyu Zhang, Quanyu Dai, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. 2025. A Survey on the Memory Mechanism of Large Language Model based Agents. *ACM Trans. Inf. Syst.* (July 2025). <https://doi.org/10.1145/3748302> Just Accepted.
- [318] Zhuosheng Zhang, Aston Zhang, Mu Li, and Alex Smola. 2023. Automatic Chain of Thought Prompting in Large Language Models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net. <https://openreview.net/forum?id=5NTt8GFjUHkr>
- [319] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. 2023. A Survey of Large Language Models. *CoRR* abs/2303.18223 (2023). <https://doi.org/10.48550/ARXIV.2303.18223>

- arXiv:2303.18223
- [320] Li Zhong, Zilong Wang, and Jingbo Shang. 2024. Debug like a Human: A Large Language Model Debugger via Verifying Runtime Execution Step by Step. In *Findings of the Association for Computational Linguistics, ACL 2024, Bangkok, Thailand and virtual meeting, August 11-16, 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, 851–870. <https://doi.org/10.18653/V1/2024.FINDINGS-ACL.49>
 - [321] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. 2024. Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net. <https://openreview.net/forum?id=njwv9BsGHF>
 - [322] Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc V. Le, and Ed H. Chi. 2023. Least-to-Most Prompting Enables Complex Reasoning in Large Language Models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net. <https://openreview.net/forum?id=WZH7099tgfM>
 - [323] Xuanhe Zhou, Guoliang Li, Zhaoyan Sun, Zhiyuan Liu, Weize Chen, Jianming Wu, Jiesi Liu, Ruohang Feng, and Guoyang Zeng. 2024. D-Bot: Database Diagnosis System using Large Language Models. *Proc. VLDB Endow.* 17, 10 (2024), 2514–2527. <https://doi.org/10.14778/3675034.3675043>
 - [324] Zhenhao Zhou, Zhuochen Huang, Yike He, Chong Wang, Jiajun Wang, Yijian Wu, Xin Peng, and Yiling Lou. 2025. Benchmarking and Enhancing LLM Agents in Localizing Linux Kernel Bugs. *CoRR* abs/2505.19489 (2025). <https://doi.org/10.48550/ARXIV.2505.19489> arXiv:2505.19489
 - [325] Wei Liu Zihao Xie YiFei Wang Rennai Qiu Yufan Dang Weize Chen Cheng Yang Ye Tian Xuantang Xiong Lei Han Zhuoyun Du, Chen Qian. 2025. Multi-Agent Collaboration via Cross-Team Orchestration. In *The Annual Meeting of the Association for Computational Linguistics (ACL)*.

Received 25 September 2024; revised 17 September 2025; accepted 31 October 2025